Technische Universität Berlin

Big Data Engineering (DAMS)
Fakultät IV
Franklinstraße 28
10587 Berlin
https://www.tu.berlin/dams

Master's Thesis

Workload-aware and Reconfigurable LLM Serving

René Richard Enjilian

Matriculation Number: 382229
15.06.2026

Supervised by
Prof. Dr. Matthias Boehm
Prof. Dr. Volker Markl

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generische KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Nutzung von KI-Tools: Zur sprachlichen Überprüfung und Verbesserung wurde auf das KI-Modell GPT-5 des Herstellers OpenAI (Produktname: ChatGPT) zurückgegriffen.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 8. März 2017. https://www.static.tu.berlin/fileadmin/www/10000060/FSC/Promotion___Habilitation/Dokumente/Grundsaetze_gute_wissenschaftliche_Praxis_2017.pdf habe ich zur Kenntnis genommen. Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, 15.06.2026

.....
(Signature)

Abstract

Large language models (LLMs) are increasingly deployed as interactive inference services, where users expect low response latency while providers must operate under limited GPU memory and compute resources. Serving such models is challenging because different model variants expose different trade-offs between output quality, latency, throughput, and memory footprint. Static serving configurations choose one model and one placement strategy ahead of time, even though request workloads, serving objectives, and resource pressure may change during operation.

Addressing these challenges matters because efficient LLM serving is not only a matter of faster inference kernels or larger hardware. A serving system must decide which model should answer requests, where model weights should reside, and when adaptation can occur without disrupting active serving. Cheaper or quantized models may improve latency and throughput, but only if they still satisfy quality constraints. Similarly, moving models across GPU memory, CPU memory, and disk can improve resource usage, but these transitions introduce their own latency and interference costs.

This thesis presents Kairos, a workload-aware and reconfigurable LLM serving system. Built on top of vLLM, Kairos continuously samples the live request stream, evaluates candidate models on observed workload data, and uses the resulting statistics to adapt the serving configuration at runtime. The system combines continuous monitoring, runtime reconfigurability, and interference-aware scheduling. Kairos can switch the actively serving model, move model weights across memory tiers, and delay disruptive reconfiguration commands until the workload is predicted to provide a sufficiently idle window.

Our results show that workload-aware reconfiguration is a viable direction for LLM serving. Kairos demonstrates that live request monitoring, runtime model evaluation, model movement across memory tiers, and concurrent request dispatching can support adaptive serving decisions. By switching to a more efficient model variant at runtime, the system can reduce request latency while preserving the configured quality constraints. The results also show that reconfiguration must be scheduled carefully, because model movement and model switching can compete with active inference for GPU resources. The timing of reconfiguration therefore directly affects whether adaptation improves serving behavior or introduces additional interference.

Zusammenfassung

Große Sprachmodelle (Large Language Models, LLMs) werden zunehmend als interaktive Inferenzdienste eingesetzt, bei denen Nutzer niedrige Antwortlatenzen erwarten, während Anbieter mit begrenztem GPU-Speicher und begrenzten Rechenressourcen arbeiten müssen. Die Bereitstellung solcher Modelle ist herausfordernd, da verschiedene Modellvarianten unterschiedliche Kompromisse zwischen Ausgabequalität, Latenz, Durchsatz und Speicherbedarf aufweisen. Statische Serving-Konfigurationen wählen ein Modell und eine Platzierungsstrategie im Voraus aus, obwohl sich Anfrage-Workloads, Serving-Ziele und Ressourcendruck während des Betriebs verändern können.

Die Bewältigung dieser Herausforderungen ist von Relevanz, da effizientes LLM-Serving nicht nur eine Frage schnellerer Inferenz-Kernel oder besserer Hardware ist. Ein Serving-System muss entscheiden, welches Modell Anfragen beantworten soll, wo Modellparameter gespeichert werden sollen und wann Anpassungen möglich sind, ohne die aktive Anfragebearbeitung zu stören. Günstigere oder quantisierte Modelle können Latenz und Durchsatz verbessern, jedoch nur, wenn sie weiterhin die Qualitätsanforderungen erfüllen. Ebenso kann das Verschieben von Modellen zwischen GPU-Speicher, CPU-Speicher und Festplatte die Ressourcennutzung verbessern, führt aber eigene Latenz- und Interferenzkosten ein.

Diese Arbeit präsentiert das LLM Serving-System Kairos. Kairos ist in der Lage Anfragen unter Beachtung der Auslastung zu bearbeiten und kann während der Nutzung rekonfiguriert werden. Kairos baut auf vLLM auf, erfasst kontinuierlich Stichproben aus dem laufenden Anfragestrom, evaluiert Kandidatenmodelle auf beobachteten Anfragedaten und nutzt die daraus gewonnenen Statistiken, um die Serving-Konfiguration während des Betriebs anzupassen. Das System kombiniert kontinuierliches Monitoring, Rekonfigurierbarkeit und interferenzbewusste Scheduling-Entscheidungen. Kairos kann das aktiv zur Anfragebearbeitung eingesetzte Modell austauschen, Modellparameter zwischen Speicherebenen verschieben und unterbrechende Rekonfigurationsbefehle verzögern, bis der Workload ein voraussichtlich ausreichend großes Leerlaufintervall bietet.

Unsere Ergebnisse zeigen, dass Rekonfiguration unter Berücksichtigung der Anfragen eine vielversprechende Richtung für LLM-Serving darstellt. Kairos demonstriert, dass Live-Monitoring von Anfragen, Modellevaluation während des Betriebs, Verschiebung von Modellen zwischen Speicherebenen und nebenläufige Anfrageweiterleitung adaptive Serving-Entscheidungen unterstützen können. Durch den Wechsel zu einer effizienteren Modellvariante während des laufenden Betriebs kann das System die Anfraglatenz reduzieren und gleichzeitig die konfigurierten Qualitätsanforderungen einhalten. Die Ergebnisse zeigen außerdem, dass Rekonfiguration sorgfältig geplant werden muss, da das Verschieben und Austauschen von Modellen mit aktiver Inferenz um GPU-Ressourcen konkurrieren kann. Der Zeitpunkt der Rekonfiguration beeinflusst daher direkt, ob Adaption das Serving-Verhalten verbessert oder zusätzliche Unterbrechungen verursacht.

Contents

1 Introduction

LLMs have become a central technology for natural language processing and are increasingly deployed as interactive inference services. Users submit prompts, the serving system executes the model, and the model generates responses token by token. This usage pattern differs from offline model evaluation because requests arrive continuously, users expect low response latency, and the system must serve many requests under limited hardware resources. Efficient LLM serving has therefore become an important systems problem.

Serving LLMs is expensive because the models require substantial GPU memory and compute during inference. Large model weights occupy a significant fraction of accelerator memory, while autoregressive generation repeatedly executes the model to produce output tokens. At the same time, different models and model variants expose different trade-offs between latency, throughput, memory footprint, and output quality. A larger model may provide stronger predictions but consume more memory and reduce throughput, whereas a smaller or quantized model may serve requests more efficiently but produce lower-quality outputs. The serving system therefore has to operate under competing objectives rather than optimizing a single metric in isolation.

In practice, these trade-offs are not fixed once and for all. The best serving configuration can depend on the current workload, the request rate, the available memory, and the service-level objectives of the application. A configuration that works well under low load may become inefficient during bursts, while a configuration optimized for throughput may violate latency constraints when requests accumulate. Static serving deployments cannot react to such changes after startup. They select a model and runtime configuration in advance, even though the conditions under which the system operates may change during serving.

Existing LLM inference systems address important parts of this problem. Systems such as Orca [65] and vLLM [32] improve serving efficiency through continuous batching, iteration-level scheduling, and more efficient KV-cache management. Other approaches reduce the cost of serving through quantization, which lowers the memory footprint and can improve throughput. These techniques substantially improve the efficiency of individual serving configurations. However, they do not by themselves decide when a serving system should switch between different model variants, move models across memory tiers, or adapt the active configuration based on observed workload behavior and service-level objectives.

This thesis presents Kairos, a workload-aware and reconfigurable serving system for LLMs. Kairos treats LLM serving as a continuously managed process rather than as a fixed deployment decision. The system monitors incoming requests, evaluates model behavior on the observed workload, and uses this information to adapt the serving con-

figuration at runtime. To make such adaptation practical, Kairos combines three design principles: continuous monitoring, runtime reconfigurability, and interference-aware scheduling. Together, these principles allow the system to reason not only about which model configuration is beneficial, but also about when adaptation can be performed with limited interference to active request serving.

The contributions of this thesis are as follows:

- We design Kairos, a workload-aware and reconfigurable LLM serving system that adapts model selection and placement decisions during runtime.
- We extend vLLM with additional model-management mechanisms that support model weight movement across GPU memory, CPU memory, and disk storage.
- We implement the core system components required for adaptive serving, including monitoring, scheduling, reconfiguration, request dispatching, and a client for generating controlled workloads.
- We evaluate Kairos through component-level microbenchmarks and end-to-end experiments, analyzing both the costs of individual mechanisms and the behavior of the complete serving system.

The remainder of this thesis is organized as follows:

- Chapter 2 introduces the foundations of LLM serving, including transformer models, inference systems, and quantization.
- Chapter 3 presents the design and architecture of Kairos.
- Chapter 4 evaluates Kairos experimentally. We first analyze individual mechanisms and validate the main design decisions through microbenchmarks, before evaluating Kairos as a complete serving system under different workloads.
- Chapter 5 discusses related work on LLM inference systems, adaptive serving, model selection, and runtime optimization, and positions Kairos with respect to existing approaches.
- Chapter 6 summarizes the findings and discusses directions for future work.

2 Foundations of LLM Serving

This chapter introduces the technical foundations needed to understand the design and implementation of LLM serving systems. We begin with the transformer architecture, which forms the basis of modern generative LLMs. The discussion covers the representation of input tokens, self-attention, transformer layers, positional encoding, decoder-only transformers, and the connection from these architectures to LLMs.

We then turn from model architecture to inference systems. This part explains how LLMs are executed during serving, including the distinction between prefill and decode, the role of the KV cache, batching, parallelism, and the serving objectives that determine system performance. Finally, we discuss quantization as an important technique for reducing the memory and computational cost of LLMs. The chapter introduces common quantization schemes such as INT8, INT4, FP8, and NF4, which are relevant for understanding how different model variants trade off efficiency and output quality.

2.1 Transformer

The transformer architecture, introduced by Vaswani et al. [58], forms the architectural basis of modern generative language models. This section introduces the main components needed for the remainder of this thesis. We first describe how tokens are represented as vectors, then explain scaled self-attention, multi-head self-attention, and the structure of a transformer layer. We then discuss positional encoding, which supplies order information to an otherwise permutation-equivariant architecture. Finally, we focus on decoder-only transformers and show how they lead to modern LLMs. For a more detailed treatment of transformers and neural language models, we refer to Bishop and Bishop [5].

2.1.1 Input Representation

The input to a transformer is a sequence of vectors $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ with $\mathbf{x}_n \in \mathbb{R}^D$ for all $n \in \{1, \ldots, N\}$. We refer to these vectors as *tokens* and to the elements x_{ni}, $i \in \{1, \ldots, D\}$, as *features*. We combine the tokens into a matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$, where the nth row represents the token $\mathbf{x}_n^\top$, as illustrated in Figure 2.1. Depending on the application, a token may correspond to a word or sequence of characters in a sentence, an image patch, or an amino acid in a biological sequence. Note that $\mathbf{X}$ represents a single token sequence, whereas most applications require data sets containing many such sequences.

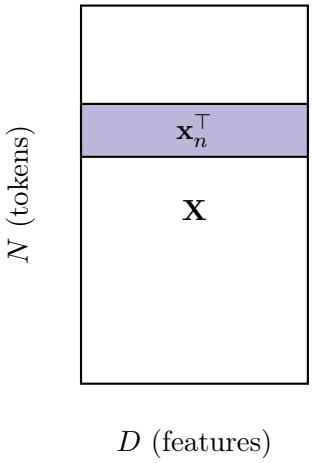

Figure 2.1: Matrix representation of a token sequence.

2.1.2 Scaled Self-Attention

Self-attention is the central mechanism that allows a transformer to exchange information between tokens in a sequence. At a high level, it first determines which tokens are relevant to one another and then uses this information to update each token representation with contextual information from the sequence. Scaled self-attention defines the basic computational form of this mechanism and forms the core operation inside transformer layers.

Attention coefficients

Assume a sequence of input tokens $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ in an embedding space. Attention maps this sequence to an output sequence $(\mathbf{y}_1, \ldots, \mathbf{y}_N)$, where $\mathbf{y}_n \in \mathbb{R}^D$ for all $n \in \{1, \ldots, N\}$, capturing richer semantic representations. Now consider a particular output embedding $\mathbf{y}_n$. The computation of $\mathbf{y}_n$ should not only depend on the corresponding input token $\mathbf{x}_n$, but on all input tokens $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$. However, this dependence should not be uniform across all input tokens, but stronger for those tokens that are particularly relevant for computing $\mathbf{y}_n$. We can achieve this property by computing $\mathbf{y}_n$ as a weighted linear combination of the input tokens $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$:

$$\mathbf{y}_n = \sum_{m=1}^{N} a_{nm}\, \mathbf{x}_m \tag{2.1}$$

where a_{nm} are called *attention weights*. The weights must fulfill the following constraints:

$$a_{nm} \geq 0 \tag{2.2}$$

$$\sum_{m=1}^{N} a_{nm} = 1. \tag{2.3}$$

From Equations (2.2) and (2.3) it follows that $0 \leq a_{nm} \leq 1$. Thus, attention weights tend toward zero for input tokens with little impact on $\mathbf{y}_n$ and are largest for those with the greatest influence. Moreover, the two constraints together ensure that if $\mathbf{y}_n$ assigns more attention to a particular input token, the increased attention to that token comes at the expense of the others. For example, the special case $a_{nm} = 1$ implies that $a_{nm} = 0$ for $n \neq m$, and therefore $\mathbf{y}_m = \mathbf{x}_m$.

Self-attention

In the following, we discuss the computation of the attention coefficients a_{nm}. Consider the token sequence $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$. For each token $\mathbf{x}_n$, we want to determine how strongly it should attend to every other input token $\mathbf{x}_m$, $m \in \{1, \ldots, N\}$. For this computation, attention assigns three distinct roles:

- A *query* vector $\mathbf{q}_n$ for the input token currently being updated.
- A *key* vector $\mathbf{k}_m$ for each token that $\mathbf{q}_n$ can attend to.
- A *value* vector $\mathbf{v}_m$ representing the information each key carries.

To compute a_{nm}, we first define a similarity score s_{nm} between the token at position n and the token at position m in the sequence. A common choice is the dot product:

$$s_{nm} = \mathbf{q}_n^\top \mathbf{k}_m. \tag{2.4}$$

To satisfy Equations (2.2) and (2.3), we define a_{nm} as s_{nm} normalized by the *softmax* function:

$$a_{nm} = \frac{\exp(\mathbf{q}_n^\top \mathbf{k}_m)}{\sum_{m'=1}^{N} \exp(\mathbf{q}_n^\top \mathbf{k}_{m'})}. \tag{2.5}$$

We call this process *self-attention* because the token vectors themselves serve as queries, keys, and values, i.e., $\mathbf{q}_n = \mathbf{x}_n$, $\mathbf{k}_m = \mathbf{x}_m$, and $\mathbf{v}_m = \mathbf{x}_m$. Note that the variable $\mathbf{x}_m$ in Equation (2.1) corresponds to $\mathbf{v}_m$. We summarize this process in Figure 2.2 by illustrating the computation of the output embedding $\mathbf{y}_1$. The corresponding input token $\mathbf{x}_1$ represents the query. Self-attention computes the attention weights $(a_{11}, a_{12}, \ldots, a_{1N})$ between the query and the keys according to Equation (2.5). It then computes the products between weights and values, which are identical to the keys. In the final step, it sums these products to obtain $\mathbf{y}_1$.
We can write Equation (2.1) in matrix notation by using the data matrix $\mathbf{X}$ as defined in Figure 2.1, together with the analogously defined output matrix $\mathbf{Y} \in \mathbb{R}^{N \times D}$, where the nth row represents the output embedding $\mathbf{y}_n^\top$:

$$\mathbf{Y} = \mathrm{Softmax}\big[\mathbf{X}\mathbf{X}^\top\big]\mathbf{X} \tag{2.6}$$

where $\mathrm{Softmax}\big[\cdot\big]$ takes the exponential of every element and then normalizes each row independently to sum to one.

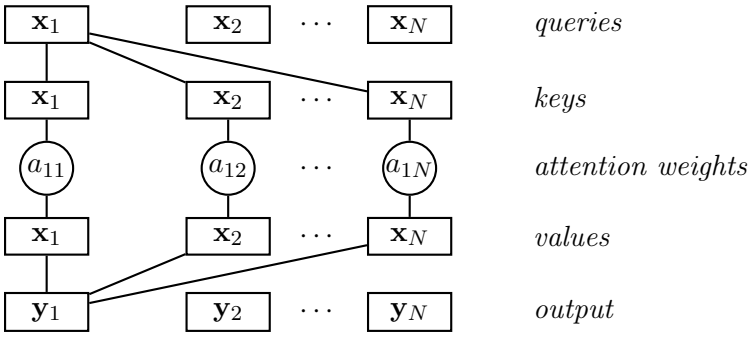

Figure 2.2: Self-attention computation for output embedding $\mathbf{y}_1$.

Model parameters

So far, the transformation from the input token sequence $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ to the output embeddings $(\mathbf{y}_1, \ldots, \mathbf{y}_N)$ is fixed and cannot learn from data. Another limitation is that each feature x_{ni} of a token $\mathbf{x}_n$ contributes equally to computing a_{nm}, which limits the model's flexibility when determining token similarity. To mitigate these issues, we redefine the token matrix $\mathbf{X}$ as

$$\widetilde{\mathbf{X}} = \mathbf{X}\mathbf{U} \tag{2.7}$$

where $\mathbf{U} \in \mathbb{R}^{D \times D}$ is a matrix of learnable parameters. Substituting Equation(2.7) into Equation (2.6) yields

$$\mathbf{Y} = \mathrm{Softmax}\big[\mathbf{X}\mathbf{U}\mathbf{U}^\top\mathbf{X}^\top\big]\mathbf{X}\mathbf{U}. \tag{2.8}$$

While Equation (2.8) has more flexibility than the static formulation in Equation (2.6), it is still constrained by the attention coefficients $\mathbf{X}\mathbf{U}\mathbf{U}^\top\mathbf{X}^\top$ and the value vectors $\mathbf{X}\mathbf{U}$ sharing the same parameter matrix $\mathbf{U}$. Accordingly, we define separate weight matrices for queries, keys, and values, resulting in distinct linear transformations:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(\mathrm{q})} \tag{2.9}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^{(\mathrm{k})} \tag{2.10}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^{(\mathrm{v})} \tag{2.11}$$

where $\mathbf{W}^{(\mathrm{q})}, \mathbf{W}^{(\mathrm{k})}, \mathbf{W}^{(\mathrm{v})} \in \mathbb{R}^{D \times D}$ represent independent parameter matrices, and $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times D}$ are the matrices (i.e., linear transformations) for queries, keys, and values, respectively. Substituting Equations (2.9), (2.10), and (2.11) into Equation (2.8) gives

$$\mathbf{Y} = \mathrm{Softmax}\big[\mathbf{Q}\mathbf{K}^\top\big]\mathbf{V}. \tag{2.12}$$

Note that we treat the bias parameters as implicit to avoid cluttering the notation. In practice, the bias terms can be absorbed by adding an additional column of ones to $\mathbf{X}$ and an additional row to the weight matrices.

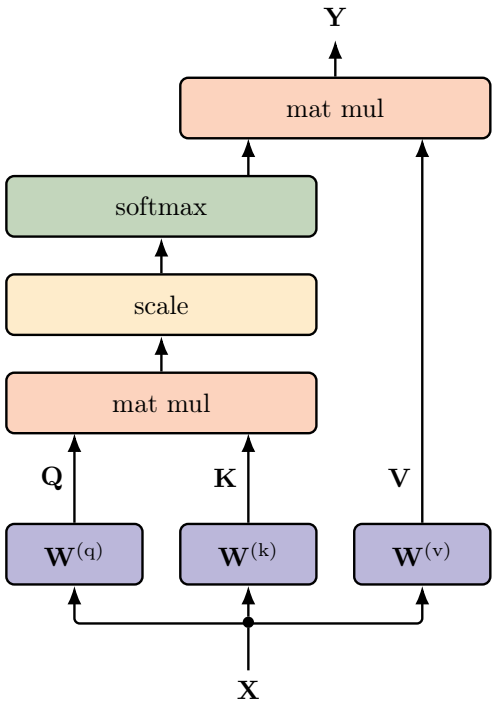

Figure 2.3: Single attention head implementing scaled self-attention.

Normalization

When training the model with attention as described in Equation (2.12), the gradients of the softmax function become exponentially small for large inputs. To mitigate this problem, we normalize $\mathbf{Q}\mathbf{K}^\top$ by $\sqrt{D_\mathrm{k}}$, where D_k is the number of features in $\mathbf{K}$. Recall that we assume $D_\mathrm{k} = D$. We opt for this quantity because if the queries and keys are roughly independent with zero mean and unit variance, their dot product has variance proportional to D_k. Thus, dividing by $\sqrt{D_\mathrm{k}}$ prevents the dot products from growing with dimension and keeps the softmax inputs at a comparable scale:

$$\mathbf{Y} = \mathrm{Attention}\big(\mathbf{Q}, \mathbf{K}, \mathbf{V}\big) \equiv \mathrm{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D_\mathrm{k}}}\right]\mathbf{V}. \tag{2.13}$$

Equation (2.13) represents the final form of the attention computation in the transformer architecture. We refer to the operation as *scaled self-attention* to emphasize the scaling factor introduced in the computation. Figure 2.3 illustrates the corresponding operators and data flow. A single application of this scaled self-attention mechanism constitutes a single *attention head*.

2.1.3 Multi-Head Self-Attention

The transformer architecture utilizes multiple attention heads in parallel to learn different underlying patterns in the input data at the same time. For instance, one head may

focus on grammatical relationships while another groups words into phrases. By having multiple attention heads, the model does not have to choose a single way to attend the input. It rather learns several complementary patterns in parallel and combines them into a semantically richer representation. These heads are structurally identical but do not share parameters. Instead, each head learns independently the weights for computing its query, key, and value matrices.

Assume we have H heads with index $h \in \{1, \ldots, H\}$ of the form

$$\mathbf{H}_h = \text{Attention}\big(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h\big) \tag{2.14}$$

where $\text{Attention}\big(\cdot, \cdot, \cdot\big)$ follows Equation (2.13) and with separately defined query, key, and value matrices for each head:

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(\text{q})} \tag{2.15}$$

$$\mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(\text{k})} \tag{2.16}$$

$$\mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(\text{v})}. \tag{2.17}$$

We concatenate the heads into a single matrix, which has a separate learnable weight matrix $\mathbf{W}^{(\text{o})} \in \mathbb{R}^{HD \times N}$. The resulting combined output is then of the form

$$\mathbf{Y}(\mathbf{X}) = \text{Concat}\big[\mathbf{H}_1, \ldots, \mathbf{H}_H\big]\mathbf{W}^{(\text{o})} \tag{2.18}$$

where $\mathbf{Y}(\mathbf{X}) \in \mathbb{R}^{N \times D}$ is the combined output of the H attention heads. Thus, the output matrix of multi-head self-attention (MHSA) has the dimensionality as the one of a single attention head. Figure 2.4 illustrates the operators and data flow in multi-head attention.

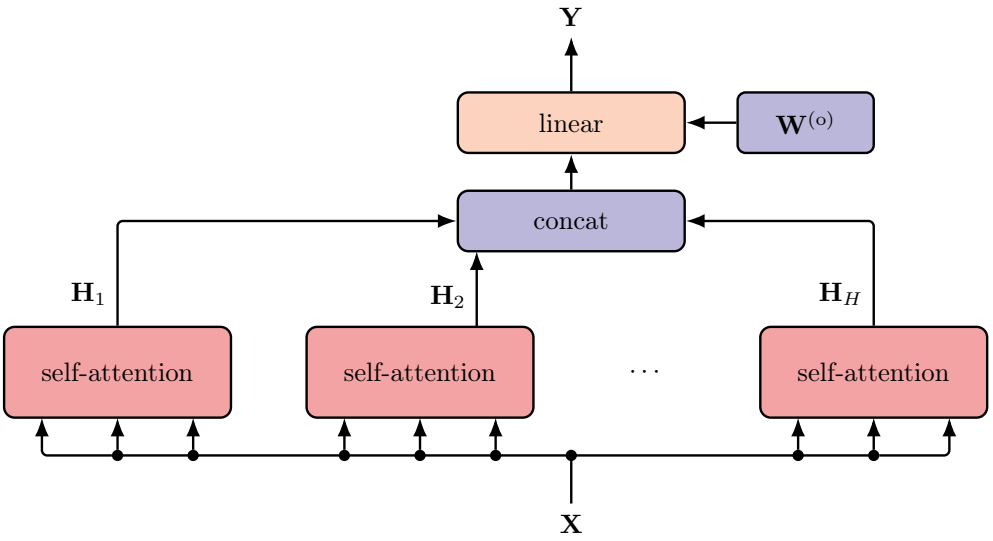

Figure 2.4: Multi-Head Attention.

2.1.4 Transformer Layer

Increasing the depth of a neural network increases its representational capacity by enabling the composition of simple transformations into more complex functions [41, 57]. Since MHSA represents the core architectural element in a transformer, we would like to stack multiple self-attention layers on top of each other. However, in practice, a transformer layer augments MHSA with additional operations that improve training stability and increase the expressiveness of the resulting token representations. The transformer layer incorporates residual connections to preserve the existing token representations while allowing the layer to add new contextual information. Given an input representation $\mathbf{X} \in \mathbb{R}^{N \times D}$, the MHSA block computes $\mathbf{Y}(\mathbf{X}) \in \mathbb{R}^{N \times D}$ as in Equation 2.18. Rather than treating $\mathbf{Y}(\mathbf{X})$ as the new representation directly, the transformer adds it to the original representation:

$$\mathbf{X}^{\text{attn}} = \mathbf{Y}(\mathbf{X}) + \mathbf{X}. \tag{2.19}$$

The residual connection preserves the existing token representation and lets the attention block contribute an additive update. Thus, the attention block does not need to build the next representation from scratch. It only needs to learn which information to add, remove, or adjust. This formulation makes deep networks easier to optimize, keeps information from earlier layers accessible, and provides a direct path for gradients through the model. Afterwards the transformer performs *layer normalization* [4] to stabilize training by normalizing each token across its feature dimension:

$$\mathbf{Z} = \text{LayerNorm}\big[\mathbf{Y}(\mathbf{X}) + \mathbf{X}\big] \tag{2.20}$$

where $\mathbf{Z} \in \mathbb{R}^{N \times D}$. Many modern architectures apply *pre-normalization* [64], where normalization takes place before MHSA leading to better-behaved gradients. In this case, we rewrite Equation 2.20 such that

$$\mathbf{Z} = \mathbf{Y}(\mathbf{X}') + \mathbf{X}, \qquad \mathbf{X}' = \text{LayerNorm}\big[\mathbf{X}\big]. \tag{2.21}$$

A great limitation of attention is that the output vectors are constrained to lie in the subspace spanned by the input vectors limiting the transfomer layer's expressiveness. We introduce a multilayer perceptron (MLP) to mitigate the aforementioned limitation. The MLP might consist of a two-layer feedforward neural network with ReLU activations. The transformer layer applies the MLP to each output vector (i.e., rows of $\mathbf{Z}$) to preserve the transformer's ability to process sequences of variable length. As with MHSA, we apply a residual connection and layer normalization to the MLP. Thus, the final output of the transformer layer has the form

$$\widetilde{\mathbf{X}} = \text{LayerNorm}\big[\text{MLP}\big[\mathbf{Z}\big] + \mathbf{Z}\big] \tag{2.22}$$

where $\widetilde{\mathbf{X}} \in \mathbb{R}^{N \times D}$. Figure 2.5 shows the overall architecture of a transformer layer. In general, a transformer network stacks multiple such layers on top of each other. These layers share the same architecture but each with separately learnable parameters.

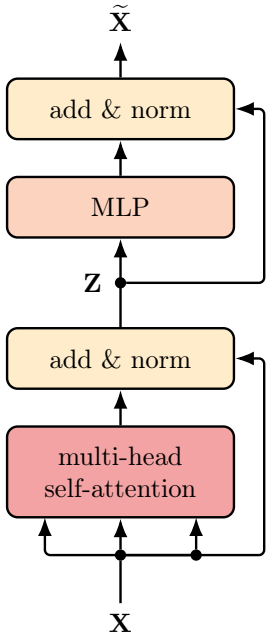

Figure 2.5: Transformer Layer.

2.1.5 Positional Encoding

The transformer architecture is permutation-equivariant by design. The query, key, and value projections, as well as the subsequent feed-forward layers, apply the same transformations to every token. Therefore, reordering the rows of the input matrix also reorders the rows of the output matrix in the same way, without changing the computation applied to each token. This property supports two central strengths of transformers: efficient parallel computation and the ability to model dependencies between distant tokens as directly as dependencies between neighbouring tokens. For sequential data, however, the same property creates a fundamental limitation. In natural language, word order often determines meaning. For example, the sentences "The dog chased the cat" and "The cat chased the dog" contain the same words, but describe different events. A model that processes language must therefore account for the positions of tokens in the sequence.

Instead of modifying the attention mechanism itself, the standard solution encodes order directly in the input representations. For each position n, the model uses a positional vector $\mathbf{r}_n$ and combines it with the corresponding token embedding $\mathbf{x}_n$. One possible approach would concatenate $\mathbf{r}_n$ and $\mathbf{x}_n$. However, concatenation increases the dimensionality of every representation passed to the subsequent layers and therefore, raises the computational cost throughout the network. The simpler approach adds both vectors:

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n. \tag{2.23}$$

The addition preserves the dimensionality of the representation as long as $\mathbf{r}_n$ and $\mathbf{x}_n$

have the same size.

At first glance, adding positional information directly to the token embedding may seem problematic, because it appears to mix two different types of information in the same vector. In practice, this addition does not destroy the token representation. In high-dimensional spaces, unrelated vectors tend to point in nearly orthogonal directions, which helps the network distinguish token identity from positional information. Residual connections further help preserve the positional signal as representations pass through the stack, while the learned linear transformations inside each transformer block can extract and recombine the token and position information needed by subsequent layers.

A useful positional encoding should satisfy several requirements at once. First, each position should receive a distinct signature. Second, the encoding values should remain within a controlled range, so that positional information does not dominate the token embeddings. Third, the construction should extend naturally to sequences longer than those observed during training. Finally, the encoding should represent relative offsets in a uniform way, such that the relation between two positions depends on their distance rather than on their absolute location in the sequence. This property matters because, in many sequence-processing tasks, the distance between tokens carries more information than their absolute positions.

Two simple schemes show how easily these requirements can fail. Assigning positions the values $1, 2, 3, \ldots$ gives each position a unique identifier, but the values grow without bound and can overpower the token embeddings. The unbounded growth also complicates generalization to longer sequences, since positions beyond the training range produce larger values. Mapping positions to the interval $(0, 1)$, for example by dividing each index by the sequence length, keeps the magnitudes controlled. However, this idea destroys uniqueness across sequences of different lengths, because the value assigned to a position then depends on the total sequence length rather than only on the position itself.

The original transformer introduced a sinusoidal construction that addresses these requirements [58]. For position n, the i-th component of $\mathbf{r}_n$ is defined as

$$r_{n,i} = \begin{cases} \sin\left(\dfrac{n}{L^{\,i/D}}\right), & \text{if } i \text{ is even,} \\ \cos\left(\dfrac{n}{L^{\,(i-1)/D}}\right), & \text{if } i \text{ is odd,} \end{cases} \tag{2.24}$$

where D denotes the dimensionality of the $\mathbf{x}_n$ and L controls the range of wavelengths.

This construction makes the entries of r_n follow a family of sine and cosine waves with different frequencies. Rapidly oscillating components capture fine differences between nearby positions, while slowly oscillating components distinguish broader regions of the sequence. Since sine and cosine values always lie in the interval $[-1, 1]$, the encoding remains bounded independently of the sequence length. At the same time, a shift from one position to another corresponds to a phase shift in each sinusoidal component, which gives the model a consistent way to represent relative distances between tokens.

Sinusoidal positional encodings therefore provide a simple and fixed way to inject order information into the input representations, thereby addressing the transformer's lack of inherent sequence order without changing the attention mechanism itself.

2.1.6 Decoder-Only Transformer

Transformers support several forms of language processing, depending on how the input and output sequences are structured. Encoder-only models process an input sequence and produce a representation that can support tasks such as classification [10, 15, 36]. Encoder-decoder models map one sequence to another and therefore suit tasks such as machine translation [33, 51, 58]. Decoder-only models, in contrast, generate text autoregressively by predicting the next token from the preceding tokens [7, 49, 50]. Since this thesis focuses on serving modern generative LLMs, the following discussion concentrates on decoder-only transformers, which form the dominant architecture for this class of models.

Decoder-only transformers can serve as generative models that produce sequences of tokens. Prominent examples include GPT-style models, where GPT stands for Generative Pre-trained Transformer [7, 45]. These models use the transformer architecture to define an autoregressive language model. Given a token sequence $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$, the model factorizes the joint probability of the sequence as

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \prod_{n=1}^{N} p(\mathbf{x}_n \mid \mathbf{x}_1, \ldots, \mathbf{x}_{n-1}). \tag{2.25}$$

Each conditional distribution predicts the next token from the tokens that precede it.

Architecturally, a decoder-only transformer consists of a stack of transformer layers that takes an input sequence $(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ and produces a contextualized representation for each position. Let $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times D}$ denote the matrix of output representations, where D is the hidden dimensionality. To obtain a probability distribution over the vocabulary at each position, the model applies a learned linear projection $\mathbf{W}^{(\mathrm{p})} \in \mathbb{R}^{D \times K}$, where K is the vocabulary size, followed by a softmax function:

$$\mathbf{Y} = \mathrm{Softmax}\big(\tilde{\mathbf{X}}\mathbf{W}^{(\mathrm{p})}\big). \tag{2.26}$$

Each row of $\mathbf{Y}$ then represents a probability distribution over the possible next tokens.

The model learns from unlabelled text through a self-supervised next-token prediction objective. Given a token sequence, each position predicts the token that follows it. Instead of treating every sequence prefix as a separate training example, the model processes an entire sequence in parallel. To do so, the input sequence is shifted to the right by one position and a special start token is prepended. As a result, the representation at position n predicts the target token at position $n + 1$.

This setup requires a mechanism that prevents the model from accessing future tokens. Otherwise, the network could simply attend to the token it is supposed to predict and would fail to learn a meaningful autoregressive model. Decoder-only transformers address this issue through *masked self-attention*, also called *causal attention*. For the prediction target at position n, the mask allows attention only to the shifted input tokens that precede this target, while all later input positions are masked. In practice, the implementation applies this restriction by setting the attention scores of masked positions

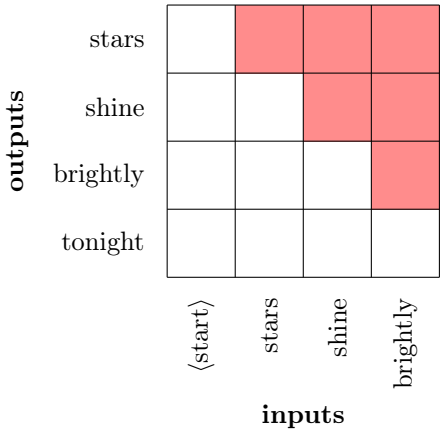

Figure 2.6: Illustration of causal masking in a decoder-only transformer. The columns show the shifted input tokens, while the rows show the corresponding prediction targets. Red cells mark masked positions whose attention probabilities are forced to zero. Thus, when predicting *brightly*, the output can depend only on ⟨start⟩, *stars*, and *shine*.

to $-\infty$ before the softmax operation. Figure 2.6 illustrates the resulting lower-triangular attention pattern.

During training, the model usually processes several sequences together in a mini-batch to exploit GPU parallelism efficiently. Since natural-language sequences often have different lengths, shorter sequences are padded to a common length. An additional padding mask then ensures that padded positions do not influence the attention computation.

At inference time, the trained model generates text autoregressively. Starting from an initial context, it predicts a distribution over the next token, selects or samples one token from that distribution, appends it to the sequence, and repeats the process. Although this procedure generates tokens one by one, practical implementations can reuse previously computed intermediate results for earlier positions, since causal attention ensures that these representations do not depend on future tokens.

2.1.7 Large Language Models

Having introduced decoder-only transformers as the architectural basis for modern generative language models, we now turn to LLMs as scaled instances of this architecture. LLMs are transformer-based neural networks trained to model natural language and, in many cases, other token sequences such as source code. The term "large" refers primarily to the number of learnable parameters, but also reflects the scale of the training data and computation used during training. In modern generative LLMs, the decoder-only transformer architecture is commonly used to learn next-token prediction from large collections of token sequences.

The development of LLMs has been driven by the combination of large text corpora, massively parallel training hardware, and the computational structure of transformers.

Since self-attention allows many token positions to be processed in parallel, transformer models can make effective use of GPUs and large accelerator clusters. Empirical scaling results further suggest that increasing model size, training data, and compute can lead to substantial improvements in model performance [30, 56]. This scaling behavior has motivated the training of increasingly large models such as the GPT family [7, 45, 50].

A central reason for the success of LLMs is the use of self-supervised learning. Instead of relying on manually labeled examples, the model obtains its training signal directly from raw token sequences. In the decoder-only setting, each token acts as a target for the context that precedes it. This objective turns unlabeled text into training data and enables training networks with very large numbers of parameters. The resulting pretraining process produces a general-purpose language model that captures statistical regularities, factual associations, and task patterns present in the training corpus.

This pretraining paradigm also changes how language models are used. Rather than training a separate model from scratch for each task, a large pretrained model can serve as a foundation model [6]. Such a model can be adapted to downstream tasks through fine-tuning, where additional labeled data modifies either all parameters or only a smaller set of task-specific parameters. However, as model capabilities have improved, many tasks can also be expressed directly through natural-language prompts. In this setting, the user provides an instruction, a question, or a few examples as part of the input context, and the model generates the continuation autoregressively.

For LLM serving, this usage pattern is particularly important. A deployed model repeatedly receives prompts, generates output tokens one after another, and must do so under latency, throughput, memory, and quality constraints. The computational cost of serving therefore depends not only on the model architecture, but also on model size, sequence length, batching behavior, and the number of generated tokens. These properties make LLM inference a demanding systems problem and motivate serving systems that manage model execution efficiently at runtime.

2.2 LLM Inference Systems

The previous section introduced modern generative LLMs as autoregressive decoder-only transformers. From a systems perspective, this property has direct consequences. A request does not require only one forward pass through the model, but a sequence of forward passes that generate output tokens one after another. LLM inference systems therefore need to manage a specific execution workflow, substantial memory requirements, and multiple concurrent requests under latency, throughput, and quality constraints. This section introduces the main concepts needed to understand such systems.

2.2.1 Inference Workflow

An LLM inference request starts with a prompt provided by the user. The system first tokenizes this prompt and passes the resulting token sequence through the model. This initial processing step is commonly called the *prefill* phase. During prefill, the model

processes all prompt tokens and computes the internal attention state needed to start generation. Since the prompt tokens are already known, the model can process this phase with a high degree of parallelism.

After prefill, the system enters the *decode* phase. In this phase, the model generates output tokens autoregressively. We refer to Section 2.1.6 for a more detailed explanation of autoregressive token generation. At each step, it predicts a probability distribution over the vocabulary, selects or samples the next token, appends this token to the sequence, and repeats the process. Unlike prefill, each decode step depends on the token generated in the previous step. This dependency makes decoding sequential within a single request, although serving systems can still group decode steps from multiple requests together.

The distinction between prefill and decode is important because the two phases stress the system differently. Prefill processes many tokens at once and can exploit parallel computation effectively [1, 68]. Decode, in contrast, performs many small repeated steps and must maintain the state of every active sequence [32, 47]. LLM serving therefore requires mechanisms that implement both phases efficiently and coordinate them across many concurrent requests [32, 65].

2.2.2 KV Cache

The key-value cache, usually called the *KV cache*, is the central optimization that makes autoregressive decoding practical. In every attention layer, each token produces key and value vectors (see Section 2.1.2). During generation, the model repeatedly attends over the previous tokens in the sequence. Without caching, each decode step would have to recompute the keys and values for the entire prefix again. With a KV cache, the system stores these vectors after their first computation and reuses them in later decode steps.

This optimization follows directly from causal attention (see Section 2.1.6). Since the representation of an earlier token does not depend on future tokens, the corresponding key and value tensors remain valid after the model generates additional tokens. At each decode step, the system only has to compute the new key and value tensors for the newly generated token and append them to the existing cache.

The KV cache greatly reduces redundant computation, but it also introduces substantial memory pressure. Its size grows with the number of active requests, the sequence length, the number of layers, and the hidden dimensionality of the model. As a result, LLM serving is often constrained not only by compute throughput, but also by the amount of GPU memory available for model weights and KV cache entries. Efficient cache management therefore becomes a core problem for high-throughput inference systems [32, 52].

2.2.3 Batching

Batching improves GPU utilization by processing multiple requests together. In conventional neural network inference, a system can often form a fixed batch, execute the model once, and return one output per input. Autoregressive LLM serving makes batching more difficult because requests differ in prompt length and generate different numbers

of output tokens. Some requests finish early, while others continue decoding for many more steps.

A static batching strategy handles this situation poorly. If the system waits until all requests in a batch finish, completed requests occupy batch slots longer than necessary [62, 65]. If the system only starts a new batch after the previous one has completed, newly arriving requests may wait even though the GPU could already process them [40, 65]. The result is inefficient resource utilization, especially under variable request lengths.

Modern LLM serving systems therefore employ *continuous batching*. Instead of treating a whole request as the scheduling unit, the system schedules generation at the level of decoding iterations. Finished requests can leave the batch, and newly arrived requests can enter the batch between decoding steps. This approach allows the server to keep the GPU busy even when requests arrive at different times and produce outputs of different lengths [32, 65]. Subsequent systems refine batched serving further by controlling how prefill and decode work are combined, interleaved, or separated during execution [1, 24, 68].

2.2.4 Parallelism

LLM inference systems often use multiple GPUs to serve models efficiently. Parallelism becomes necessary when a model does not fit into the memory of a single GPU, or when a single GPU cannot provide the required throughput or latency. In the context of LLM serving, we distinguish three different types of parallelism.

Data parallelism (DP) creates multiple replicas of the same model and places each replica on a different GPU or group of GPUs. Each replica can process different requests independently. This approach increases aggregate throughput when the model fits into the memory available to each replica [43, 47]. However, data parallelism does not reduce the memory required for one model replica, since every replica still stores the full model weights.

Tensor parallelism (TP) splits individual tensor operations inside a model layer across multiple GPUs. For example, the matrix multiplications in attention or feed-forward layers can be partitioned so that each GPU computes only part of the result. This strategy reduces the amount of model state stored per GPU and can reduce the latency of large layer computations. At the same time, tensor parallelism introduces communication overhead between GPUs, because partial results must be exchanged or combined during the forward pass [47, 53].

Pipeline parallelism (PP) partitions the model by layers and assigns different layer groups to different GPUs. A request then passes through the pipeline stage by stage. This approach allows the model weights to be distributed across multiple devices and can make very large models serveable even when they exceed the memory capacity of a single GPU. However, pipeline parallelism introduces dependencies between stages and can lead to idle time when some stages wait for inputs from earlier stages [27, 47].

These forms of parallelism address different bottlenecks. Data parallelism primarily increases throughput by replicating the model. Tensor parallelism and pipeline parallelism primarily distribute one model across several GPUs, which helps with memory

capacity and can improve latency for large models. Modern inference systems may combine these techniques, for example by using tensor or pipeline parallelism within one model replica and data parallelism across several replicas. The right choice depends on model size, GPU memory capacity, communication bandwidth, request rate, and latency requirements [47].

2.2.5 Serving Objectives

For the purposes of this thesis, we consider two primary serving objectives for LLM inference systems. *Latency* measures the time between submitting a request and receiving the corresponding response. In interactive applications, latency directly affects the user-perceived responsiveness of the system. A serving system therefore often has to satisfy a latency target, for example by ensuring that requests complete below a predefined threshold.

Throughput measures how many requests the system completes per unit of time. In this thesis, we define throughput as requests per second. Higher throughput indicates that the system can serve more users or process a larger workload with the same hardware resources. Throughput therefore relates directly to hardware utilization and serving cost.

Latency and throughput are closely connected. Larger batches can improve throughput because they allow the GPU to process more work in parallel. However, batching can also increase latency if requests wait longer before execution or if long-running requests delay shorter ones. LLM inference systems must therefore balance both objectives rather than optimizing either one in isolation. A system that maximizes throughput while violating latency requirements may be unsuitable for interactive serving, while a system that minimizes latency without considering throughput may use expensive hardware inefficiently [47, 65].

2.2.6 vLLM

vLLM [32] is an LLM inference and serving system designed for high-throughput generation. Its central contribution is PagedAttention, a memory-management technique for the KV cache inspired by virtual memory paging. Instead of requiring each request's KV cache to occupy one contiguous memory region, PagedAttention stores KV-cache blocks in non-contiguous memory and manages them through a block table. The block-table layout reduces fragmentation and allows the system to use GPU memory more efficiently.

This memory-management design is important because the KV cache grows and shrinks dynamically as requests enter, decode tokens, and finish. By reducing wasted KV-cache memory, vLLM can admit larger effective batches and improve throughput under latency constraints. In addition, vLLM supports continuous batching of incoming requests, which allows the serving engine to add and remove requests during generation rather than processing only fixed batches (see Section 2.2.3).

This thesis builds on vLLM as the underlying inference engine. The concepts introduced above therefore form the technical basis for the later system design. Prefill and

decode define the basic execution workflow, the KV cache creates a major memory-management challenge, batching determines how efficiently concurrent requests use the GPU, and serving objectives define the constraints under which runtime decisions must operate.

2.3 Quantization

Quantization refers to the process of reducing the numerical precision of a neural network's parameters by mapping high-precision floating-point values to representations with a smaller bit-width. In the context of LLMs, quantization has emerged as one of the most important techniques for model compression, since modern LLMs contain billions of parameters whose memory footprint and compute demands may exceed hardware capabilities. The primary motivations are threefold: reducing the GPU memory required to store model weights (possibly including activations) and the key-value cache, decreasing inference latency through higher instruction-level parallelism, and enabling deployment on edge devices or consumer-grade GPUs with more limited memory budgets. Figure 2.7 illustrates the basic principle of weight quantization by showing how high-precision weights are mapped to a discrete low-bit grid.

Quantization-Aware Training (QAT) [29, 37] and Post-Training Quantization (PTQ) [42] represent the two general paradigms. QAT simulates low-precision quantization during training, so the model learns to adapt its weights to the numerical errors that will occur after deployment in the quantized format. PTQ compresses an already trained model without further gradient updates. For LLMs, PTQ dominates in practice because retraining such models is prohibitively expensive. Prominent PTQ methods include GPTQ [17], AWQ [34], SmoothQuant [63], and GGUF k-quant variants [18, 31]. GPTQ quantizes weights layerwise while using approximate second-order information to reduce the quantization error. AWQ focuses on the most important weight channels, which are identified through activation statistics, and preserves them more carefully during quantization. SmoothQuant targets weight-and-activation quantization by rescaling weights and activations so that activation outliers become easier to quantize. GGUF k-quant methods are commonly used in llama.cpp-style CPU inference [18] and provide practical low-bit formats for efficient local deployment.

Beyond the choice of algorithm, practitioners must also select a numerical scheme that determines how values are represented. Common schemes include FP8 (8-bit floating point), INT8 (8-bit integer), and INT4 (4-bit integer), each offering a different trade-off between compression ratio, numerical accuracy, and hardware support. These schemes can apply either only to weights (weight-only quantization) or to both weights and activations. Even within the same numerical format, quantization methods can differ in how they map values to low-bit representations and in how locally they choose the scaling factors [19, 66]. Figure 2.8 illustrates the bit layouts of common numerical formats used in LLM quantization.

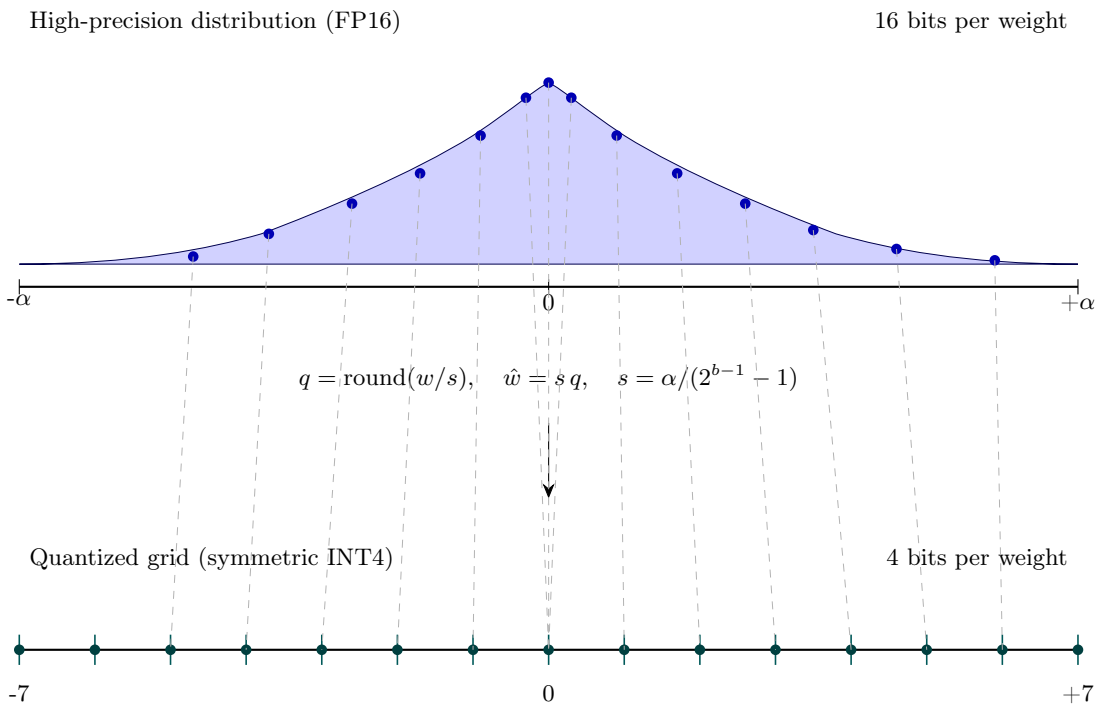

Figure 2.7: Illustration of symmetric INT4 quantization for model weights. The upper curve represents the density of high-precision FP16 weight values w over the range $[-\alpha, \alpha]$. Quantization maps each weight to the nearest value on a discrete low-bit grid by computing the integer value $q = \text{round}(w/s)$ with scale factor $s = \alpha/(2^{b-1} - 1)$. For INT4, where $b = 4$, this yields the symmetric grid $q \in \{-7, \ldots, 7\}$. The dashed lines show how individual FP16 weights are assigned to their nearest quantized levels, and the lower axis visualizes the resulting INT4 grid. During dequantization, the system reconstructs an approximation of the original weight as $\hat{w} = s\,q$.

2.3.1 FP8

FP8 [38] is an 8-bit floating-point format proposed for deep learning in two main variants, E4M3 and E5M2. In contrast to integer formats, FP8 preserves a floating-point structure with a sign bit, exponent bits, and mantissa bits. E4M3 uses four exponent bits and three mantissa bits, which gives it higher precision but a smaller dynamic range. E5M2 uses five exponent bits and two mantissa bits, which increases the dynamic range at the cost of precision. As a result, E4M3 is commonly used for weights and activations, while E5M2 is often used for gradients or tensors with larger numerical ranges. Figure 2.8 shows the bit allocation of both variants alongside the integer formats.

FP8 is attractive for LLM inference because it reduces the memory footprint by roughly a factor of two compared to FP16 or BF16 while retaining a larger dynamic range than same-width integer formats. This dynamic range makes FP8 particularly useful for tensors with outliers, such as LLM activations. Figure 2.9 places FP8 in this broader context

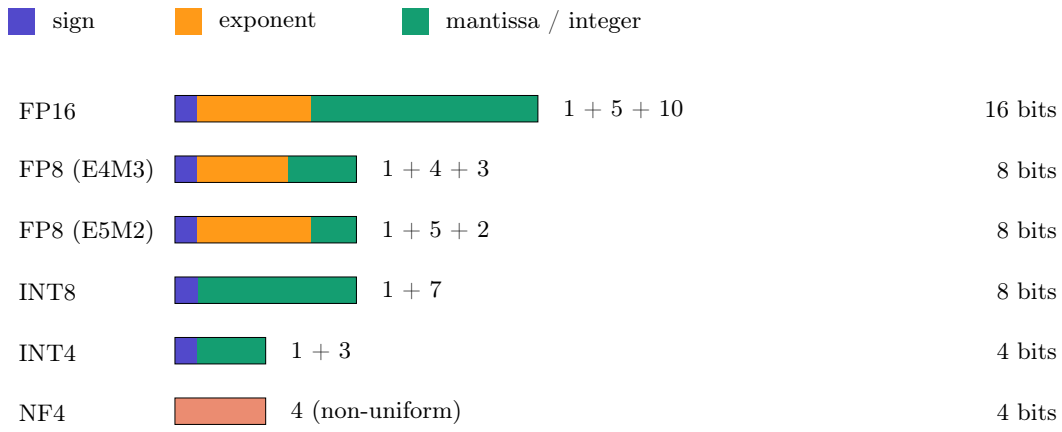

Figure 2.8: Bit layouts of common numerical formats used in LLM quantization. FP16 and the FP8 variants E4M3 and E5M2 consist of a sign bit, exponent bits, and mantissa bits, but allocate different numbers of bits to each field. INT8 and INT4 are shown as signed integer formats, where the leading bit acts as a sign-indicating bit and the full bit pattern encodes the signed integer value. NF4 is shown as a 4-bit non-uniform format, since it does not use a sign-exponent-mantissa layout.

by showing that FP8 reduces the idealized storage cost from 2 bytes to 1 byte per weight, matching INT8 and halving the storage required by FP16. On hardware with native FP8 support, such as NVIDIA Hopper GPUs, FP8 Tensor Cores can also improve throughput by reducing memory traffic and accelerating matrix multiplications. Modern inference frameworks such as TensorRT-LLM and vLLM therefore support FP8 quantization as a practical option for serving LLMs on recent accelerators.

2.3.2 INT8 (W8A8)

INT8 quantization in the W8A8 configuration refers to the setting in which both weights (W) and activations (A) are quantized to 8-bit integer values. The scheme maps floating-point values to a discrete integer range using a scale factor and, in asymmetric variants, a zero point. Symmetric INT8 quantization commonly uses a signed range centered around zero, while affine variants may shift the representable range through the zero point. Figure 2.8 shows the corresponding 8-bit integer layout.

The main advantage of W8A8 over weight-only schemes is that matrix multiplications can be executed directly with INT8 operands, typically with accumulation in INT32 before converting the result back to a floating-point format. This execution pattern is illustrated in the upper row of Figure 2.9. Since both weights and activations use 8-bit values, W8A8 reduces the idealized storage cost by roughly a factor of two compared to FP16. On hardware with efficient INT8 support, W8A8 can also improve inference throughput by reducing memory traffic and accelerating matrix multiplications.

However, activation quantization is more difficult for LLMs than weight-only quan-

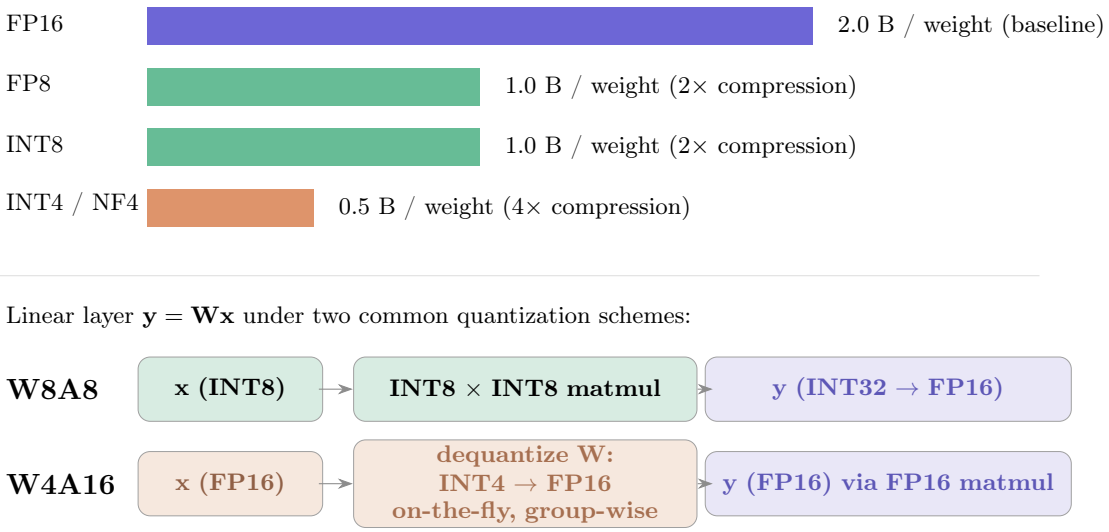

Figure 2.9: Storage cost and computation flow of common quantization schemes. The upper part shows the idealized byte cost per weight relative to an FP16 baseline, excluding additional metadata such as scales and zero-points. FP8 and INT8 reduce the storage cost from 2 bytes to 1 byte per weight, while INT4 and NF4 reduce it to 0.5 bytes per weight. The lower part contrasts W8A8 and W4A16 execution for a linear layer $\mathbf{y} = \mathbf{W}\mathbf{x}$. In W8A8, both weights and activations use INT8 values, and the matrix multiplication is performed in integer arithmetic with accumulation in INT32 before casting back to FP16. In W4A16, only the weights are stored in 4-bit precision; activations remain in FP16, and the weights are dequantized on the fly, typically group-wise, before the FP16 matrix multiplication.

tization. Transformer activations often contain large outlier channels, and naive INT8 activation quantization can therefore lead to noticeable accuracy loss. Methods such as LLM.int8() [13] address this problem by computing outlier channels in higher precision while using INT8 for the remaining channels. SmoothQuant [63] instead applies per-channel scaling to shift quantization difficulty from activations to weights, enabling accurate W8A8 post-training quantization for LLMs. With such techniques, INT8 W8A8 can provide a strong trade-off between accuracy, memory reduction, and hardware efficiency.

2.3.3 INT4 (W4A16)

INT4 quantization in the W4A16 configuration compresses model weights to 4-bit integers while keeping activations in 16-bit floating-point precision, typically FP16 or BF16. Figure 2.8 shows the corresponding 4-bit integer layout, while Figure 2.9 shows the idealized storage reduction from 2 bytes to 0.5 bytes per weight relative to an FP16 baseline.

This scheme is especially attractive for LLM inference because autoregressive decoding is often limited by memory bandwidth, particularly at small batch sizes. In this regime,

reducing the amount of weight data loaded from GPU memory can improve inference latency and throughput. However, the speedup is not purely proportional to the compression ratio, because practical performance also depends on kernel support, dequantization overhead, batching, and hardware characteristics [16, 34].

W4A16 avoids the difficult problem of activation quantization by keeping activations in FP16 or BF16. The 4-bit weights are dequantized on the fly during computation, often with group-wise scaling, as illustrated in the lower row of Figure 2.9. In group-wise quantization, a separate scale, and sometimes a zero point, is stored for small groups of weights rather than for the entire tensor. Group-wise scaling improves accuracy compared to coarse per-tensor quantization while keeping the metadata overhead manageable.

W4A16 is commonly combined with PTQ algorithms such as GPTQ [17] and AWQ [34], since 4-bit weight quantization is aggressive. GPTQ uses approximate second-order information to reduce layer-wise reconstruction error , while AWQ uses activation statistics to identify salient weight channels and quantizes them more carefully. With such methods, W4A16 can achieve substantial memory savings with limited accuracy degradation, making it a practical format for serving larger LLMs under constrained GPU memory.

2.3.4 NF4

NormalFloat 4-bit (NF4) is a 4-bit data type introduced in QLoRA [14] for memory-efficient fine-tuning of LLMs. Unlike INT4, NF4 does not represent values on a uniformly spaced integer grid. Instead, it uses a non-uniform set of representable values chosen for normally distributed weights. The non-uniform value set makes NF4 well suited for pre-trained LLM weights, which are often approximately zero-centered after normalization. Figure 2.8 therefore shows NF4 separately from the sign-exponent-mantissa layouts of floating-point formats and from the signed integer layouts of INT8 and INT4.

In terms of storage, NF4 has the same idealized cost as INT4: each weight uses 4 bits, corresponding to 0.5 bytes per weight. As shown in Figure 2.9, the 4-bit representation yields a 4× reduction in weight storage relative to an FP16 baseline, excluding additional metadata such as scaling factors. In practical implementations, NF4 is usually combined with block-wise or group-wise scaling, where each group of weights shares quantization metadata. QLoRA further reduces the metadata overhead through double quantization, which quantizes the quantization constants themselves.

During computation, NF4 weights are typically dequantized to a higher-precision compute type such as FP16 or BF16 before or during the matrix multiplication. Thus, NF4 is primarily a weight-storage format rather than a format for fully low-precision arithmetic. The storage-oriented design makes NF4 particularly useful when the main bottleneck is GPU memory capacity, for example when loading or fine-tuning models that would otherwise not fit into available memory. In practice, NF4 is closely associated with the bitsandbytes library [12].

3 Kairos: System Architecture

Kairos is a workload-aware and reconfigurable serving system for LLMs that adapts its runtime behavior to the observed request stream. Instead of treating the serving configuration as fixed after startup, Kairos continuously observes incoming requests, evaluates serving alternatives, and reconfigures the system when the collected information indicates that a different configuration better satisfies the serving objectives. The goal is to improve resource efficiency and throughput while respecting the latency and accuracy constraints specified by the user.

The name Kairos refers to the Ancient Greek term *kairos*, which denotes the right, critical, or opportune moment for action. In contrast to purely chronological time, the term emphasizes timing as a qualitative decision. An action matters not only because it is possible, but because it is performed at the right moment. This idea captures a central aspect of the system. Kairos must not only determine which adaptation is beneficial, but also when this adaptation can be performed with minimal interference to active serving.

This chapter describes the architecture that realizes this idea. We first introduce the design principles that guide the system: continuous monitoring, runtime reconfigurability, and interference-aware scheduling. We then give an overview of the full architecture before elaborating on the individual layers, including how they realize these principles.

3.1 Design Principles

To enable workload-aware and reconfigurable serving of LLMs, we first derive key design principles that set the scope of Kairos and shape its overall system architecture. We elaborate on these principles before discussing how they specifically materialize in the system design.

P1 – Continuous Monitoring: A central requirement for workload-aware serving is the continuous monitoring of incoming client requests. Sampling requests during operation enables direct comparison of relevant models with respect to predefined objectives on the actual workload observed by the system. This online monitoring can detect changes in the request distribution, allowing the serving system to adapt its model selection and placement accordingly.

P2 – Runtime Reconfigurability: Workload-aware serving requires the ability to modify the serving configuration during operation. Instead of treating deployment choices as fixed after startup, the system should support controlled changes to its configuration while continuing to serve requests. Based on the information collected through continu-

ous monitoring, runtime reconfigurability enables the serving system to react to changing workload characteristics, objective violations, or resource constraints by adapting its behavior dynamically.

P3 – Interference-aware Scheduling: Reconfigurable serving requires coordination between adaptation actions and ongoing request processing. Since configuration changes can consume resources or temporarily affect serving behavior, the system should schedule such actions in a way that minimizes interference with active request serving. Interference-aware scheduling uses information about the current workload pattern and system state to decide when to perform adaptation allowing the serving system to remain responsive while still adapting to changing conditions.

3.2 System Overview

Kairos is structured into four layers: the KairosAPI, the inter-process communication (IPC) layer, the KairosCore, and the vLLM runtime layer. Together, these layers separate external request handling, process communication, orchestration logic, and inference:

- **KairosAPI:** This is the external entry point for client requests. It validates incoming inference requests, adds API-level metadata such as the arrival timestamp, forwards the request to the Core via IPC, and returns the Core's result back to the client.
- **Inter-Process Communication (IPC):** The IPC layer connects the API process with the Core process. It transports inference requests from the API to the Core and routes the corresponding responses back to the correct pending client request.
- **KairosCore:** This contains the central orchestration logic of the system. It receives requests from the IPC layer, dispatches inference requests to the currently serving model, collects monitoring information, and executes control commands such as pausing dispatch, evaluating models, and reconfiguring model placement.
- **vLLM:** Kairos uses vLLM as its inference engine. Beyond standard inference serving, this layer exposes the mechanisms that Kairos requires to manage model availability across memory levels. These mechanisms include sleeping a model to release GPU memory, waking a model to make it available for inference again, prefetching model data from disk into CPU memory, and evicting model data from CPU memory when it is no longer needed.

Before serving begins, the user specifies the set of models in the configuration file that Kairos should consider during runtime. Each model is registered through its Hugging Face model identifier (`model_id`) and becomes part of the model catalog maintained by the system. For each model, the configuration file exposes a field (`relation`), with possible values `base`, `quantized`, and `independent`. The base model serves as the ground

truth against which Kairos compares the performance of the other models during runtime. The registered models may be related to each other. For example, if a model represents a quantized variant of the base model, the `relation` field indicates this relationship accordingly. However, a registered model may also come from a different model family than the base model, which the configuration can express as well. Besides the models, the configuration file specifies the service-level indicators (SLIs) with their corresponding target values called service-level objectives (SLOs). The system's overall goal is to maximize throughput while selecting the most resource-efficient serving model that still satisfies the SLOs. In Kairos, we define latency and accuracy as SLIs. The user can set an upper bound for the p95 request latency, for example, requiring client requests to be answered in less than 200 ms. For the accuracy constraint, we assume the base model's predictions as ground truth, and the user sets a lower bound for this objective, for example, requiring 95% of client requests to match the base model. The system also allows the user to assign weights to the SLOs to emphasize their relative importance during model scoring. If the user does not provide any weights, both constraints contribute equally to the scoring. Furthermore, the configuration provides additional tunable parameters specific to KairosCore. These parameters influence aspects such as monitoring, scheduling, reconfiguration, and the following sections describe them in more detail. Figure 3.1 shows an example configuration containing the aforementioned parameters.

The KairosAPI forms the external entry point of the system and runs in a separate process from KairosCore. This separation isolates client-facing request handling from the internal orchestration logic and allows the Core to operate independently of the web-serving layer. Incoming client requests are received through the API endpoint, validated according to the expected request schema, and enriched with API-level metadata such as the arrival timestamp. After this preprocessing step, the KairosAPI forwards the request to KairosCore through the IPC layer and waits for the corresponding result. Once the Core has completed the request, the KairosAPI returns the produced response to the client without exposing the system's internal orchestration logic.

The IPC layer connects the KairosAPI process with the KairosCore process. It consists of two parts: an IPC client that runs inside the KairosAPI process and an IPC server that runs inside the KairosCore process. The purpose of this layer is to decouple client-facing request handling from the internal processing performed by KairosCore.

Kairos uses ZeroMQ for this communication. When the API receives an inference request, the IPC client sends a message to the Core-side IPC server. This message contains a message kind, a unique request identifier, and the request payload. The payload contains the actual inference request, while the request identifier is used to match the later Core response to the correct pending API request.

This request identifier is necessary because request processing is asynchronous. The API process can receive and forward multiple requests without waiting for each request to finish before accepting the next one. At the same time, KairosCore may process requests concurrently, for example by dispatching several inference requests to vLLM to exploit continuous batching. As a result, responses do not necessarily have to arrive in the same order as the original requests. The IPC client therefore keeps track of pending

config.yaml

```yaml
models:
  - model_id: meta-llama/Llama-3.1-8B-Instruct
    relation: base
    port: 8001
    gpu_factor: 1.3

  - model_id: RedHatAI/Llama-3.2-3B-Instruct-quantized.w8a8
    relation: quantized
    port: 8002
    gpu_factor: 1.3

  - model_id: Qwen/Qwen3-1.7B
    relation: independent
    port: 8003
    gpu_factor: 1.3

slo:
  accuracy: 0.95
  latency: 200
  weight_acc: 0.7
  weight_lat: 0.3

monitoring:
  sample_rate: 10
  sample_size: 50
  monotonicity: true
  discard: 3
  recycle: 3

scheduler:
  method: holt
  window: 10
```

Figure 3.1: Example configuration file. The configuration specifies the models considered during runtime, specifies their relation to the base model, and defines the SLOs for accuracy and latency, including their relative weighting. It also exposes tunable parameters for monitoring and scheduling. For monitoring, the configuration specifies the sampling rate, the number of requests to sample, whether to account for monotonicity among quantized models, and how to handle models that repeatedly fail to satisfy the SLOs. For scheduling, the user can select the time-series forecasting method and set the time window, in seconds, for idle-time prediction.

requests by their request identifiers and resumes the corresponding API request once the matching response arrives from the Core.

This separation allows the API side to focus on receiving requests, forwarding them, and returning responses to clients, while the Core side performs the internal processing. The IPC layer therefore acts as the boundary between external request handling and

internal orchestration, while preserving the ability to process multiple requests concurrently across both processes.

KairosCore contains the internal orchestration logic of the system. After a request has been received through the IPC layer, the Core is responsible for deciding how the request is processed, which model should serve it, how runtime information is collected, and when the system should reconfigure its model placement. KairosCore is implemented around multiple asynchronous tasks that run concurrently within the Core process. Request dispatching, control-command processing, monitoring, scheduling, and IPC handling therefore do not form one strictly sequential execution path. Instead, these activities cooperate through asynchronous queues, shared model-state metadata, and control commands, allowing KairosCore to dispatch inference requests, collect monitoring data, and prepare reconfiguration actions without blocking the entire system on a single operation.

The controller acts as the central coordination point inside KairosCore. It receives requests from the IPC layer, places them into the internal request queue, dispatches them to the currently active model, and sends the completed results back through IPC. In addition to inference dispatching, the controller also processes control commands that affect the runtime state of the system, such as pausing request dispatch, starting or stopping model servers, or triggering sleep and wake-up operations.

The monitoring component observes the behavior of the system during serving. It continuously samples a fraction of request-result pairs, records latency and accuracy information, and provides the data needed to compare the actively serving model with alternative models. This information forms the basis for deciding whether the current serving configuration still satisfies the desired SLOs.

The model catalog maintains the set of models that are available to Kairos. It stores the metadata associated with each registered model, including its Hugging Face identifier, runtime port, relation to other variants, memory requirements, and current placement in the memory hierarchy. The catalog therefore provides the shared model representation used by the other components of KairosCore.

The reconfiguration component uses the information collected by monitoring to decide whether the system should change its current serving configuration. It compares models with respect to the configured objectives and derives an updated model preference and placement decisions. In this way, reconfiguration determines which models should be active, evaluated, or moved between memory tiers.

The scheduler is responsible for turning such decisions into executable runtime actions. It observes request arrivals, forecasts suitable idle windows, and schedules control commands for model movement and evaluation. Its role is to decide when reconfiguration actions can be performed without unnecessarily interfering with active serving.

Finally, KairosCore uses the vLLM interface to interact with vLLM model servers. The interface sends inference requests to the selected model server and exposes the runtime operations required by Kairos, such as sleep, wake-up, prefetching, and eviction. Centralizing vLLM communication in this interface keeps the Core logic independent of the concrete HTTP endpoints exposed by vLLM while still allowing KairosCore to control model execution directly. Figure 3.2 shows an overview of Kairos' architecture.

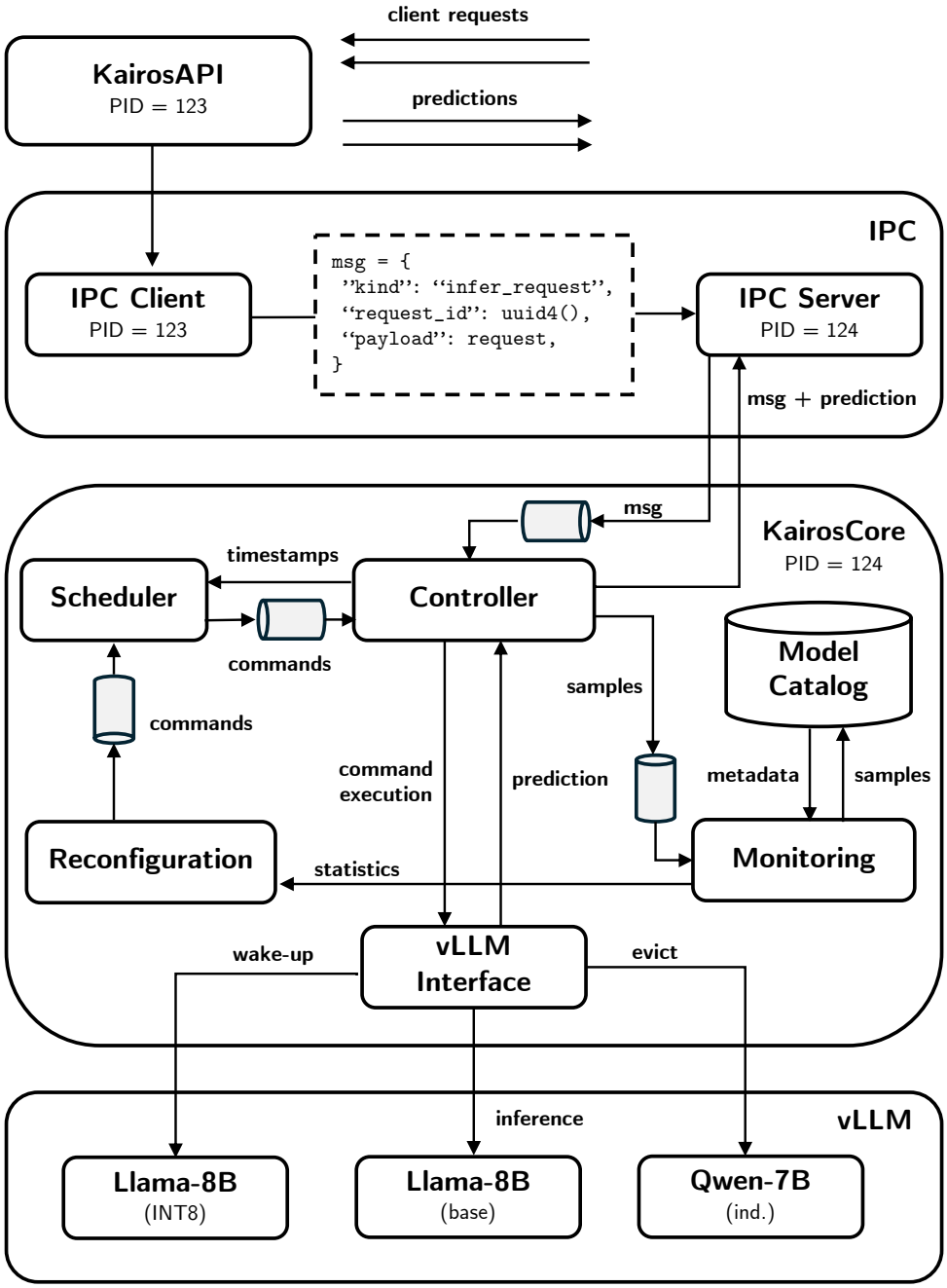

Figure 3.2: Overview of the Kairos system architecture. Kairos separates the client-facing KairosAPI process from the KairosCore process through an IPC layer. KairosAPI forwards client requests as inference messages to KairosCore. Inside KairosCore, the controller, monitoring, scheduler, reconfiguration logic, model catalog, and vLLM interface exchange samples, timestamps, statistics, predictions, and commands. The vLLM layer executes inference requests and model placement actions on the model servers.

3.3 Client

Kairos uses a separate client program to generate request workloads and send them to the KairosAPI. The client does not belong to the serving system itself. Instead, it acts as an external workload generator that emulates users submitting inference requests to Kairos. This separation keeps the serving system and the evaluation driver independent. Kairos exposes its inference endpoint, while the client controls the input data, request timing, the number of concurrent in-flight requests, and logging behavior.

The client reads its configuration from a YAML file. The configuration specifies the dataset, the KairosAPI port, the inference endpoint, the maximum number of requests, the maximum number of concurrent in-flight requests, the logging mode, and the workload pattern. The client loads the selected dataset from a preprocessed JSONL file. Each row already follows the request format expected by Kairos, so the client does not perform additional prompt construction or preprocessing. It reads one row at a time and sends the row unchanged to the KairosAPI.

The client uses asynchronous HTTP requests to support realistic request arrivals. The client does not wait for one request to finish before launching the next one. Instead, it creates an independent asynchronous task for each launched request. Creating separate tasks allows multiple requests to remain in flight at the same time, reflects realistic client behavior, and enables Kairos to exercise its internal batching and scheduling mechanisms. A configurable `max_in_flight` parameter limits the number of outstanding requests and prevents the client from creating an unbounded number of pending connections.

The workload pattern determines the time between consecutive request launches. After launching a request, the client waits for the next inter-arrival delay produced by the configured workload pattern before launching another request. Request completion proceeds independently from request generation. As a result, the client models the arrival process explicitly. Workload patterns control when requests enter Kairos, while response times determine when the corresponding asynchronous tasks complete.

For evaluation purposes, the client can write request results to a JSONL log file. Each log entry contains the client-side request identifier, the observed end-to-end latency, and the response returned by Kairos. The client assigns request identifiers according to launch order. If the configured maximum number of requests exceeds the number of rows in the dataset, the client continues from the beginning of the dataset and keeps assigning new request identifiers. This approach allows long workload runs even with smaller datasets while preserving a unique identifier for every launched request.

The client supports several synthetic workload patterns to evaluate Kairos under different request arrival behaviors. A single fixed workload would only capture one operating condition and would make it difficult to observe how the system reacts to idle phases, bursts, gradual load changes, or sudden shifts in request intensity. The patterns therefore create controlled traffic scenarios that stress different aspects of the system.

Each workload pattern controls the time between consecutive request launches. After the client sends one request, the selected pattern determines the delay before the next request is launched. The client does not wait for the previous response before sending

another request. Request generation and request completion therefore remain independent. This design allows the client to model the arrival process directly, while Kairos determines how quickly it can process the resulting requests.

The workload patterns use request rates measured in requests/s. These rates describe expected arrival intensities rather than strict per-second request counts. For example, a rate of five requests/s means that the generator produces five requests/s on average over time. Individual one-second intervals may contain fewer or more requests because the generator samples stochastic inter-arrival times. This distinction matters when interpreting workload plots, since the plotted bars show realized request counts in time buckets, while the configured rates define the expected behavior of the generator.

The evaluation uses multiple patterns because each pattern highlights a different system behavior. Poisson traffic provides a random baseline with a stable average rate. ON/OFF traffic creates explicit idle windows that Kairos can exploit for reconfiguration. Periodic traffic introduces repeating load structure. Bursty traffic tests how the system behaves under sudden request clusters. Ramp and step workloads introduce gradual and sudden changes in request intensity. Together, these patterns provide a compact but expressive workload suite for evaluating workload-aware serving behavior.

3.3.1 Poisson

The Poisson workload pattern serves as the random baseline for evaluating Kairos. It generates requests with a stable average arrival rate, but it does not introduce explicit structure such as phases, bursts, or trends. Table 3.1 summarizes the parameters of this pattern. The client samples the delay between consecutive requests from an exponential distribution parameterized by `rate`. Thus, `rate` controls the average arrival intensity, with larger values producing shorter delays between requests. As a result, the generated traffic fluctuates randomly around the configured rate.

Table 3.1: Parameters of the Poisson workload pattern.

Variable	Description
`rate`	Expected number of request arrivals per second.

Figure 3.3 illustrates this behavior. The realized request rate varies across individual time buckets, although the workload uses a fixed expected rate. This variation occurs because the configured rate defines an average arrival intensity rather than a strict number of requests per second. For example, a rate of 2 requests per second does not force the client to send exactly two requests in every second. Some seconds may contain fewer requests, while others may contain more. Over a longer time interval, the average request rate approaches the configured value.

This workload pattern provides a sanity check for the evaluation. Since Poisson traffic does not contain planned idle phases, periodic structure, or systematic load changes,

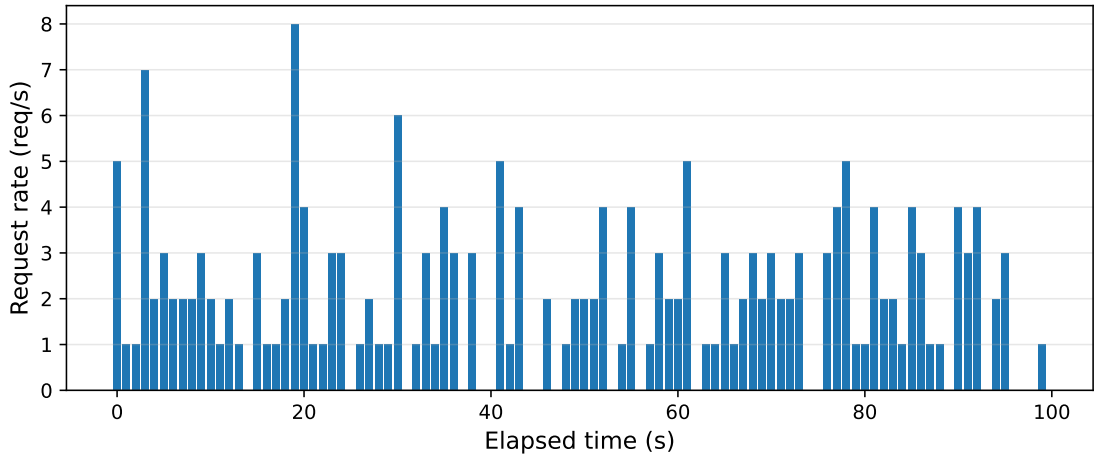

Figure 3.3: Poisson workload pattern with `rate` set to 2.0 requests per second. The workload uses a fixed expected arrival rate, while the realized request count varies across individual one-second time buckets due to stochastic inter-arrival sampling.

Kairos should not rely on deterministic patterns in the arrival stream. Instead, the system must handle random fluctuations while maintaining stable serving behavior. The Poisson pattern therefore establishes a baseline against which the more structured workload patterns can be compared.

3.3.2 ON/OFF

The ON/OFF workload pattern generates requests in alternating active and idle phases. During an active phase, the client sends requests with a stable average arrival rate. During an idle phase, the client sends no requests. Table 3.2 lists the parameters of this pattern. The parameter `on_duration_seconds` defines how long each active phase lasts, while `off_duration_seconds` defines how long each idle phase lasts. During the active phases, the client samples the delay between consecutive requests from an exponential distribution parameterized by `on_rate`. Figure 3.4 illustrates this behavior.

This workload pattern directly targets the reconfiguration behavior of Kairos. The OFF phases create explicit idle windows in which Kairos can perform expensive op-

Table 3.2: Parameters of the ON/OFF workload pattern.

Variable	Description
`on_duration_seconds`	Duration of each active phase in seconds.
`off_duration_seconds`	Duration of each idle phase in seconds.
`on_rate`	Expected number of request arrivals per second during active phases.

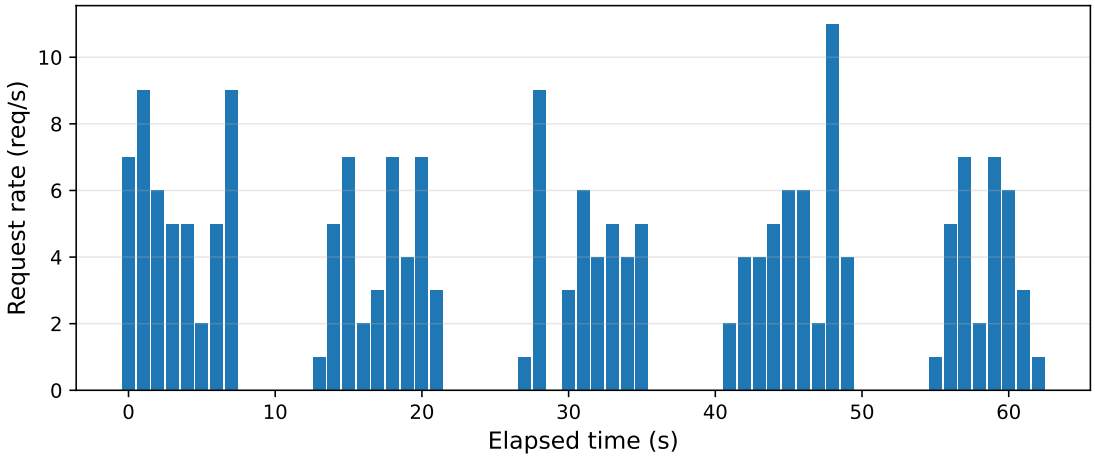

Figure 3.4: ON/OFF workload pattern with `on_duration_seconds` set to 8.0 seconds, `off_duration_seconds` set to 6.0 seconds, and `on_rate` set to 5.0 requests per second. The workload alternates between active phases with stochastic request arrivals and idle phases without request arrivals.

erations, such as model movement or model evaluation, with reduced interference to foreground inference requests. The following ON phases then test whether Kairos can resume serving once traffic returns. The ON/OFF pattern therefore establishes a controlled workload for evaluating whether Kairos can exploit idle periods while maintaining stable serving behavior during active periods.

3.3.3 Periodic

The periodic workload pattern serves as the predictable-structure workload for evaluating Kairos. This pattern generates requests with an expected arrival rate that follows a sinusoidal curve over time. Table 3.3 lists the parameters of this pattern. The parameter `base_rate` defines the center of the sinusoidal curve, while `amplitude` defines how strongly the expected rate moves above and below this center. The parameter `period_seconds` defines the duration of one full cycle.

Table 3.3: Parameters of the periodic workload pattern.

Variable	Description
`base_rate`	Center of the periodic request rate in requests per second.
`amplitude`	Maximum deviation from `base_rate` in requests per second.
`period_seconds`	Duration of one full periodic cycle in seconds.

The client uses these parameters to define a time-varying expected arrival rate. In the configured sinusoidal pattern, the expected rate oscillates between `base_rate` ±

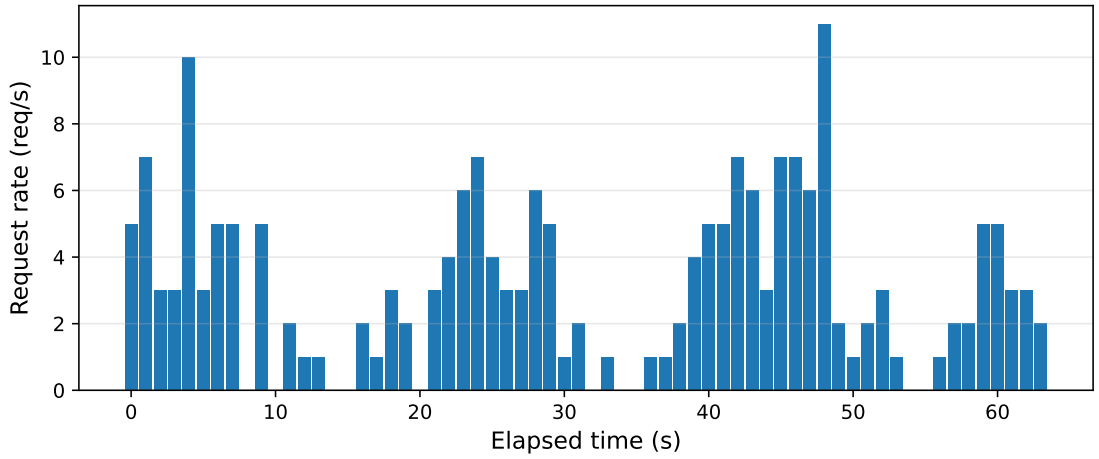

Figure 3.5: Periodic workload pattern with `base_rate` set to 3.0 requests per second, `amplitude` set to 2.5 requests per second, and `period_seconds` set to 20.0 seconds. The expected arrival rate oscillates between 0.5 and 5.5 requests per second, while the realized request count varies across individual one-second time buckets.

`amplitude`. This range describes the expected arrival intensity of the generator, not a hard upper bound on the number of requests observed in each one-second bucket. Larger values of `amplitude` create stronger differences between low-load and high-load phases. Larger values of `period_seconds` stretch the wave over a longer time interval. The client then generates request arrivals stochastically according to this time-varying rate.

Figure 3.5 illustrates this behavior. The request rate follows a repeating wave-like structure, but the realized request count still varies across individual one-second time buckets. This variation occurs because the configured sinusoidal rate defines an expected arrival intensity rather than a strict number of requests per second. Consequently, some one-second buckets may contain more requests than the current expected rate, especially near the high-load parts of the wave.

This workload pattern evaluates whether Kairos can handle recurring changes in load. Unlike the Poisson pattern, the periodic pattern introduces predictable temporal structure into the arrival stream. Unlike the ON/OFF pattern, it does not switch abruptly between active and idle phases. Instead, the load changes gradually and repeatedly. Kairos can exploit this structure if its scheduling component detects repeated high-load and low-load phases. The periodic pattern therefore tests whether the system remains stable under cyclic load variation and whether its adaptation mechanisms can benefit from recurring workload behavior.

3.3.4 Bursty

The bursty workload pattern serves as the sudden-spike workload for evaluating Kairos. This pattern combines low background traffic with occasional dense request clusters.

Table 3.4: Parameters of the bursty workload pattern.

Variable	Description
`base_rate`	Background request rate in requests per second outside burst periods.
`burst_event_rate`	Expected number of burst events per second.
`burst_size`	Number of requests generated by each burst event.
`burst_spread_seconds`	Time interval over which the requests of one burst are distributed.

Table 3.4 lists the parameters of this pattern. The parameter `base_rate` defines the background arrival intensity outside bursts. The parameter `burst_event_rate` controls how often bursts occur. Each burst then generates `burst_size` requests, and `burst_spread_seconds` defines how tightly the requests of one burst are packed in time.

The client uses `base_rate` to generate regular background arrivals. Independently, the client uses `burst_event_rate` to determine when a new burst starts. When a burst occurs, the client launches `burst_size` requests within `burst_spread_seconds`. Smaller values of `burst_spread_seconds` create sharper bursts because the same number of requests arrive in a shorter time window. Larger values spread the burst over a longer interval and therefore reduce its instantaneous intensity.

Figure 3.6 illustrates this behavior. Most time buckets contain only few requests or no requests at all, while burst intervals contain sudden spikes in the realized request rate.

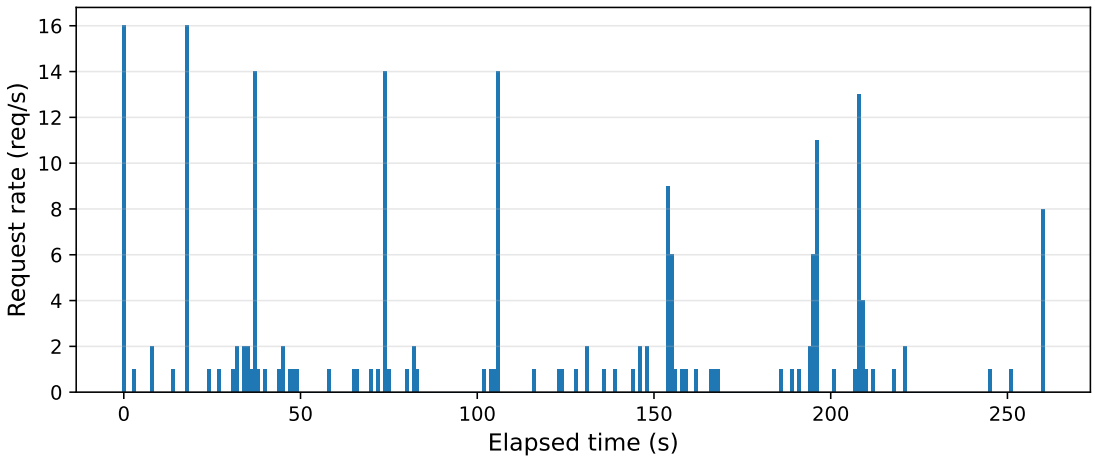

Figure 3.6: Bursty workload pattern with low background traffic and occasional request clusters. The parameters `base_rate`, `burst_event_rate`, `burst_size`, and `burst_spread_seconds` control the background arrivals, burst frequency, burst size, and temporal concentration of each burst.

The height of a spike depends on the configured burst size, the burst spread, and the random placement of requests within the burst interval. As with the other stochastic patterns, the plotted bars show realized request counts in one-second time buckets rather than strict configured rates.

The bursty workload pattern evaluates whether Kairos can detect sudden traffic spikes after quiet periods. Bursty traffic differs from the Poisson pattern because it does not fluctuate around a single stable average rate. It also differs from the ON/OFF pattern because quiet periods do not follow a fixed phase structure, and from the periodic pattern because bursts do not follow a smooth repeating wave. These properties make the bursty pattern important for testing false-idle predictions. Kairos may observe a quiet interval and schedule a reconfiguration, but a burst can arrive shortly afterwards and create latency pressure. The bursty workload therefore stresses the scheduler's robustness under irregular request spikes.

3.3.5 Ramp

The ramp workload pattern evaluates how Kairos responds to a gradual increase in request rate. This pattern generates requests with an expected arrival rate that changes linearly over time. Table 3.5 lists the parameters of this pattern. The parameter `start_rate` defines the initial expected request rate, while `end_rate` defines the expected request rate reached at the end of the ramp. The parameter `ramp_duration_seconds` defines how long this transition takes.

Table 3.5: Parameters of the ramp workload pattern.

Variable	Description
`start_rate`	Initial expected request rate in requests per second.
`end_rate`	Final expected request rate in requests per second.
`ramp_duration_seconds`	Time interval over which the expected request rate changes from `start_rate` to `end_rate`.

The client uses these parameters to define a time-varying expected arrival rate. At the beginning of the workload, the expected rate equals `start_rate`. During the ramp interval, the expected rate moves linearly toward `end_rate`. After `ramp_duration_seconds`, the expected rate remains at `end_rate`. Since the rate changes over time, the client models the workload as a time-varying Poisson process. It samples candidate inter-arrival times from an exponential distribution using the maximum rate of the ramp and keeps each candidate with probability equal to the current expected rate divided by this maximum rate. As a result, the observed request counts may fluctuate across individual one-second time buckets, although the underlying expected rate follows a gradual trend.

Figure 3.7 illustrates this behavior. The realized request rate increases over time, but individual buckets still show random variation around the underlying trend. This variation occurs because the configured rate at each point in time defines an expected arrival

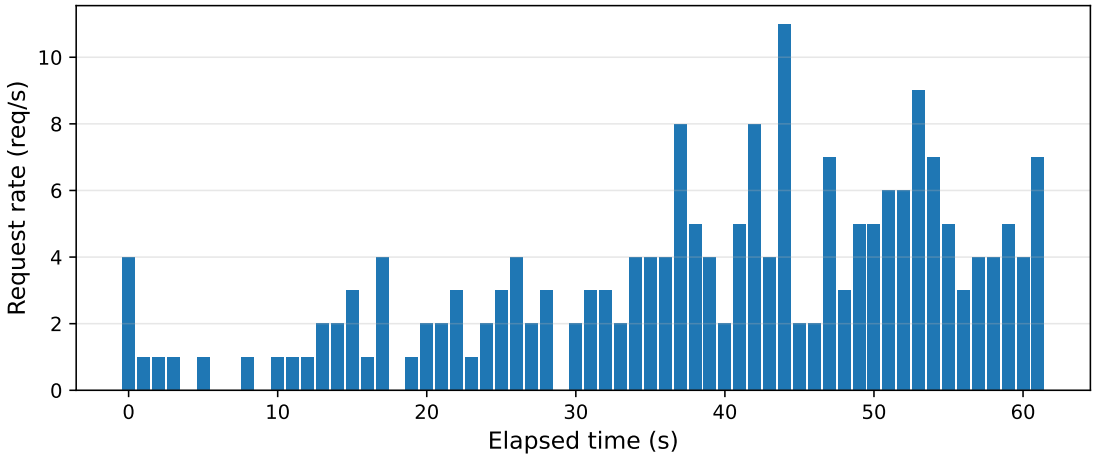

Figure 3.7: Ramp workload pattern with `start_rate` set to 0.5 requests per second, `end_rate` set to 6.0 requests per second, and `ramp_duration_seconds` set to 60.0 seconds. The expected arrival rate increases linearly during the ramp interval, while the realized request count varies across individual one-second time buckets.

intensity rather than a strict number of requests per second. Consequently, the plot may contain local decreases or spikes even though the expected rate increases smoothly.

This workload pattern evaluates how Kairos performs under gradual load change. In contrast to the Poisson pattern, the ramp pattern does not maintain a stable average request rate. It also differs from the ON/OFF pattern because it does not introduce explicit idle and active phases. Instead of repeating a cyclic structure, as in the periodic pattern, it creates a sustained trend in the arrival process.

3.3.6 Step

The step workload pattern evaluates how Kairos responds to a sudden change in request rate. This pattern generates requests with one expected arrival rate before the step and another expected arrival rate after the step. Table 3.6 lists the parameters of this

Table 3.6: Parameters of the step workload pattern.

Variable	Description
`before_rate`	Expected request rate before the step change in requests per second.
`after_rate`	Expected request rate after the step change in requests per second.
`switch_time_seconds`	Time at which the expected request rate changes from `before_rate` to `after_rate`.

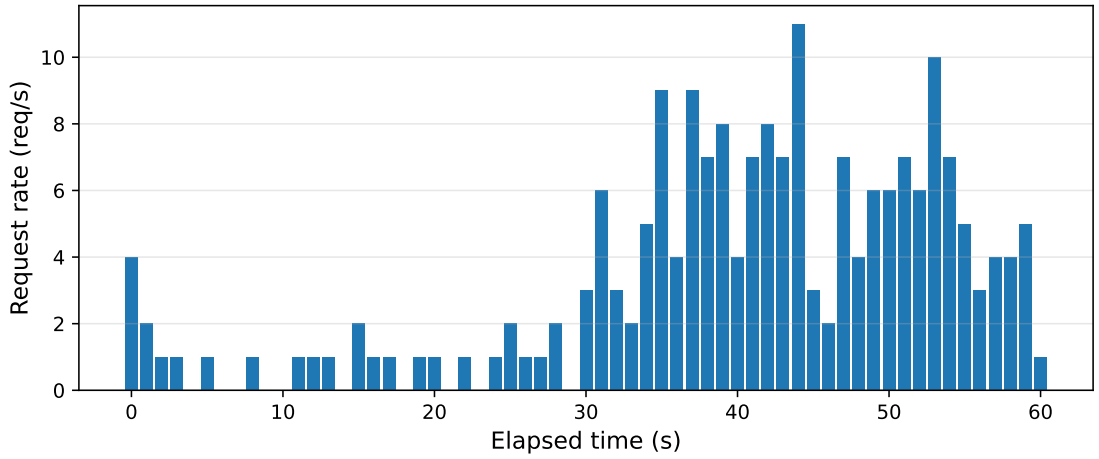

Figure 3.8: Step workload pattern with `before_rate` set to 1.0 requests per second, `after_rate` set to 6.0 requests per second, and `switch_time_seconds` set to 30.0 seconds. The expected arrival rate changes abruptly at the switch time, while the realized request count varies across individual one-second time buckets.

pattern. The parameter `before_rate` defines the expected request rate before the change, while `after_rate` defines the expected request rate after the change. The parameter `switch_time_seconds` defines when the change occurs.

The client uses these parameters to define two phases. Before `switch_time_seconds`, the current request rate is given by `before_rate`. From `switch_time_seconds` onward, the current request rate becomes `after_rate`. Within each phase, the client samples the delay between consecutive requests from an exponential distribution parameterized by the current request rate. As a result, the average delay between requests changes abruptly at the switch time. The observed request counts may still fluctuate across individual one-second time buckets because the configured rates define expected arrival rates rather than strict per-second request counts.

Figure 3.8 illustrates this behavior. The first part of the workload follows the lower expected request rate. At `switch_time_seconds`, the expected rate changes abruptly, and the second part of the workload follows the higher expected request rate. The step workload pattern complements the ramp workload pattern. Both patterns change the request rate over time, but they stress different adaptation behavior. Ramp introduces a gradual trend, while step introduces an abrupt arrival change.

3.4 vLLM

Kairos uses vLLM to serve the registered models. Each registered model is started as a separate vLLM server, which exposes endpoints for inference and model-management actions. These model-management endpoints contribute to runtime reconfigurability (P2) by exposing the low-level mechanisms for model swapping, placement, and eviction

without restarting the model servers. In Kairos, these mechanisms operate across three storage levels: GPU memory for models that serve requests, CPU memory for models staged close to the GPU, and disk storage for models that are currently not kept in memory. Internally, Kairos represents these actions as reconfiguration commands, while the actual execution happens through HTTP endpoints exposed by the corresponding vLLM model server. Table 3.7 summarizes the command-to-endpoint mapping used by Kairos to trigger these model-placement actions through vLLM.

Table 3.7: Mapping between Kairos commands and the corresponding vLLM endpoints.

Kairos Command	**vLLM Endpoint**
(a) `L1_SLEEP`	`/sleep?level=1`
(b) `L2_SLEEP`	`/sleep?level=2`
(c) `PREFETCH`	`/prefetch`
(d) `WAKE_UP_FROM_CPU`	`/wake_up`

(e) `WAKE_UP_FROM_DISK`	`/wake_up`
	`/collective_rpc {"method":"reload_weights"}`
	`/reset_prefix_cache`

(f) `WAKE_UP_FROM_PREFETCH`	`/wake_up`
	`/collective_rpc`
	`{"method":"reload_weights_from_prefetch"}`
	`/reset_prefix_cache`

(g) `WAKE_UP_PERSISTENT`	`/wake_up?persistent=true`
(h) `EVICT`	`/evict`

To provide this endpoint set, Kairos uses a modified version of vLLM. We forked vLLM version 0.13.0 and extended it with the additional functionality required for Kairos-specific model-placement commands. The unmodified vLLM version already provides the mechanisms corresponding to `L1_SLEEP`, `L2_SLEEP`, `WAKE_UP_FROM_CPU`, and `WAKE_UP_FROM_DISK`. For the remaining commands in Table 3.7, we added both the HTTP endpoints and the internal mechanisms that execute the corresponding model-placement actions when invoking these endpoints. From this modified vLLM version, we build a Docker image. Every model server that Kairos starts, runs as a container based on this image, ensuring that all registered models expose the same interface.

The commands in Table 3.7 describe Kairos-internal control commands. Some commands map to a single vLLM endpoint, while others consist of a short sequence of endpoint calls. The sleep functionality keeps a model server alive while releasing most of the GPU memory associated with the served model. Compared to shutting down the server and later starting a new vLLM instance, sleep mode preserves expensive serving infrastructure such as the process state, CUDA context, memory allocator state, CUDA graphs, and GPU kernel JIT cache [61]. While starting a model server can take multiple

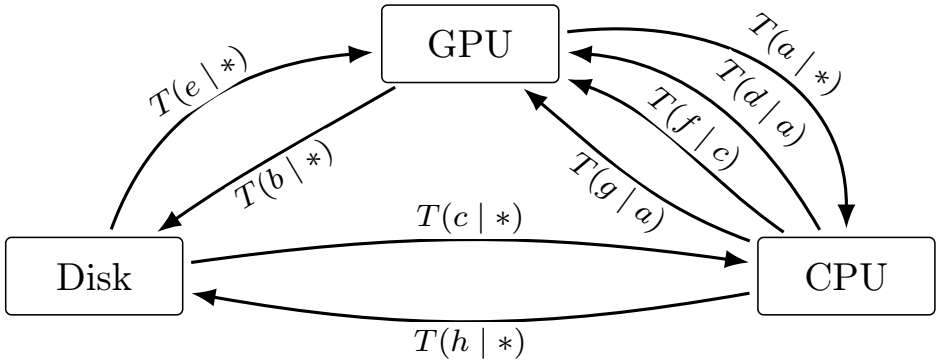

Figure 3.9: State-transition model for model placement across disk, CPU memory, and GPU memory. Each transition $T(x \mid y)$ denotes action x conditioned on the previous action y, where $*$ indicates that the transition is independent of the previous action. The actions correspond to the system-internal Kairos commands described in Table 3.7.

minutes, reloading weights into GPU memory is typically an operation on the order of seconds, making runtime reconfigurability practical.

vLLM distinguishes between two sleep levels. `L1_SLEEP` offloads the model weights to CPU memory and discards the model's KV cache. `L2_SLEEP` discards both the model weights and the KV cache, keeping only selected auxiliary buffers in CPU memory. In models using rotary position embeddings, these auxiliary buffers may include RoPE-related tensors [55]. Discarding the model weights is possible because they already exist on disk. During the first server start, vLLM downloads the model files and stores them persistently in the local Hugging Face cache directory [28].

`WAKE_UP_FROM_CPU` reloads the previously offloaded model weights from CPU memory back into GPU memory. `WAKE_UP_FROM_DISK` performs the analogous operation from disk to GPU memory. However, this command requires a sequence of vLLM operations. First, Kairos calls `/wake_up`, which lets vLLM allocate the required GPU memory for the model weights and KV cache. Afterwards, Kairos issues a collective RPC call and specifies the method `reload_weights`. This operation loads the model weights into the pre-allocated GPU memory. After reloading the weights, Kairos resets the prefix cache to prevent vLLM from reusing stale cached prefix entries computed before the reload.

The modified vLLM version on which Kairos builds also provides a hybrid strategy between `L1_SLEEP` and `L2_SLEEP`. When Kairos calls `WAKE_UP_PERSISTENT`, vLLM keeps the model weights in CPU memory while reloading them into GPU memory. As a result, when Kairos later initiates `L1_SLEEP`, vLLM recognizes that the model weights already exist in CPU memory. It can therefore discard the GPU-resident weights without copying them back to CPU memory again.

`PREFETCH` combined with `WAKE_UP_FROM_PREFETCH` can be understood as a staged wake-up from `L2_SLEEP`. `PREFETCH` loads the model weights from disk into CPU memory and stores them as pinned memory. During the subsequent `WAKE_UP_FROM_PREFETCH`, Kairos instructs vLLM to load these weights back into GPU memory. The sequence

of operations is almost identical to `WAKE_UP_FROM_DISK`. The difference is that Kairos invokes `reload_weights_from_prefetch` as the corresponding RPC method. Unlike `reload_weights`, this method loads the pinned model weights from CPU memory instead of loading them from disk. Prefetching model weights from disk to CPU memory allows Kairos to move a model closer to the GPU in advance, anticipating that this model may be swapped in in the near future.

`EVICT` allows Kairos to deprioritize a model currently resident in CPU memory by removing its weights from CPU memory. After eviction, the weights remain only on disk, similar to the state reached after `L2_SLEEP`. However, `EVICT` and `L2_SLEEP` are not applicable to quantized models. For such models, vLLM cannot safely restore weights from disk because the checkpoint tensors do not necessarily match the quantization-specific runtime layout that vLLM constructs during initial model loading. As a result, Kairos can place quantized models only in CPU memory or GPU memory during serving. Figure 3.9 illustrates the possible transitions between storage states and their dependencies.

3.5 KairosAPI

The KairosAPI represents the client-facing layer of Kairos. It runs in a separate process from KairosCore and exposes the external HTTP interface through which clients submit inference requests. The process separation is intentional: the API process handles request reception and response delivery, while KairosCore performs internal orchestration, scheduling, monitoring, and interaction with the vLLM runtime. The separation keeps the API layer lightweight and isolates client communication from the system internals.

Clients submit incoming requests to the `/infer` endpoint. Each request follows a fixed schema containing the instruction, the prompt, the reference answer, and fields filled during processing, such as the Kairos-generated answer, correctness information, inference latency, and the model used for serving. The API validates the request format before forwarding the request further into the system. Format validation ensures that KairosCore receives structured input rather than arbitrary JSON objects and that all later components can rely on a consistent request representation. Figure 3.10 illustrates the request schema expected by the KairosAPI, including both the input fields provided by the client and the fields filled during processing.

The KairosAPI also records the arrival timestamp of each request. The API adds this timestamp when the request reaches the `/infer` endpoint and before forwarding the request to KairosCore. The scheduler later uses these timestamps to observe the external request arrival process and forecast idle windows. Recording the timestamp at the API boundary ensures that the scheduler reasons about request arrivals as they enter Kairos.

After preprocessing, the KairosAPI forwards the request to KairosCore through the IPC client. From the perspective of the API endpoint, request forwarding is an asynchronous operation. The endpoint sends the request to the Core and waits until the corresponding response returns. While an individual endpoint invocation waits for its own result, the API process can still accept and forward other requests concurrently. Asynchronous forwarding is essential because the internal processing order inside KairosCore

endpoint: /infer

```
{
  "instruction": "Answer the question using the given context.",
  "prompt": "Context: ... Question: ...",
  "answer": "yes",
  "kairos": null,
  "correct": null,
  "infer_latency_ms": null,
  "active_model": null
}
```

Figure 3.10: Example request object submitted to the KairosAPI. The request contains the input fields required for inference and evaluation, while fields such as the generated answer, correctness value, inference latency, and active model are filled during processing. The `answer` field contains the reference answer from the dataset. The `kairos` field stores the generated answer, and `correct` records whether this generated answer matches the reference answer. The `active_model` field identifies the LLM that served the request.

may differ from the order in which requests arrive, especially when multiple requests are dispatched concurrently to exploit vLLM's continuous batching.

Once KairosCore completes the request, the result returns to the API process through the IPC layer. The KairosAPI then sends this result back to the original client. The returned object preserves the structure of the incoming request but contains the fields filled by Kairos, including the model output, correctness value, inference latency, and active model identifier. In this way, clients interact with a stable HTTP interface while the internal execution remains hidden behind the API boundary.

3.6 Inter-Process Communication

The IPC layer connects the KairosAPI process with the KairosCore process. Since both layers run as separate processes, the API cannot forward requests through direct in-memory function calls. Kairos therefore uses an explicit message-based communication layer that transfers inference requests from the API process to the Core process and returns the corresponding results. This layer forms the boundary between external request handling and internal system orchestration.

Socket Pattern: Kairos implements this communication using ZeroMQ with a DEALER/ROUTER socket pattern [22, 23]. On the API side, the IPC client uses a DEALER socket. On the Core side, the IPC server uses a ROUTER socket. The DEALER socket allows the API-side client to send multiple requests without enforcing a strict send-wait-send pattern. The ROUTER socket receives each message together with a routing identity, which identifies the API-side connection from which the message originated. KairosCore later uses this identity to send the response back to the correct IPC client.

Message Format: Each message sent from the API process to the Core process con-

tains three main fields: a message kind, a request identifier, and the request payload. The message kind specifies the type of message, for example an inference request. The payload contains the structured request object produced by the KairosAPI, including the instruction, prompt, reference answer, and arrival timestamp. The IPC client generates the request identifier and uses it to distinguish concurrent requests within the API process.

Routing and Request Matching: The ZeroMQ routing identity and the Kairos request identifier serve different purposes. The routing identity belongs to the transport layer and allows the ROUTER socket to address the correct IPC client connection. It is not part of the logical request semantics. The request identifier belongs to the Kairos message protocol and allows the API-side IPC client to match a returned response to the pending request that is waiting for it. This separation is necessary because multiple requests may be in flight concurrently, and responses may arrive in a different order than the original requests were sent.

API-side Request Tracking: On the API side, the IPC client maintains a mapping of pending requests. When it sends a request to KairosCore, it creates a future object and stores it under the generated request identifier. The API endpoint then waits on this future. When a response later arrives from KairosCore, the IPC client reads the request identifier from the response, looks up the corresponding future, and completes it with the returned result. The original API endpoint invocation can then return the completed result to the external client.

Core-side Message Handling: On the Core side, the IPC server continuously receives messages from the ROUTER socket. For each incoming message, it decodes the JSON payload and forwards the message, together with the ZeroMQ routing identity, to the controller. The controller validates the message kind and payload structure before placing the request into the internal request queue. The IPC server itself does not decide how the request should be processed. Its role is limited to receiving messages, decoding them, and preserving the routing information needed for the response path.

Response Delivery and Concurrency: After KairosCore completes a request, the controller builds a response message containing the original request identifier, a success flag, and either the result or an error message. The IPC server sends this response through the ROUTER socket using the stored routing identity. The API-side IPC client receives the response through its DEALER socket and uses the request identifier to resume the correct pending API request. This design decouples request submission from request completion: the API process can forward multiple requests while earlier requests are still being processed, and KairosCore can dispatch multiple requests concurrently to the active vLLM server. Routing identities and request identifiers together preserve correct response delivery even when completion order differs from arrival order.

3.7 KairosCore

KairosCore contains the internal orchestration logic of Kairos. In the following, we describe the main components that implement this logic. The controller coordinates incom-

ing requests and control commands, while monitoring collects runtime information from served and evaluated requests. The model catalog maintains metadata about the registered models. The reconfiguration component derives valid model-state transitions, and the scheduler decides when reconfiguration and evaluation actions should be executed. Finally, the vLLM interface provides the communication layer between KairosCore and the vLLM model servers.

3.7.1 Controller

The controller is the central coordination component of KairosCore. It receives validated IPC messages, admits inference requests into the internal request queue, and coordinates the asynchronous dispatch of these requests to the active vLLM model server. In parallel, the controller processes control commands issued by the scheduler, such as commands for setting the active model, pausing or resuming request dispatch, and executing model-state transitions. The controller therefore connects the external request path with the internal reconfiguration path and ensures that request serving, monitoring, and command execution are coordinated in one component.

Request Admission: The controller admits incoming inference requests through `handle_ipc_message()`. For each IPC message, the controller first checks whether the message kind corresponds to an inference request and whether the payload is a valid request object. Invalid messages are rejected immediately by sending an error response through the IPC server. For valid requests, the controller records the arrival timestamp with the scheduler, if such a timestamp is present in the payload. The controller also queries the monitor to decide whether the request should be sampled for later evaluation. Finally, the controller wraps the payload, request identifier, IPC routing identity, and sampling decision into a `RequestItem` and places it into the internal request queue.

Concurrent Dispatching: The dispatcher consumes admitted requests from the internal request queue. Before dispatching a request, the controller waits until dispatching is enabled. The dispatch gate allows other components to temporarily pause request forwarding during selected control operations. Once dispatching is enabled, the controller acquires a semaphore that limits the number of concurrent in-flight requests. The dispatcher then creates an asynchronous task for the request and immediately returns to the queue. Each task sends the request to the active vLLM model server through the vLLM interface, completes the result object with the generated answer, correctness value, inference latency, and active model identifier, and returns the response through the IPC server. If the request was marked for sampling during admission, the task also forwards the original payload and completed result to the monitor. Creating separate asynchronous tasks allows multiple requests to be processed concurrently and exposes overlapping requests to vLLM, enabling continuous batching at the model server.

Control-command Execution: The controller also runs a separate control loop that consumes commands from the internal control queue. These commands represent system-level actions such as starting or stopping model servers, moving models between storage states, pausing or resuming request dispatch, and setting the active model. For each command, the controller invokes the corresponding Docker or vLLM interface oper-

ation and updates the affected model metadata when the command changes the model's storage state. Some commands temporarily disable dispatching before they execute, for example when the currently serving baseline model is moved out of the active GPU state. The control loop therefore provides the execution path through which reconfiguration decisions take effect in the running system.

Model Evaluation: The controller also provides methods for evaluating candidate models on sampled requests. For a single sample, the controller sends the instruction and prompt to the selected model through the vLLM interface and computes the same result fields as in normal inference, including the generated answer, correctness value, inference latency, and model identifier. For a set of samples, the controller evaluates requests concurrently under a configurable per-model concurrency limit. The controller can also evaluate multiple models on the same sample set in parallel, returning the results grouped by model. The monitoring component can use these grouped results to compare candidate models on identical sampled requests.

3.7.2 Monitoring

The monitoring component implements the continuous monitoring principle (P1) of Kairos. The controller forwards sampled requests to the monitor together with the result produced by the active model. The monitor stores these sampled requests in a bounded sample window and later receives evaluation results for candidate models on the same sample set. Once the expected model results are available, the monitor computes the model-level SLIs used by Kairos, namely accuracy relative to the base model and p95 inference latency. The resulting statistics are stored in the model catalog and provide the basis for subsequent reconfiguration decisions.

Sample collection. Kairos does not evaluate every incoming request, because full evaluation of all models on all requests would introduce unnecessary overhead. Instead, the controller marks requests for sampling at a configurable rate and forwards the selected requests to the monitor after inference completes. The user can set both the sampling rate and the size of the rolling sample window in the configuration file, as shown in Figure 3.1. For each sampled request, the monitor receives the original input payload together with the result produced by the active model. The payload is stored in a bounded rolling sample window of fixed size. Once enough samples have been collected, the monitor creates an evaluation snapshot from the current window. The evaluation snapshot fixes the inputs used for the next model comparison, ensuring that all candidate models are evaluated on the same sampled requests.

Evaluation handling. The monitor separates the sampled inputs from the model outputs used to evaluate them. During the first monitoring round, the baseline model is active by construction. The monitor can therefore reuse the results produced during normal serving as the baseline outputs for the first evaluation snapshot instead of evaluating the baseline model again. Candidate models are evaluated on the same frozen snapshot and submit their results back to the monitor once evaluation completes. For later rounds, all relevant model results arrive through explicit evaluation events. The monitor records these results per model and waits until every expected model has reported results for

the current round. Only then does the monitor compute the SLIs for the round. The monitor then stores the resulting accuracy and latency values in the model catalog and records whether each model satisfies the configured SLOs.

Monitoring Policies: The monitoring component also includes policy parameters that control which model variants remain part of future evaluation rounds. The discard policy removes a model from the evaluation set after it fails to satisfy the configured SLOs for D consecutive monitoring rounds. The recycle policy allows a discarded model to become a candidate again after it has been excluded for R monitoring rounds, which can be useful when the request distribution changes over time. The monotonicity policy applies only to quantized models. Under this policy, if a quantized model fails to satisfy the SLOs, the monitor can skip the evaluation of more aggressively quantized variants of the same base model, for example skipping INT4 after INT8 already fails. The user can configure the discard threshold, recycle interval, and monotonicity policy in the configuration file, as shown in Figure 3.1.

3.7.3 Model Catalog

The model catalog is the central registry of all models known to KairosCore. It stores one model object for each configured model and makes this metadata available to the other Core components. Components such as monitoring, reconfiguration, scheduling, and the controller use the catalog to retrieve model identifiers, server ports, model relations, storage locations, resource estimates, and evaluation statistics. The catalog therefore provides a shared view of the model pool and keeps model-specific state in one place instead of duplicating it across components.

Each catalog entry contains the information required to identify, serve, evaluate, and place a model at runtime. The model identifier and port define how Kairos reaches the corresponding vLLM server. The relation field distinguishes between the base model, quantized variants, and independent alternatives. Additional metadata describes the model size, compute data type, quantization method, and quantization format when applicable. The catalog also stores the current storage location of each model, which can be disk, CPU memory, or GPU memory, as well as an estimated GPU memory allocation derived from the model size and the configured GPU factor.

3.7.4 Reconfiguration

The reconfiguration component realizes runtime reconfigurability (P2) by translating monitoring results into concrete changes to the serving configuration. After each monitoring round, the monitoring component triggers reconfiguration with the current set of sampled requests and the updated model statistics for accuracy and latency. Given these measurements, the current placement of all registered models, and the configured SLOs, reconfiguration determines how the system should adapt. The component decides which model should serve future requests, which additional models should remain close to the GPU, and which models should be evaluated on the collected samples. The output is a

sequence of Kairos control commands that transforms the current model placement into the desired placement while respecting the available memory capacity.

Problem Statement

Kairos models reconfiguration as a transition between system states. Let $\mathcal{M}$ denote the set of models, and let $\mathcal{L} = \{\text{Disk}, \text{CPU}, \text{GPU}\}$ denote the set of memory locations. At any point in time, the placement function

$$p : \mathcal{M} \to \mathcal{L} \tag{3.1}$$

assigns each model to one memory location. For example, $p(m_1) = \text{CPU}$, while $p(m_2) = \text{GPU}$. In addition, Kairos maintains a dedicated active model m_{active}, which denotes the model currently used for serving incoming requests. The active model is not treated as a separate placement. Instead, activity is represented as a dedicated role assigned to one model. This separation keeps memory placement and request routing conceptually distinct.

Further, let $\mathcal{S} = \{s_1, \ldots, s_n\}$ denote the set of sampled requests, and let $\mathcal{T}$ denote the set of placement transitions supported by Kairos. A transition $\tau \in \mathcal{T}$ describes a legal movement of a model between memory locations. We define a command $c = (m, \tau)$ as the application of transition τ to model m. For example, the command `L1_SlEEP`(m) applies the L1-sleep transition to model m and moves it from GPU to CPU memory. Section 3.4 describes the concrete set of commands implemented by Kairos.

Applying a command transforms the current system state into a new system state. Let Σ denote the set of possible system states. A state $\sigma \in \Sigma$ is represented by the triple $\sigma = (\mathcal{M}, \mathcal{L}, p)$ and is valid if the placement induced by p satisfies the CPU and GPU memory constraints. Thus, a valid command can be viewed as a state transition:

$$c : \Sigma \to \Sigma. \tag{3.2}$$

Given the current system state σ, the active model m_{active}, and the sampled requests $\mathcal{S}$, the reconfiguration problem is to compute a command sequence $C = (c_1, \ldots, c_k)$ that moves Kairos into a better serving configuration. A configuration is considered better if it selects a model that satisfies the accuracy and latency objectives whenever possible, keeps useful alternative models close to the GPU for future adaptation, and schedules evaluations on $\mathcal{S}$ to update the statistics used in later rounds. The command sequence must be feasible in the sense that every command c_i is valid in the state in which it is applied, and every intermediate state produced by the sequence remains in Σ.

Model Scoring

The first step of the reconfiguration algorithm is to define a utility function

$$\text{score} : \mathcal{M} \to \mathbb{R}^+ \tag{3.3}$$

that assigns each model a score based on its observed behavior on the sampled requests $\mathcal{S}$. For each model $m \in \mathcal{M}$, Kairos maintains an accuracy estimate a_m and a latency

estimate ℓ_m. The accuracy estimate measures how well the model performs on $\mathcal{S}$ relative to the base model m_{base}, while the latency estimate captures the observed serving latency of the model on the same samples. The reconfiguration objective is defined by an accuracy SLO a_{slo} and a latency SLO ℓ_{slo}. A model is feasible if it satisfies both objectives:

$$a_m > a_{\text{slo}} \qquad \text{and} \qquad \ell_m < \ell_{\text{slo}}. \tag{3.4}$$

Kairos ranks feasible and infeasible models separately. Feasible models are preferred because they satisfy the configured SLOs. Among feasible models, Kairos rewards both accuracy above the required threshold and latency below the required threshold. To make both quantities comparable, the deviations are normalized by the corresponding SLO values:

$$s_{\text{acc}}(m) = \frac{a_m - a_{\text{slo}}}{a_{\text{slo}}}, \tag{3.5}$$

$$s_{\text{lat}}(m) = \frac{\ell_{\text{slo}} - \ell_m}{\ell_{\text{slo}}}. \tag{3.6}$$

The score of a feasible model is then defined as

$$\text{score}(m) = w_{\text{acc}} \cdot s_{\text{acc}}(m) + w_{\text{lat}} \cdot s_{\text{lat}}(m) + K, \tag{3.7}$$

where w_{acc} and w_{lat} control the relative importance of accuracy and latency, and K is a large constant offset (e.g., $K = 1000$). The offset ensures that every feasible model receives a higher score than any infeasible model.

Infeasible models are not discarded immediately, because their statistics may still be useful for later reconfiguration rounds. Instead, Kairos ranks them by the magnitude of their SLO violation. The normalized accuracy and latency violations are defined as

$$v_{\text{acc}}(m) = \frac{\max(0, a_{\text{slo}} - a_m)}{a_{\text{slo}}}, \tag{3.8}$$

$$v_{\text{lat}}(m) = \frac{\max(0, \ell_m - \ell_{\text{slo}})}{\ell_{\text{slo}}}. \tag{3.9}$$

The weighted violation is

$$v(m) = w_{\text{acc}} \cdot v_{\text{acc}}(m) + w_{\text{lat}} \cdot v_{\text{lat}}(m). \tag{3.10}$$

Kairos assigns infeasible models the score

$$\text{score}(m) = \frac{1}{1 + v(m)}. \tag{3.11}$$

This score decreases as the violation increases. As a result, infeasible models that are close to satisfying the objectives are ranked above models that violate the objectives more strongly, while all infeasible models remain below feasible models due to the offset K used for feasible scores.

The resulting scores provide an ordering over model variants. This ordering does not by itself determine the final memory placement. Instead, it serves as the utility signal for the subsequent placement step, which decides which model should become active and which additional models should remain close to the GPU under the available memory constraints.

Model Placement

After assigning scores to all models, Kairos determines a target placement for the next serving configuration. The goal of this step is to decide which model should become active, which additional models should remain close to the GPU, and which models can be placed on lower memory tiers. The placement step treats the model scores as utility values and selects the highest-value set of models that fits within the available GPU and CPU memory budgets.

Let $\text{score}(m)$ denote the score of model m. Kairos first selects the highest-scoring model as the target active model:

$$m^\star = \arg\max_{m \in \mathcal{M}} \text{score}(m). \tag{3.12}$$

The selected model $m^\star$ is assigned to GPU because it is the target model for future serving. Once the corresponding reconfiguration commands have been executed and request routing is redirected, $m^\star$ becomes the new active model m_{active}. The remaining GPU capacity is then used to keep additional high-value models close to the active model. These models are not necessarily selected for immediate serving, but keeping them on GPU reduces the cost of future reconfiguration and evaluation on the samples.

For the remaining models $\mathcal{M} \setminus \{m^\star\}$, Kairos solves a placement problem under the GPU memory budget. Let $\mu_{\text{GPU}}(m)$ denote the GPU memory required by model m, and let B_{GPU} denote the available GPU memory after placing $m^\star$. The GPU placement problem is formulated as a $0/1$ knapsack problem:

$$\max_{x_m \in \{0,1\}} \sum_{m \in \mathcal{M} \setminus \{m^\star\}} x_m \cdot \text{score}(m) \quad \text{s.t.} \quad \sum_{m \in \mathcal{M} \setminus \{m^\star\}} x_m \cdot \mu_{\text{GPU}}(m) \leq B_{\text{GPU}}. \tag{3.13}$$

A model with $x_m = 1$ is added to the GPU placement, while a model with $x_m = 0$ remains to be considered for lower memory tiers.

The same idea is applied to CPU memory. Let $\mathcal{R}$ denote the set of models that were not selected for GPU placement. Kairos uses the remaining CPU memory budget to keep useful models prefetched in CPU memory. Let $\mu_{\text{CPU}}(m)$ denote the CPU memory required by model m, and let B_{CPU} denote the available CPU memory. The CPU placement problem is

$$\max_{y_m \in \{0,1\}} \sum_{m \in \mathcal{R}} y_m \cdot \text{score}(m) \quad \text{s.t.} \quad \sum_{m \in \mathcal{R}} y_m \cdot \mu_{\text{CPU}}(m) \leq B_{\text{CPU}}. \tag{3.14}$$

Models selected by the CPU placement remain in CPU memory, while all remaining models are assigned to disk. The placement decisions define a target placement $p^\star : \mathcal{M} \to \mathcal{L}$, where $p^\star(m^\star) = \text{GPU}$ and every other model is assigned to GPU, CPU, or disk according to the selected placement.

Algorithm 1 summarizes the placement procedure. The algorithm separates the selection of the active model from the placement of additional models. This separation ensures that the serving decision is driven directly by the model ranking, while the remaining memory capacity is used to keep other high-scoring models on GPU or CPU.

Algorithm 1 Model Placement

Input: Model set $\mathcal{M}$, scores $\{\text{score}(m)\}_{m\in\mathcal{M}}$, GPU budget B_{GPU}, CPU budget B_{CPU}
Output: Target placement $p^\star$, target active model $m^\star$

$m^\star \leftarrow \arg\max_{m\in\mathcal{M}} \text{score}(m)$
$p^\star(m^\star) \leftarrow \text{GPU}$
$\mathcal{R} \leftarrow \mathcal{M} \setminus \{m^\star\}$
$\mathcal{M}_{\text{GPU}} \leftarrow \textsc{Knapsack}(\mathcal{R}, \text{score}, \mu_{\text{GPU}}, B_{\text{GPU}})$
for all $m \in \mathcal{M}_{\text{GPU}}$ **do**
    $p^\star(m) \leftarrow \text{GPU}$
end for
$\mathcal{R} \leftarrow \mathcal{R} \setminus \mathcal{M}_{\text{GPU}}$
$\mathcal{M}_{\text{CPU}} \leftarrow \textsc{Knapsack}(\mathcal{R}, \text{score}, \mu_{\text{CPU}}, B_{\text{CPU}})$
for all $m \in \mathcal{M}_{\text{CPU}}$ **do**
    $p^\star(m) \leftarrow \text{CPU}$
end for
$\mathcal{R} \leftarrow \mathcal{R} \setminus \mathcal{M}_{\text{CPU}}$
for all $m \in \mathcal{R}$ **do**
    $p^\star(m) \leftarrow \text{Disk}$
end for
return $p^\star, m^\star$

The subroutine Knapsack returns the highest-scoring subset of candidate models whose memory requirements fit within the given budget as defined in Equations 3.13 and 3.14.

Reconfiguration Planning

The previous placement step computes a target placement $p^\star$, but it does not specify how the system should reach this placement. Reconfiguration planning fills this gap. Given the current state σ_0 and the target placement $p^\star$, the planner computes a command sequence $C = (c_1, \ldots, c_k)$ that transforms the current state into a state whose placement matches $p^\star$. The planner must ensure that every command is legal in the state in which it is applied and that every intermediate state remains in Σ.

Kairos treats this as a graph-search problem over system states. Each node in the graph is a state $\sigma \in \Sigma$. Each edge corresponds to a command $c = (m, \tau)$ that applies a placement transition $\tau \in \mathcal{T}$ to a model $m \in \mathcal{M}$. Executing the command produces a successor state σ'. Since only states in Σ are considered, the search never explores states that violate memory constraints. The planning objective is to find a path from the current state σ_0 to any state whose placement equals the target placement $p^\star$.

Kairos uses breadth-first search for this planning step. Breadth-first search explores command sequences in increasing length and therefore returns a sequence with the minimum number of placement commands, assuming that all commands have equal cost. This search strategy is appropriate because Kairos manages only a small number of registered models, while avoiding unnecessary movement commands is important for reducing re-

Algorithm 2 Reconfiguration Planning

Input: Current state σ_0, target placement $p^\star$, target active model $m^\star$, transition set $\mathcal{T}$
Output: Command sequence C

```
Q ← empty FIFO queue
V ← σ_0
PUSH(Q, (σ_0, ()))
while Q is not empty do
    (σ, C) ← POP(Q)
    if placement(σ) = p* then
        C ← C ∘ (SETACTIVE(m*))
        return C
    end if
    for all (σ', c) ∈ SUCCESSORS(σ, T) do
        if σ' ∉ V then
            V ← V ∪ {σ'}
            PUSH(Q, (σ', C ∘ (c)))
        end if
    end for
end while
return failure
```

configuration latency.

Once the target placement has been reached, Kairos must also update request routing. The planner appends the control command SETACTIVE($m^\star$), which marks the target model $m^\star$ as the new active model used for serving requests. Unlike placement-transition commands, SETACTIVE does not move model weights between memory locations. The command only changes the model selected by the dispatcher for future requests.

Algorithm 2 summarizes the planning procedure. The subroutine SUCCESSORS encodes the transition constraints of the runtime. For a state σ, the method returns all state-command pairs (σ', c) that can be reached by applying one legal placement command in σ. A command is legal only if its preconditions are satisfied in the current state and the resulting state σ' remains in Σ.

Evaluation Planning

After the serving placement has been reached, Kairos schedules evaluations on the sampled requests $\mathcal{S}$. This step is necessary because the model scores used in later reconfiguration rounds depend on how the registered models perform on the current workload. Evaluation planning is therefore separate from serving placement. Serving placement decides which model should handle future requests, while evaluation planning decides how the remaining models are evaluated on $\mathcal{S}$.

An evaluation command does not change the memory placement of a model. Instead, it evaluates one or more GPU-resident models on the sampled requests. We write this

command as EVALUATE($\mathcal{B}, \mathcal{S}$), where $\mathcal{B} \subseteq \mathcal{M}$ is the batch of models evaluated together. Evaluating multiple models in one batch is useful when several candidate models are already on GPU or can be placed on GPU at the same time.

Kairos evaluates models in rounds. At the beginning of each evaluation round, the algorithm first checks which unevaluated models are already GPU-resident. If such models exist, they can be evaluated immediately. Otherwise, Kairos selects a batch of unevaluated models that can be placed on GPU together, constructs a temporary target placement for that batch, and invokes the reconfiguration planner to produce the required placement commands. Separating evaluation planning from placement-command generation keeps evaluation planning independent of the concrete model movement details. The evaluation planner decides which models should be evaluated together, while the reconfiguration planner decides how to move the system into the required placement.

During the initial monitoring round, the base model has already produced the reference outputs while serving the sampled requests. Therefore, Kairos can mark the base model as already evaluated and only schedules evaluation commands for the remaining model variants. In later rounds, all relevant model variants can be considered for evaluation, depending on the current monitoring policy.

Algorithm 3 summarizes the evaluation planning procedure. The set $\mathcal{E}$ contains models that have already been evaluated on $\mathcal{S}$, while $\mathcal{U}$ contains the remaining unevaluated models. The subroutine GPUMODELS returns the models currently placed on GPU

Algorithm 3 Evaluation Planning

Input: Current state σ, model set $\mathcal{M}$, sampled requests $\mathcal{S}$, transition set $\mathcal{T}$, initially evaluated models $\mathcal{E}_0$
Output: Evaluation command sequence C_{eval}

$C_{\text{eval}} \leftarrow ()$
$\mathcal{E} \leftarrow \mathcal{E}_0$
$\mathcal{U} \leftarrow \mathcal{M} \setminus \mathcal{E}$
while $\mathcal{U} \neq \emptyset$ **do**
    $\mathcal{B} \leftarrow \mathcal{U} \cap \text{GPUMODELS}(\sigma)$
    if $\mathcal{B} = \emptyset$ **then**
        $\mathcal{B} \leftarrow \text{SELECTEVALUATIONBATCH}(\sigma, \mathcal{U})$
        $p_{\mathcal{B}} \leftarrow \text{TEMPORARYPLACEMENT}(\sigma, \mathcal{B})$
        $C_{\text{move}} \leftarrow \text{PLANPLACEMENT}(\sigma, p_{\mathcal{B}}, \mathcal{T})$
        $C_{\text{eval}} \leftarrow C_{\text{eval}} \circ C_{\text{move}}$
        $\sigma \leftarrow \text{APPLY}(C_{\text{move}}, \sigma)$
    end if
    $C_{\text{eval}} \leftarrow C_{\text{eval}} \circ (\text{EVALUATE}(\mathcal{B}, \mathcal{S}))$
    $\mathcal{E} \leftarrow \mathcal{E} \cup \mathcal{B}$
    $\mathcal{U} \leftarrow \mathcal{M} \setminus \mathcal{E}$
end while
return C_{eval}

in state σ. The subroutine SelectEvaluationBatch chooses a non-empty subset of unevaluated models that should be evaluated in the next round. The subroutine TemporaryPlacement constructs a target placement in which the selected batch is GPU-resident.

Reconfiguration Algorithm

We previously introduced the individual building blocks of reconfiguration algorithm. Model scoring assigns a utility value to each model, model placement computes the target placement and target active model, reconfiguration planning derives the command sequence needed to reach this placement, and evaluation planning schedules model evaluations on the sampled requests. The complete reconfiguration algorithm combines these steps into one procedure.

Kairos distinguishes the initial monitoring round from later rounds. In the initial round, the base model has already served the sampled requests and therefore provides the reference outputs for the other models. Since no evaluation statistics for the remaining models are available yet, Kairos does not perform model scoring or placement selection in this round. Instead, it only schedules evaluation commands for the non-base models. In later rounds, Kairos uses the available model statistics to compute scores, selects a new target placement, plans the required placement transitions, and schedules evaluations for future reconfiguration decisions.

Algorithm 4 summarizes the complete procedure. The algorithm returns a command sequence C that can be submitted to the scheduler for execution.

Algorithm 4 Reconfiguration Algorithm

Input: Current state σ_0, model set $\mathcal{M}$, sampled requests $\mathcal{S}$, transition set $\mathcal{T}$, monitoring round r, GPU budget B_{GPU}, CPU budget B_{CPU}
Output: Command sequence C

$C \leftarrow ()$
if $r = 0$ **then**
  $\mathcal{E}_0 \leftarrow m_{\text{base}}$
  $C_{\text{eval}} \leftarrow \text{EvaluationPlanning}(\sigma_0, \mathcal{M}, \mathcal{S}, \mathcal{T}, \mathcal{E}_0)$
  $C \leftarrow C_{\text{eval}}$
  return C
end if
score $\leftarrow \text{ModelScoring}(\mathcal{M}, \mathcal{S}, B_{\text{GPU}}, B_{\text{CPU}})$
$(p^\star, m^\star) \leftarrow \text{ModelPlacement}(\mathcal{M}, \text{score})$
$C_{\text{place}} \leftarrow \text{ReconfigurationPlanning}(\sigma_0, p^\star, m^\star, \mathcal{T})$
$\sigma^\star \leftarrow \text{Apply}(C_{\text{place}}, \sigma_0)$
$\mathcal{E}_0 \leftarrow \emptyset$
$C_{\text{eval}} \leftarrow \text{EvaluationPlanning}(\sigma^\star, \mathcal{M}, \mathcal{S}, \mathcal{T}, \mathcal{E}_0)$
$C \leftarrow C_{\text{place}} \circ C_{\text{eval}}$
return C

Time Complexity

Let $n = |\mathcal{M}|$ denote the number of registered models, let $t = |\mathcal{T}|$ denote the number of placement-transition types, and let $s = |\Sigma|$ denote the number of valid system states considered by the planner. The following analysis considers the computational cost of planning a reconfiguration. It does not include the runtime cost of executing model movement commands or evaluating models on the sampled requests.

Model scoring visits each model once and computes a constant number of arithmetic operations per model. Therefore, the scoring step has time complexity

$$\mathcal{O}(n). \tag{3.15}$$

Model placement first selects the highest-scoring model, which requires $\mathcal{O}(n)$ time. The remaining placement decisions are formulated as 0/1 knapsack problems for GPU and CPU memory. A standard dynamic-programming solution for a 0/1 knapsack problem with n candidate models and a discretized memory budget B has pseudo-polynomial complexity $\mathcal{O}(nB)$. Applied to GPU and CPU placement, the complexities are

$$\mathcal{O}(nB_{\text{GPU}}) \qquad \text{and} \qquad \mathcal{O}(nB_{\text{CPU}}), \tag{3.16}$$

respectively.

In Kairos, the knapsack problem is implemented using a sparse dynamic program rather than a dense table over all memory values up to the budget. The sparse dynamic program stores only memory usages that are reachable by selecting some subset of the models considered so far. For example, if the considered models have memory costs 2, 5, and 7, then the reachable memory usages are subset sums such as 0, 2, 5, 7, 9, 12, and 14, rather than every value between 0 and the budget. Let h_{GPU} and h_{CPU} denote the maximum number of reachable memory usages stored during the GPU and CPU knapsack computations. One sparse knapsack call then costs $\mathcal{O}(nh)$, where h is the number of stored reachable memory usages. Therefore, the sparse placement complexity is

$$\mathcal{O}(nh_{\text{GPU}} + nh_{\text{CPU}}). \tag{3.17}$$

Since each reachable memory usage corresponds to at least one subset of candidate models, $h_{\text{GPU}} \leq 2^n$ and $h_{\text{CPU}} \leq 2^n$. In practice, the number of registered models is small, and the number of reachable memory usages is much smaller than the raw memory budgets measured in bytes.

Reconfiguration planning performs BFS over the state graph. In the worst case, BFS visits every state in Σ. For each visited state, the planner may consider applying each transition type to each model, resulting in at most nt candidate commands per state. Assuming that successor validity is checked in constant time using maintained memory information, the worst-case time complexity is

$$\mathcal{O}(snt). \tag{3.18}$$

The space complexity is

$$\mathcal{O}(s), \tag{3.19}$$

because BFS stores the set of already visited states and the queue of states that still need to be explored. Since each model can be assigned to one of $|\mathcal{L}|$ locations, the number of possible states is bounded by $s \leq |\mathcal{L}|^n$ before memory constraints are applied. Thus, the state space can grow exponentially with the number of models. However, Kairos operates on a small set of models, which keeps the reachable state space manageable in practice.

Evaluation planning proceeds in rounds. In each round, Kairos selects a non-empty batch of unevaluated models that can be evaluated together, computes the temporary placement required for this batch, and invokes the reconfiguration planner to reach this placement. Since each round evaluates at least one previously unevaluated model, there can be at most n evaluation rounds.

Selecting an evaluation batch is again a knapsack-style packing problem over the remaining unevaluated models. As in model placement, Kairos uses a sparse dynamic program that stores only reachable memory usages. Let h_{eval} denote the maximum number of reachable memory usages stored during one evaluation-batch selection. One batch-selection step therefore costs

$$\mathcal{O}(nh_{\text{eval}}). \tag{3.20}$$

For each evaluation round, Kairos may also call the reconfiguration planner. From the previous paragraph, one planning call costs $\mathcal{O}(s \cdot n \cdot t)$. Therefore, the worst-case time complexity of evaluation planning is

$$\mathcal{O}\big(sn^2t + n^2h_{\text{eval}}\big). \tag{3.21}$$

This bound describes the planning cost only. It does not include the runtime cost of executing the generated placement commands or evaluating the models on the samples.

Combining the individual steps, the regular reconfiguration round is dominated by evaluation planning. Model scoring requires only $\mathcal{O}(n)$ time, model placement requires $\mathcal{O}(nh_{\text{GPU}} + nh_{\text{CPU}})$ time, and the initial reconfiguration-planning call requires $\mathcal{O}(snt)$ time. Evaluation planning may perform up to n evaluation rounds. In each round, it may select an evaluation batch using sparse dynamic programming and invoke the reconfiguration planner to reach the corresponding temporary placement. Therefore, the overall worst-case planning complexity of a regular reconfiguration round is given by Equation 3.21.

3.7.5 Scheduler

The scheduler coordinates the execution of command sequences generated by the reconfiguration component. Each command carries an interference flag that indicates whether execution may affect active request serving. Non-interfering commands are forwarded directly to the controller, while interfering commands are delayed until the workload is predicted to enter a sufficiently idle window. The scheduler therefore realizes the interference-aware scheduling principle (P3) by aligning disruptive adaptation actions with short periods of low load.

To make this decision, Kairos maintains a history of recent request arrival timestamps, aggregates them into per-second request counts, and applies a configurable forecasting method to estimate the request volume over the next scheduling window. If the predicted number of requests remains below a configurable threshold, the scheduler releases the command to the controller. Otherwise, the scheduler waits and repeats the prediction. The scheduler preserves the order of reconfiguration commands while reducing the impact of runtime adaptation on user-facing inference. The user can configure both the forecasting method and the forecasting window size in the configuration file, as shown in Figure 3.1. We first describe the individual forecasting methods before explaining how the scheduler uses them to decide when interfering commands should be released.

Moving Average

The moving average forecaster serves as a simple baseline for short-term workload prediction. Let x_t denote the number of requests observed in time bucket t, where Kairos uses one-second buckets. Given a lookback window of length L, the method estimates the recent request rate by averaging the last L observations:

$$\bar{x}_t = \frac{1}{L} \sum_{j=t-L+1}^{t} x_j \tag{3.22}$$

The forecast for each of the next W seconds is then set to this average:

$$\hat{x}_{t+h} = \bar{x}_t, \qquad h = 1, \ldots, W. \tag{3.23}$$

Here, W denotes the forecasting horizon passed to the method by the scheduler. The lookback length is chosen as $L = 2W$, so the estimate is based on a short recent history while still smoothing out individual request spikes. Since the moving average produces a constant forecast over the prediction horizon, it cannot model trends or periodic behavior, but it provides a transparent baseline for estimating short-term request load.

Exponentially Weighted Moving Average

The Exponentially Weighted Moving Average (EWMA) forecaster estimates the short-term request volume by smoothing the observed request counts over time. In contrast to the simple moving average, EWMA assigns larger weight to recent observations while the influence of older observations decays gradually. Let x_t denote the number of requests observed in time bucket t, and let s_t denote the smoothed request estimate at time t. Given a smoothing factor $\alpha \in (0, 1]$, the estimate is updated as

$$s_t = \alpha x_t + (1 - \alpha) s_{t-1}. \tag{3.24}$$

A larger value of α makes the estimate more responsive to recent changes in the workload, while a smaller value produces a smoother estimate that reacts more slowly. The forecast for each of the next W seconds is then given by the current smoothed estimate:

$$\hat{x}_{t+h} = s_t, \qquad h = 1, \ldots, W. \tag{3.25}$$

Here, W denotes the forecasting horizon considered by the scheduler. Since EWMA produces a constant forecast over the prediction horizon, it does not explicitly model trends or periodic behavior. However, compared to the moving average, it reacts more quickly to recent changes in request load and is therefore a suitable default method for short-term idle-window prediction.

Holt's Linear Trend Method

Holt's linear trend method extends exponential smoothing with an additional trend component. While EWMA estimates only the current request level, Holt's method also estimates whether the request volume is increasing or decreasing. Let x_t denote the number of requests observed in time bucket t, let s_t denote the smoothed level estimate, and let b_t denote the estimated trend. Given smoothing factors $\alpha \in (0, 1]$ and $\beta \in (0, 1]$, the level and trend are updated as

$$s_t = \alpha x_t + (1 - \alpha)(s_{t-1} + b_{t-1}), \tag{3.26}$$

$$b_t = \beta(s_t - s_{t-1}) + (1 - \beta)b_{t-1}. \tag{3.27}$$

The level s_t represents the current smoothed request volume, while the trend b_t captures the estimated change in request volume between consecutive time buckets. The forecast for the next W seconds is then computed by extrapolating the level using the trend:

$$\hat{x}_{t+h} = s_t + hb_t, \qquad h = 1, \ldots, W. \tag{3.28}$$

This allows Holt's method to predict rising or falling request load more effectively than moving average or EWMA. It is therefore useful when the workload changes gradually over time, although it does not explicitly model recurring periodic patterns.

Holt-Winters Seasonal Method

The Holt-Winters seasonal method extends Holt's linear trend method with a seasonal component. While Holt's method models the current request level and a linear trend, Holt-Winters additionally captures recurring deviations from the level, making the method suitable for workloads with repeated phases of high and low request volume.

Let x_t denote the number of requests observed in time bucket t. The method maintains three quantities: the level s_t, the trend b_t, and the seasonal component c_t. The level represents the smoothed request volume, the trend represents the estimated change between consecutive buckets, and the seasonal component represents recurring deviations from the level. Given a seasonal period P and smoothing factors $\alpha, \beta, \gamma \in (0, 1]$, the additive Holt-Winters updates are defined as

$$s_t = \alpha(x_t - c_{t-P}) + (1 - \alpha)(s_{t-1} + b_{t-1}). \tag{3.29}$$

The trend is updated as

$$b_t = \beta(s_t - s_{t-1}) + (1 - \beta)b_{t-1}. \tag{3.30}$$

The seasonal component is updated as

$$c_t = \gamma(x_t - s_t) + (1 - \gamma)c_{t-P}. \tag{3.31}$$

The forecast for the next W seconds combines the current level, the estimated trend, and the seasonal component corresponding to the predicted future bucket:

$$\hat{x}_{t+h} = s_t + hb_t + c_{t+h-P\lceil h/P \rceil}, \qquad h = 1, \ldots, W. \tag{3.32}$$

Holt-Winters is the most expressive of the considered forecasting methods. Moving average and EWMA produce constant forecasts over the prediction horizon, while Holt's method adds a linear trend. Holt-Winters additionally models recurring workload structure, which makes it useful for periodic or on-off traffic patterns. However, the method requires sufficient historical observations to estimate the seasonal component reliably.

Scheduling Algorithm

The forecasting methods described above estimate the expected request volume over a future window of length W. The scheduler uses these forecasts to decide when commands marked as interfering can be released to the controller. Let $\hat{x}_{t+h}$ denote the predicted number of requests in future bucket $t + h$, for $h = 1, \ldots, W$, and let θ denote the idle threshold. The predicted request volume over the scheduling window is defined as

$$\hat{X}_{t,W} = \sum_{h=1}^{W} \hat{x}_{t+h}. \tag{3.33}$$

The scheduler considers the window idle if the predicted request volume does not exceed the threshold:

$$\hat{X}_{t,W} \leq \theta. \tag{3.34}$$

When a command is marked as non-interfering, the scheduler forwards it directly to the controller. For an interfering command, the scheduler first aggregates the recent arrival timestamps into per-second request counts and applies the configured forecasting method. If the predicted window satisfies the idle condition, the command is released. Otherwise, the scheduler waits for the next polling interval and repeats the prediction. Since commands are processed in the order in which they are received from the reconfiguration component, this procedure preserves the command sequence while delaying only those commands whose execution may affect active request serving.

Algorithm 5 summarizes this procedure. The function FORECAST denotes the configured forecasting method, which can be instantiated as moving average, EWMA, Holt's method, or Holt-Winters.

3.7.6 vLLM Interface

The vLLM interface forms the boundary between KairosCore and the modified vLLM runtime. It encapsulates all HTTP-based interactions with vLLM and provides KairosCore

Algorithm 5 Interference-aware Scheduling

Input: Command sequence C, forecasting horizon W, idle threshold θ, polling interval Δ
Output: Commands are forwarded to the controller in order

for each command $c \in C$ **do**
  if c is non-interfering **then**
    Forward c to the controller
  else
    repeat
      Aggregate recent request arrivals into counts $x_1, \ldots, x_t$
      Compute forecast $\hat{x}_{t+1}, \ldots, \hat{x}_{t+W} \leftarrow \textsc{Forecast}(x_1, \ldots, x_t, W)$
      $\hat{X}_{t,W} \leftarrow \sum_{h=1}^{W} \hat{x}_{t+h}$
      if $\hat{X}_{t,W} \leq \theta$ **then**
        Forward c to the controller
      else
        Wait for Δ seconds
      end if
    until c has been forwarded
  end if
end for

with a single internal interface for inference requests and runtime control operations. Instead of exposing vLLM-specific endpoints throughout the Core, the controller accesses vLLM through this interface.

The interface is intentionally not bound to a fixed model. Since Kairos may change the active model during runtime, each operation specifies its target model server through the corresponding model metadata, in particular the model identifier and the port of the vLLM server. Model-specific targeting allows the same interface to send inference requests to the currently active model and to issue control operations to other model servers, for example when they are sleeping, waking up, being prefetched, or evaluated on sampled requests.

For inference, the vLLM interface translates the internal Kairos request representation (see Figure 3.10) into a chat completion request. The `instruction` field is passed as the system message, while the `prompt` field is passed as the user message. Figure 3.11 illustrates the chat completion request structure. The interface then extracts the generated answer from the vLLM response and returns it to KairosCore together with the measured inference latency. KairosCore uses this result to complete the original request object by filling the generated Kairos answer, the correctness value, the inference latency, and the identifier of the model that produced the response.

In addition to inference, the vLLM interface provides the runtime-control operations used during reconfiguration. These operations correspond to the Kairos commands listed in Table 3.7 and cover the HTTP calls needed to move models between memory states

vLLM chat completion request

```
POST http://<host>:<port>/v1/chat/completions

{
  "model": "<model_id>",
  "messages": [
    {
      "role": "system",
      "content": "<instruction>"
    },
    {
      "role": "user",
      "content": "<prompt>"
    }
  ],
  "temperature": <temperature>,
  "max_tokens": <max_tokens>
}
```

Figure 3.11: Example chat completion request sent by the vLLM interface. The request targets the vLLM server associated with the selected model. The `messages` field follows the chat-completion format: the system message carries the instruction that defines the task behavior, while the user message contains the prompt to be answered by the model.

or prepare them for serving and evaluation. Centralizing these operations in the vLLM interface keeps endpoint-specific HTTP logic out of the controller, scheduler, and reconfiguration components.

The interface uses a shared asynchronous HTTP client across its methods. Reusing the same client avoids creating a new client for every inference or control request and allows multiple vLLM requests to remain in flight concurrently. Concurrent vLLM requests matter for two reasons. First, the active model can receive overlapping inference requests, enabling vLLM to exploit continuous batching. Second, Kairos can evaluate additional models or issue control commands while other requests are still being processed. The vLLM interface therefore supports the asynchronous execution model of KairosCore while keeping the concrete communication with vLLM localized in one component.

4 Experiments

This chapter experimentally evaluates Kairos. We divide the evaluation into two parts. The first part, microbenchmarks individual mechanisms and components of the system in isolation. These experiments quantify the costs and behavior of the building blocks introduced in the system architecture, such as scheduling, or vLLM mechanisms used for model placement. The goal of these experiments is not only to measure individual overheads, but also to validate the design decisions on which Kairos builds.

The second part evaluates Kairos as a complete serving system. These end-to-end experiments measure how the system behaves under different request workloads and serving conditions. In contrast to the microbenchmarks, the end-to-end evaluation captures the interaction between monitoring, scheduling, reconfiguration, request dispatching, and vLLM-based inference. Together, both parts provide a layered evaluation: the microbenchmarks explain the behavior of the individual mechanisms, while the end-to-end experiments show how these mechanisms affect the complete system during serving.

4.1 Experimental Setup

We conduct all experiments on a single local workstation. Table 4.1 summarizes the hardware configuration, including the GPU, CPU, system memory, and storage device used for model execution and weight movement. All experiments use a single GPU with 24 GB of on-board memory. The available GPU memory bounds the model sizes considered in the experiments, while the system memory limits how many models can be kept outside the GPU at the same time.

Table 4.1: Hardware Setup.

Component	**Specification**
GPU	NVIDIA RTX A5000
GPU memory	24GB GDDR6
CPU	Intel Core i9-10980XE (18 cores, 36 threads)
System memory	64 GB DDR4 RAM
Storage	1 TB NVMe SSD

Table 4.2 reports the software stack used in the experiments. Kairos runs as a Python application and controls separate vLLM serving instances. The vLLM servers run inside

Docker containers and rely on PyTorch and CUDA for model execution.

Table 4.2: Software Setup.

Component	**Specification**
OS	Ubuntu 22.04.5 LTS
Python	3.12
CUDA	12.9
GPU driver	575.51.03
vLLM	0.13.0
PyTorch	2.9.0
Docker	29.2.1

We use four datasets for inference-related experiments: BoolQ, LogiQA, MMLU, and OpenBookQA. These datasets cover different question-answering formats and task types. Table 4.3 summarizes the datasets and their answer formats.

Table 4.3: Datasets used for inference-related experiments.

Dataset	**Type**
BoolQ	True/false
LogiQA	Multiple choice
MMLU	Multiple choice
OpenBookQA	Multiple choice

- BoolQ [9] is a reading-comprehension dataset consisting of naturally occurring yes/no questions. Each example contains a question, a passage, and a binary answer. The task requires the model to determine whether the passage supports a positive or negative answer to the question.
- LogiQA [35] is a multiple-choice reading-comprehension dataset focused on logical reasoning. The questions are derived from expert-written logical reasoning problems and require deductive reasoning over a given context. Each example contains a passage, a question, several answer options, and one correct choice.
- MMLU [21] is a multiple-choice benchmark covering a broad range of academic and professional subjects. The dataset spans domains such as mathematics, history, computer science, law, and other areas of knowledge. Each example consists of a question with several answer options and a single correct answer.
- OpenBookQA [39] is a multiple-choice science question-answering dataset modeled after open-book exams. The questions are based on elementary-level science facts

and often require applying such facts to new situations. Each example contains a question, several answer options, and one correct answer.

4.2 Microbenchmarks

The microbenchmarks evaluate individual components and mechanisms of Kairos introduced in Chapter 3 and validate the system's architectural design decisions. These experiments cover the effect of quantization on request throughput and accuracy, the cost of moving model weights across memory tiers, batching strategies, multi-model inference, and idle-time prediction for command scheduling.

4.2.1 Model Selection Trade-offs

Kairos aims to improve throughput while satisfying the configured SLOs. Runtime reconfiguration can support this goal only if alternative serving models provide useful trade-offs between inference latency and accuracy. We therefore first study whether quantized variants of the base model and smaller independent models can reduce inference latency while still satisfying the accuracy constraint. These experiments evaluate the models independently of the full system behavior and establish whether runtime model selection has meaningful alternatives to choose from. For each model, we measure two SLIs: accuracy relative to the base model and inference latency under different request rates. Table 4.4 summarizes the model candidates used in this evaluation, including the base model, its quantized variants, and independent alternative models. Since all experiments run on a single GPU, we use the trivial parallelism configuration TP = 1, PP = 1, and DP = 1.

Table 4.4: Model candidates used in the model-selection evaluation. The set includes the BF16 Llama-3.1-8B-Instruct base model, quantized variants of the same model, and independent alternative models with smaller dense or sparse active parameter counts. For OLMoE, the table reports both total parameters and active parameters per token.

Model	**Precision**	**Parameters**
Llama-3.1-8B-Instruct	BF16	8B
Meta-Llama-3.1-8B-Instruct-FP8	FP8	8B
Meta-Llama-3.1-8B-Instruct-quantized.w8a8	INT8	8B
Meta-Llama-3.1-8B-Instruct-quantized.w4a16	INT4	8B
Meta-Llama-3.1-8B-Instruct-bnb-4bit	NF4	8B
OLMoE-1B-7B-0924-Instruct	BF16	7B (1B active)
Qwen2.5-7B-Instruct	BF16	7B
Qwen3-4B-Instruct-2507	BF16	4B

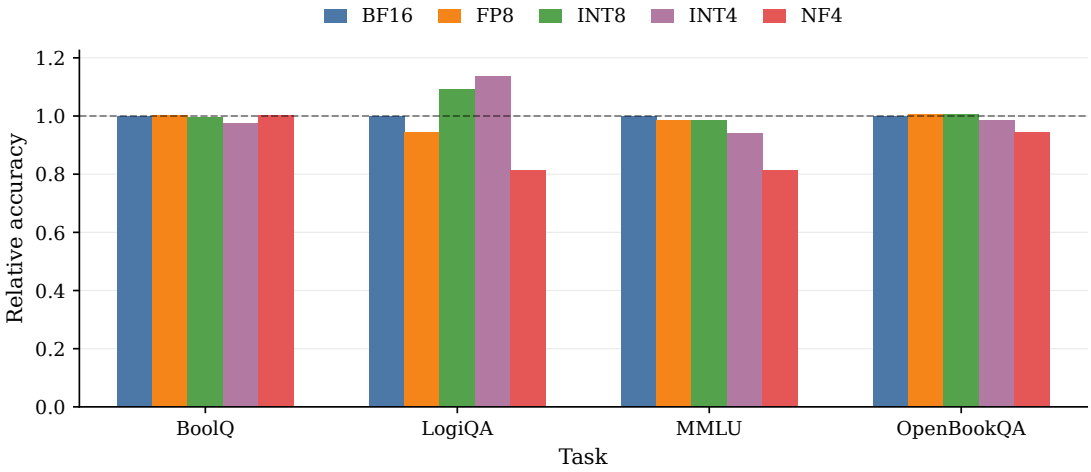

Figure 4.1: Relative accuracy of quantized Llama-3.1-8B-Instruct variants across tasks. The BF16 model serves as the base model, and all values are normalized by its task accuracy on the same dataset. Values below one indicate lower task accuracy than the base model, while values above one indicate that the alternative model outperforms it.

Figure 4.1 shows the relative accuracy of the quantized Llama-3.1-8B-Instruct variants across the four evaluation tasks specified in Table 4.3. The BF16 model serves as the reference point, so its relative accuracy is 1.0 for every task. Overall, the quantized variants remain close to the base model, indicating that lower-precision representations can preserve most of the task accuracy of the original model. This result is important for Kairos because it shows that quantized variants can serve as less resource-intensive alternatives for inference while still satisfying an accuracy SLO.

However, Figure 4.1 also shows that the impact of quantization depends on both the quantization scheme and the task.The task- and scheme-dependent behavior motivates monitoring accuracy at runtime instead of assuming that all variants are equally suitable. For BoolQ and OpenBookQA, all variants remain very close to the baseline. For the other two tasks, however, the NF4 variant shows a considerable accuracy drop. For LogiQA, the INT8 and INT4 variants even outperform the base model.

Figure 4.2 compares independent model alternatives against the Llama-3.1-8B-Instruct base model. The Qwen models perform competitively across all tasks and exceed the base model on several datasets. The advantage is most pronounced on LogiQA, where Qwen2.5-7B-Instruct and Qwen3-4B-Instruct-2507 achieve substantially higher relative accuracy than the base model. The Qwen models also remain above the base model on MMLU and OpenBookQA, while matching it on BoolQ.

In contrast, OLMoE-1B-7B, a mixture-of-experts (MoE) model, remains below the base model on all evaluated tasks. The gap is especially visible on MMLU and LogiQA. Although this MoE model has 7B total parameters, only 1B parameters are active dur-

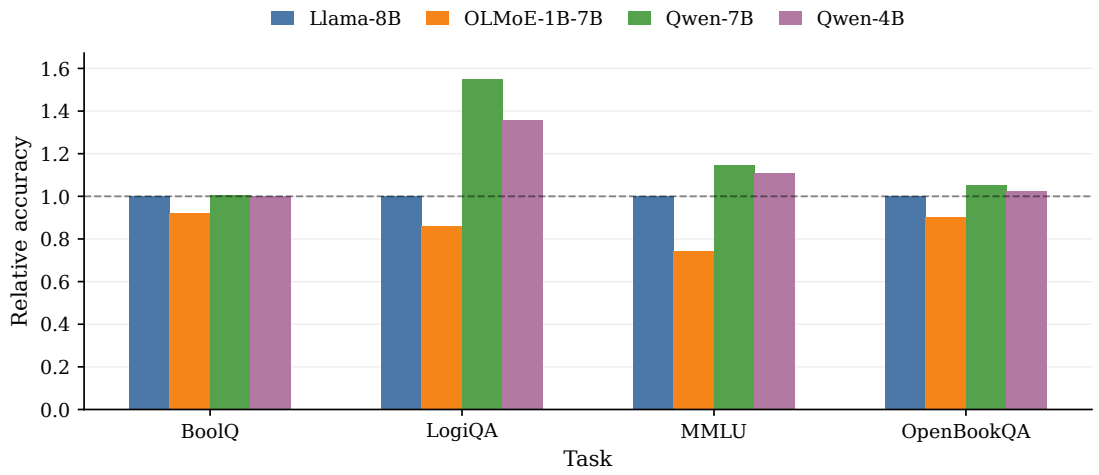

Figure 4.2: Relative accuracy of independent model alternatives across tasks. Llama-3.1-8B-Instruct serves as the base model, and all values are normalized by its task accuracy on the same dataset. Values below one indicate lower task accuracy than the base model, while values above one indicate that the alternative model outperforms the base model.

ing inference, which helps explain the lower relative accuracy. These results show that independent alternatives with fewer parameters can provide useful model-selection candidates, but their suitability strongly depends on the model architecture.

Inference Latency Under Load

Figure 4.3 shows the p95 inference latency of the Llama-3.1-8B-Instruct base model and its quantized variants under request rates of 2, 5, and 10 requests per second. Across all tasks, tail latency generally increases with the offered request rate. The increase is expected because higher request rates create more overlapping requests inside vLLM. Continuous batching can improve GPU utilization, but requests may also spend more time waiting for scheduling iterations or being processed together with other requests in larger batches. As a result, higher load increases p95 latency even when the per-request computation remains unchanged.

The NF4 variant shows the highest p95 latency across all tasks and request rates. The effect is especially visible for LogiQA and MMLU, where tail latency increases strongly as the request rate rises. Although NF4 quantization substantially reduces the model memory footprint, the reduced footprint does not directly translate into lower serving latency in this setup. The bitsandbytes execution path is optimized primarily for memory-efficient model loading and execution, while 4-bit weights require additional dequantization. As a result, the NF4 model reduces memory pressure but performs poorly as a low-latency serving candidate.

The FP8 variant does not substantially reduce p95 latency compared to the BF16 base model. The result is consistent with the hardware support available in the experimental

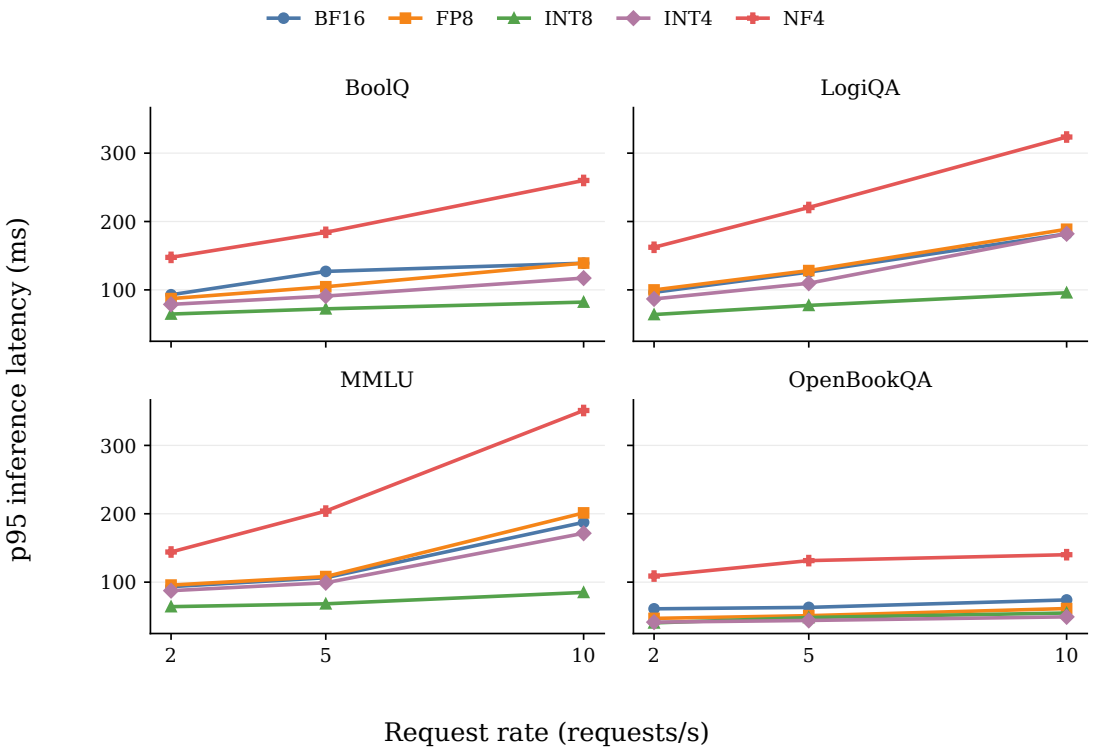

Figure 4.3: p95 inference latency of quantized Llama-3.1-8B-Instruct variants under increasing request rates. Each subplot reports one task, and each point shows the p95 of the inference latency for the corresponding model and offered request rate.

setup. vLLM supports fully hardware-accelerated FP8 weight-and-activation quantization on Hopper and Ada Lovelace GPUs, whereas Ampere GPUs use a weight-only FP8 path through Marlin kernels [59]. Since the experiments run on an Ampere GPU, the FP8 model benefits from reduced weight memory requirements but does not use the same W8A8 execution path available on newer accelerators. As a result, FP8 preserves accuracy well but provides only limited latency improvement in this setup.

INT4 quantization provides latency benefits particularly at low request rates [60]. At the lowest request rate, INT4 achieves p95 latency close to INT8. However, as the offered request rate increases, INT8 provides consistently lower tail latency. The difference arises from the weight-only W4A16 format used by INT4: weights are stored in 4-bit precision, but computation still uses 16-bit activations, requiring unpacking and dequantization before execution. Under higher load, the conversion overhead becomes more noticeable. INT8, in contrast, uses a W8A8 format in which both weights and activations are represented in 8-bit precision, avoiding the same 4-bit unpacking and weight-dequantization overhead.

Figure 4.4 shows the p95 inference latency of the independent model alternatives under increasing request rates. OLMoE-1B-7B-0924-Instruct achieves low tail latency across all

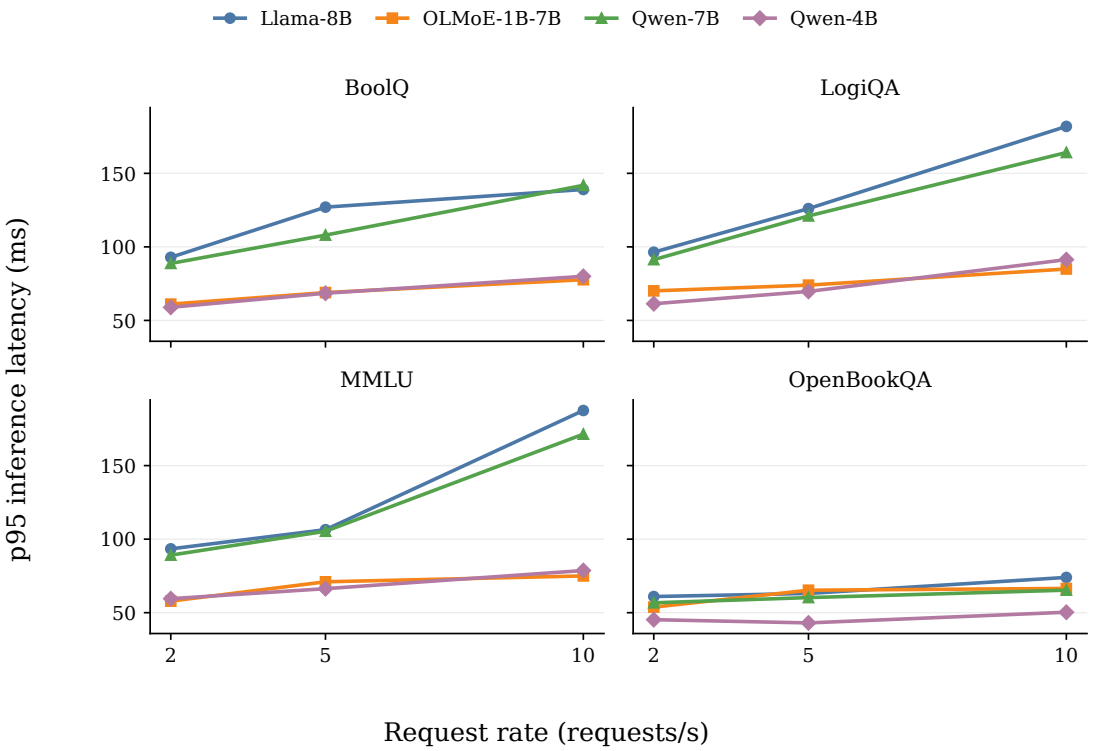

Figure 4.4: p95 inference latency of independent model alternatives under increasing request rates. Each subplot reports one task, and each point shows the p95 of the inference latency for the corresponding model and offered request rate. The Llama-3.1-8B-Instruct model serves as the base model, while OLMoE-1B-7B-0924-Instruct, Qwen2.5-7B-Instruct, and Qwen3-4B-Instruct-2507 represent alternative serving candidates.

tasks and remains close to Qwen3-4B-Instruct-2507, while staying clearly below Llama-3.1-8B-Instruct and Qwen2.5-7B-Instruct in most settings. This behavior is expected because OLMoE is a sparse MoE model with 7B total parameters but only about 1B active parameters per token. The lower active parameter count reduces the amount of computation required during inference and therefore improves latency compared to dense models of similar total size.

Qwen2.5-7B-Instruct follows a latency profile similar to the Llama-3.1-8B-Instruct base model. Across tasks, its p95 latency increases with the offered request rate, indicating that the dense 7B model is affected by higher load in a similar way as the base model. However, Qwen2.5-7B-Instruct is usually slightly faster than Llama-3.1-8B-Instruct, especially on LogiQA and MMLU, where the gap becomes more visible at 10 requests per second. This behavior is expected because Qwen2.5-7B-Instruct has a smaller dense parameter count than the Llama model, while still requiring dense computation for every generated token. As a result, Qwen2.5-7B-Instruct reduces latency compared to the base model, but not as strongly as the smaller Qwen model or the sparse MoE.

Qwen3-4B-Instruct-2507 achieves the lowest latency among the dense models. Its p95 inference latency remains below both the base model and Qwen2.5-7B-Instruct across all tasks and request rates. The difference becomes especially visible at higher request rates, where the larger dense models show stronger latency growth. This behavior is expected because Qwen3-4B-Instruct-2507 performs dense inference with substantially fewer parameters than the 7B–8B models, reducing the computation required per request.

At the same time, Qwen3-4B-Instruct-2507 remains close to OLMoE-1B-7B-0924-Instruct in several settings, despite being a dense model. Thus, Qwen3-4B-Instruct-2507 and similar models are important candidates for Kairos. They provide a strong latency reduction relative to the base model without relying on sparse MoE execution.

Overall, the results show that several model types are suitable candidates for runtime reconfigurability. Among the quantized models, INT8 and INT4 exhibit strong behavior with respect to both accuracy and latency. The FP8 variant also remains close to the base model in terms of accuracy and provides a modest latency improvement, making it suitable for Kairos as well. However, the FP8 results are limited by the GPU used in our experimental setup. The NF4 variant performs poorly in both latency and accuracy and is therefore not considered for end-to-end serving.

Among the independent model alternatives, the Qwen models stand out. Both models match or exceed the base model in terms of accuracy while providing noticeable latency improvements. The 4B model in particular offers the strongest trade-off between accuracy and latency. We therefore focus on similar dense independent models for runtime reconfigurability. The MoE model provides strong latency improvements but does not satisfy the accuracy SLI. As a result, we do not use this model type for serving in the end-to-end evaluation. Larger MoE models with more active parameters may satisfy both SLI requirements, but our experimental setup is constrained by the available GPU memory.

4.2.2 Model State Transitions

In Section 3.4 of the previous chapter, we introduced the Kairos-internal commands used to interact with vLLM model servers. These commands allow Kairos to move model weights across memory tiers while keeping the model server process alive. We evaluate the commands in three groups. First, we compare the sleep-related commands. We then evaluate the commands for waking up model servers and finally analyze eviction policies.

Table 4.5: Llama 3 models across three different sizes. We assume a `gpu_factor` of 1.3 to realistically accommodate for KV cache and activations besides model weights.

Model	**Parameters**	**GPU footprint (MiB)**
Llama-3.2-1B-Instruct	1B	21,894
Llama-3.2-3B-Instruct	3B	9,954
Llama-3.1-8B-Instruct	8B	3,670

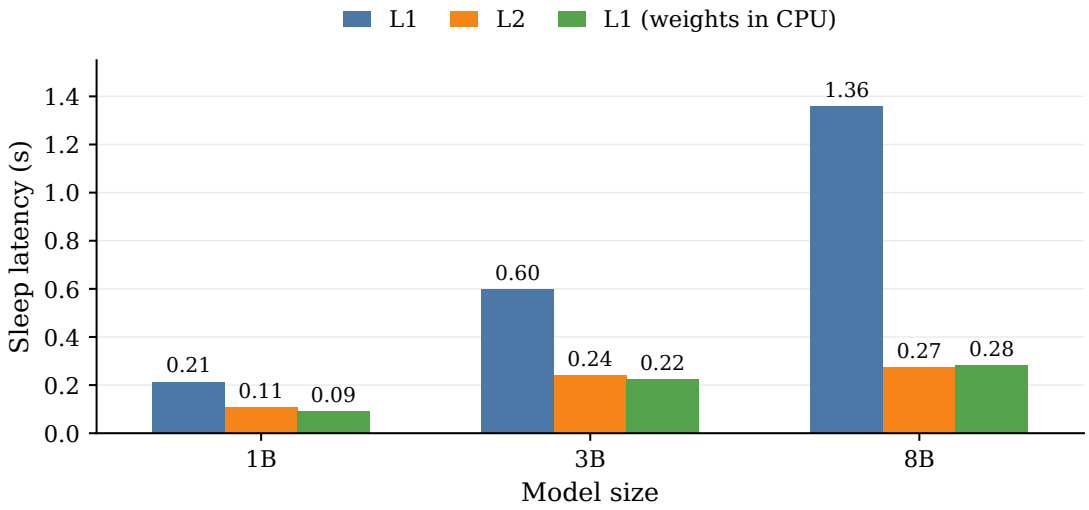

Figure 4.5: Latency of vLLM sleep mechanisms across Llama-3 models of different sizes.

These experiments also verify the usefulness of the commands added to vLLM. Table 4.5 lists the models used for these experiments. We rely on models of different sizes from the Llama-3 model family [20] to evaluate how the command costs change with increasing model size and GPU memory footprint.

Sleep Mechanisms

Figure 4.5 compares the latency of different sleep mechanisms for the Llama model family. The standard L1 sleep operation shows a clear dependence on model size. Its latency increases from 0.21 s for the 1B model to 0.60 s for the 3B model and 1.36 s for the 8B model. This behavior is expected because L1 sleep must copy the model weights to CPU memory before releasing the corresponding GPU allocations. As the model size increases, this transfer becomes increasingly expensive.

In contrast, L2 sleep and L1 sleep with already CPU-resident weights remain below 0.3 s for all evaluated model sizes. The lower latency of these two mechanisms arises because vLLM can release the allocated GPU memory directly and does not need to perform an additional weight transfer. The persistent case is particularly relevant for Kairos because it shows that keeping a CPU-resident copy of the model weights substantially reduces the cost of later sleep operations. For the 8B model, persistent L1 sleep reaches a latency of 0.28 s, compared to 1.36 s for standard L1 sleep. The result shows that the envisioned L1 sleep variant with persistent CPU-resident weights effectively acts as a hybrid between standard L1 sleep and L2 sleep.

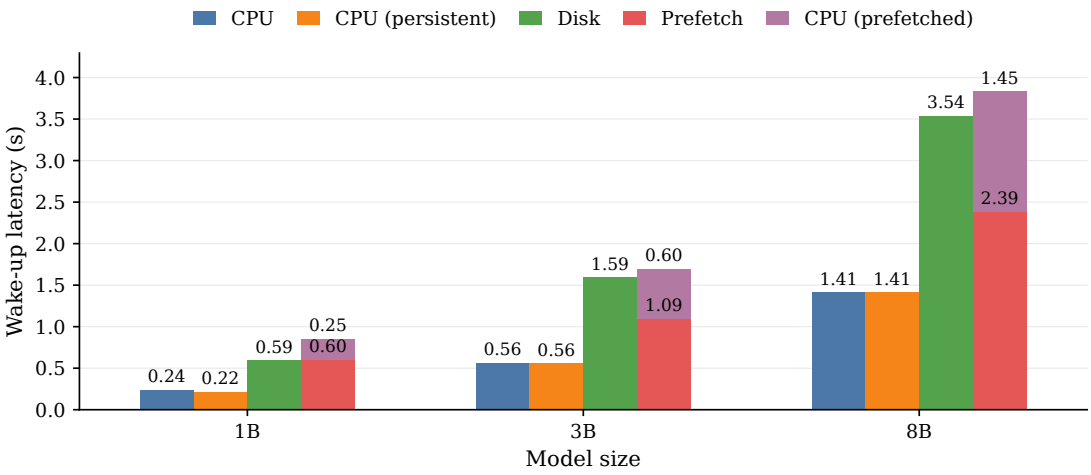

Figure 4.6: Latency of vLLM wake-up mechanisms across Llama-3 models.

Wake-up Mechanisms

Figure 4.6 compares the latency of different wake-up mechanisms for the Llama model family. Wake-up from CPU memory scales with model size, increasing from 0.24 s for the 1B model to 0.56 s for the 3B model and 1.41 s for the 8B model. Persistent wake-up shows the same latency, as expected, because maintaining a CPU-resident copy of the weights does not add overhead to the wake-up operation.

Wake-up from disk is substantially more expensive because the model weights must be loaded from storage before the model can become active on the GPU. For the 8B model, disk-based wake-up requires 3.54 s, compared to 1.41 s when the model is already resident in CPU memory. The prefetch-based path separates this cost into two phases. The disk-to-CPU prefetch phase can run before the model is needed, while only the final CPU-to-GPU wake-up remains on the request-critical path. Although the total stacked latency of the prefetch-based path is slightly higher than direct disk wake-up, the request-critical part is reduced to 0.25 s, 0.60 s, and 1.45 s for the 1B, 3B, and 8B models, respectively. These results supports Kairos' design choice of using prefetching to move expensive weight-loading work out of the critical path.

Eviction

Figure 4.7 compares direct eviction with the workaround required when CPU-resident eviction is not available. Direct eviction removes CPU-resident model weights without reactivating the model on the GPU and therefore completes almost immediately. Across the evaluated Llama models, eviction latency remains close to 1 ms. In contrast, the indirect path (i.e., wake-up followed by L2 sleep) first restores the model to GPU memory and then returns it to the sleeping state. This path is substantially more expensive and scales with model size, reaching 1.68 s for the 8B model.

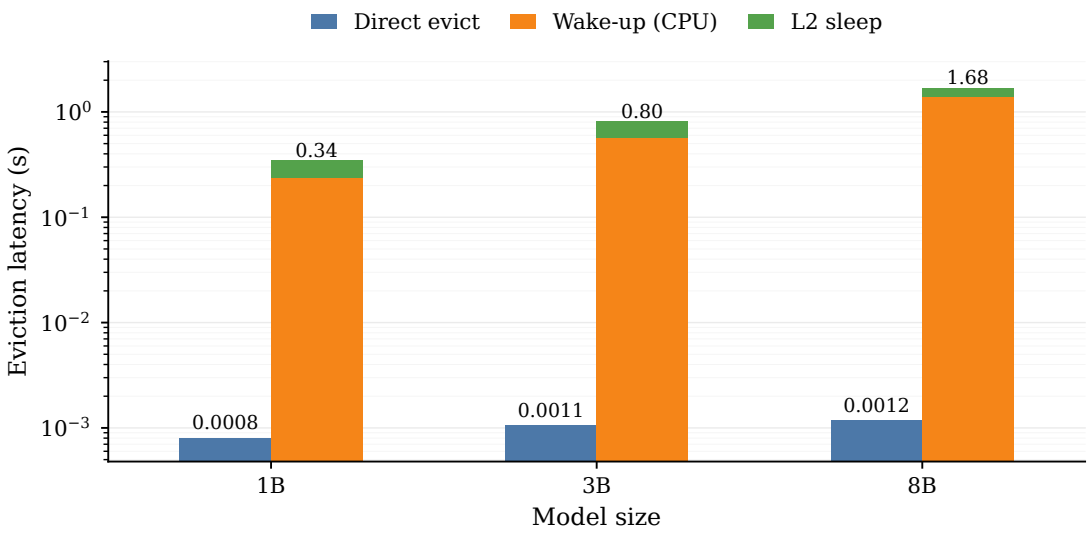

Figure 4.7: Eviction latency of Llama-3 models. Direct eviction removes CPU-resident weights, while the indirect path restores the model to GPU memory before L2 sleep.

Figure 4.8 shows the corresponding peak GPU memory requirement during eviction. Direct eviction keeps the model in its sleeping state and therefore requires only the residual GPU memory footprint of the sleeping model. The indirect path temporarily

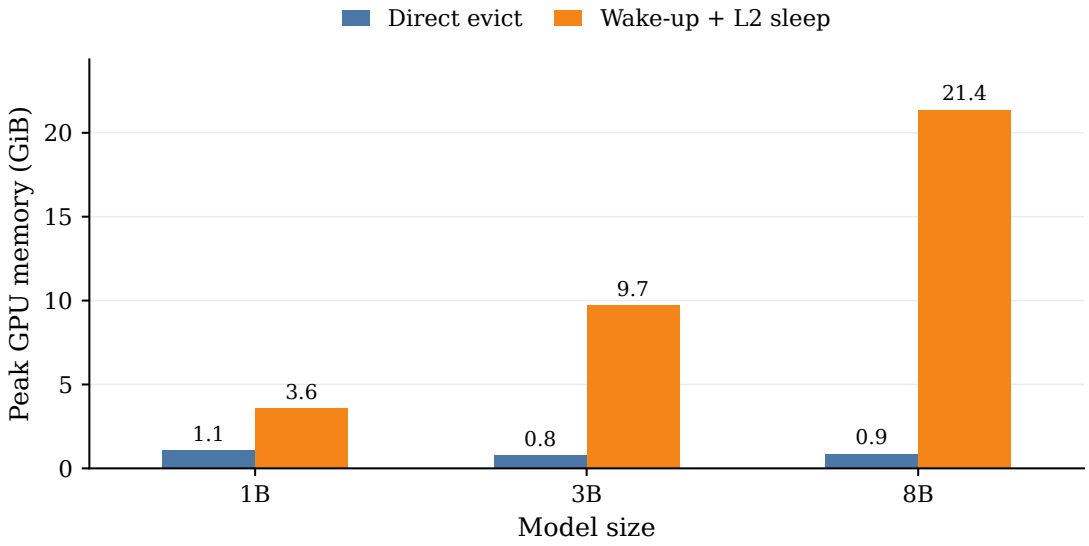

Figure 4.8: Peak GPU memory during eviction. Direct eviction keeps the model in the sleeping state, whereas indirect path temporarily requires the full GPU memory footprint.

reactivates the model and reaches the full active GPU memory footprint. For the 8B model, the reloading increases peak GPU memory from 0.9 GiB to 21.4 GiB. Direct eviction is therefore not only faster, but also avoids transient GPU memory pressure. This property is important for Kairos because it allows the system to demote CPU-resident models even when the GPU does not have enough free memory to reactivate them temporarily. To validate that the observed behavior is not specific to the Llama-3 model family, we repeat the sleep, wake-up, and eviction microbenchmarks for Qwen models and one sparse MoE model. Appendix A.1 shows that these experiments follow the same qualitative trends as the Llama results presented in this section.

4.2.3 Dispatch Concurrency

In Subsection 3.7.1 of the previous chapter, we introduced the design of the controller's dispatcher. The dispatcher forwards incoming requests concurrently to the corresponding vLLM model server. This experiment evaluates the importance of concurrent dispatching for end-to-end serving. In contrast to Experiment 4.2.1, which also considered model-level latency, this experiment focuses on end-to-end serving latency. We define end-to-end serving latency as the time between sending a request from the client and receiving the corresponding result.

To isolate the effect of dispatcher concurrency, we compare the default concurrent dispatcher against a sequential dispatcher variant. The concurrent dispatcher creates an asynchronous task for each request, allowing multiple requests to remain in flight and overlap at the vLLM server. Overlapping requests allow vLLM to exploit continuous batching. The sequential dispatcher serves as a controlled baseline. It removes one request from the queue, sends it to vLLM, waits until the result is returned, and only then processes the next request. Sequential dispatching therefore prevents Kairos from exposing overlapping requests to vLLM, causing requests that arrive during processing to wait in the Kairos queue.

Comparing both dispatchers shows how much the concurrent design contributes to end-to-end serving latency. Both variants use the same workload pattern, model, and dataset. The only difference is how Kairos forwards requests from its internal request queue to vLLM. Since all experiments run on a single GPU, we use the trivial parallelism configuration TP = 1, PP = 1, and DP = 1. Table 4.6 summarizes the experimental setup.

Table 4.6: Experimental setup for the dispatcher-concurrency experiment.

Setting	**Configuration**
Dataset	BoolQ
Model	Llama-3.1-8B-Instruct
Request rates	10, 20, 50 requests/s
Workload pattern	Poisson

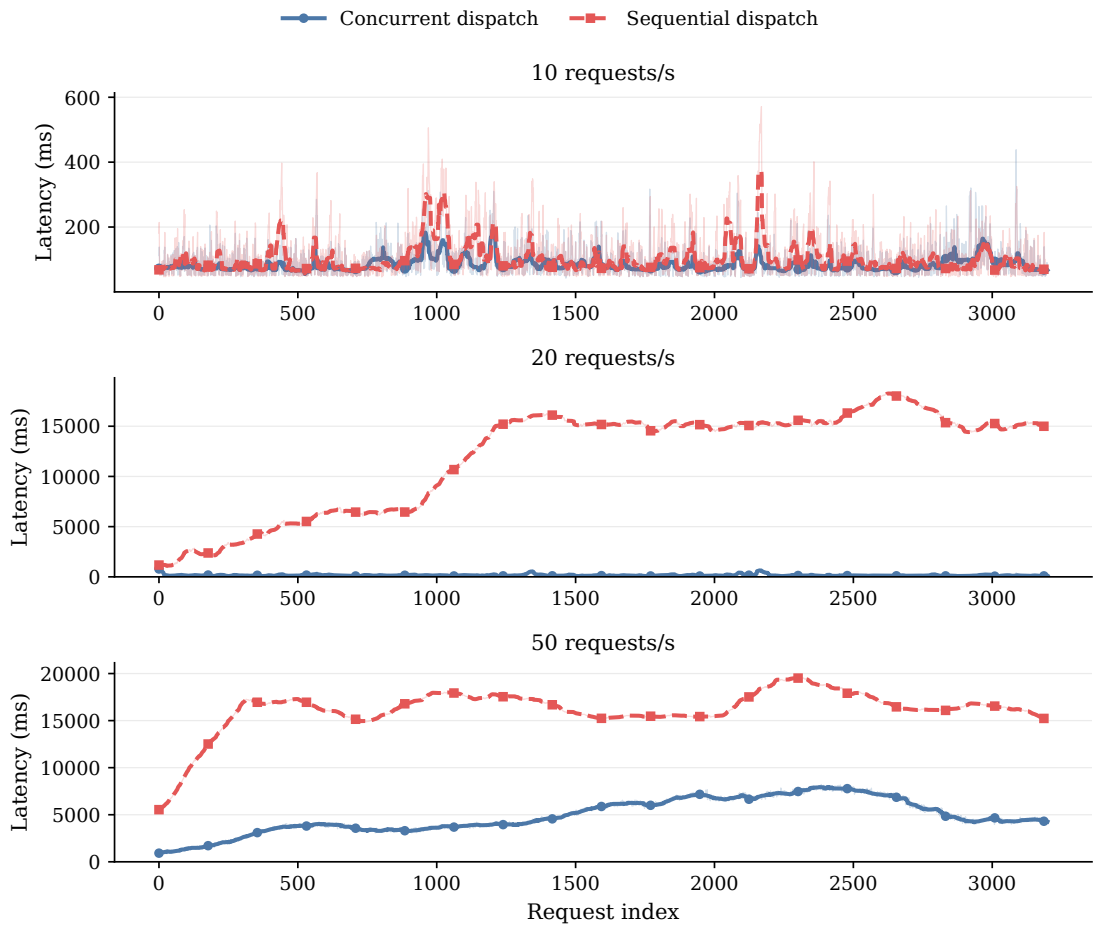

Figure 4.9: End-to-end request latency for sequential and concurrent dispatch under 10, 20, and 50 requests/s. Sequential dispatch processes one request at a time before sending the next request to vLLM, while concurrent dispatch allows multiple requests to remain in flight. The concurrent design exposes overlapping requests to vLLM and enables its internal continuous batching, which substantially reduces latency under higher load.

Figure 4.9 compares the end-to-end latency of the sequential and concurrent dispatchers under increasing request rates. At 10 requests/s, both dispatchers remain in a similar latency range. In this regime, the arrival rate is low enough for the sequential dispatcher to process most requests without accumulating a large queue. The benefit of concurrency is therefore limited at low load, but remains visible.

The difference becomes clear at 20 requests/s. The sequential dispatcher accumulates requests in its queue because it forwards only one request at a time to vLLM and waits for the result before processing the next request. As a result, end-to-end latency grows to several seconds and eventually stabilizes around 15 s. In contrast, the concurrent dispatcher keeps latency in the millisecond range because multiple requests can be forwarded

to vLLM at the same time. Concurrent dispatching allows vLLM to observe overlapping requests and exploit its internal continuous batching.

At 50 requests/s, both dispatchers experience higher latency, indicating that the system approaches or exceeds its sustainable serving capacity. However, the concurrent dispatcher remains substantially below the sequential dispatcher. The sequential variant reaches end-to-end latencies of up to 20 s because requests spend most of their time waiting before they are sent to vLLM. The results show that concurrent dispatch is essential for Kairos. Without concurrent dispatch, the system serializes request forwarding at the dispatcher and prevents vLLM from using continuous batching effectively under load.

4.2.4 Idle-time Forecasting

In Section 3.7.5 of the previous chapter, we introduced the scheduling algorithm and the forecasting methods used to predict idle windows. In this experiment, we evaluate how well the different forecasters predict such idle windows. We use the ON/OFF workload pattern described in Section 3.3.2 with a forecasting window of length 3. Table 4.7 summarizes the experimental setup and client configuration.

Table 4.7: Experimental setup for the Idle-time forecasting.

Parameter	**Value**
Window size	3
Workload pattern	ON/OFF
on_duration_seconds	20
off_duration_seconds	10
on_rate	10

Figure 4.10 shows the idle-window prediction quality of the evaluated forecasting methods under the ON/OFF workload pattern. The figure reports precision, recall, and F1 score. In this evaluation, an idle window is treated as the positive class. A positive prediction therefore means that the scheduler predicts the next window to be idle.

- **Precision** measures the reliability of idle-window predictions. Precision is defined as the fraction of predicted idle windows that are actually idle. High precision means that only a small fraction of windows classified as idle are false positives.
- **Recall** measures the coverage of actual idle windows. Recall is defined as the fraction of truly idle windows that are correctly classified as idle. High recall means that only a small fraction of available idle windows are missed.
- **F1 score** summarizes precision and recall in a single metric. The F1 score is defined as the harmonic mean of precision and recall and therefore penalizes cases where one of the two metrics is substantially lower than the other. In this setting, the F1 score captures the balance between avoiding false idle predictions and detecting available idle windows.

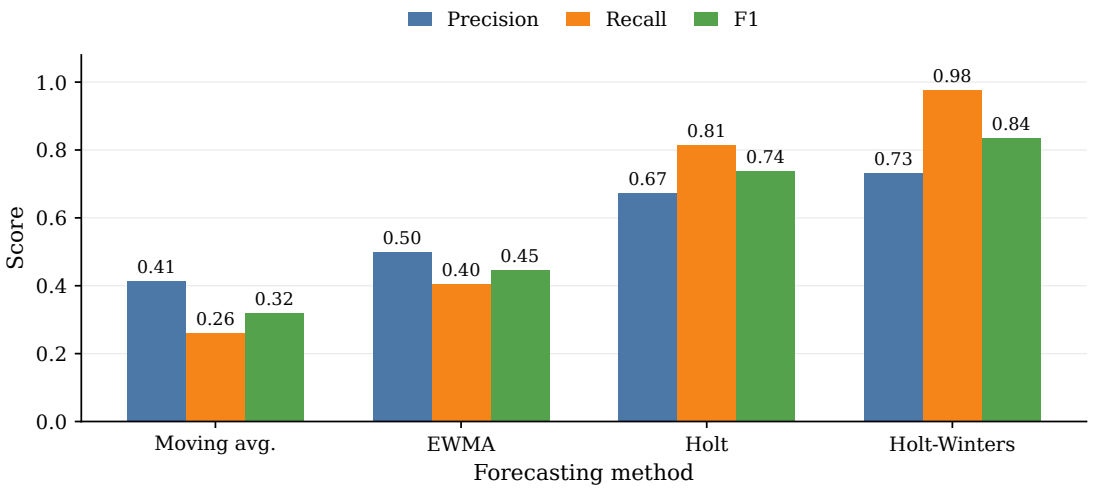

Figure 4.10: Idle-window prediction quality of the evaluated forecasting methods. The figure reports precision, recall, and F1 score, treating idle-window predictions as the positive class. Holt-Winters achieves the best overall prediction quality, followed by Holt, while moving average and EWMA miss a larger fraction of available idle windows.

The results show that Holt-Winters achieves the best overall prediction quality. Holt-Winters obtains the highest recall and F1 score, meaning that it detects most idle windows while maintaining reliable idle predictions. Holt performs similarly well, but remains

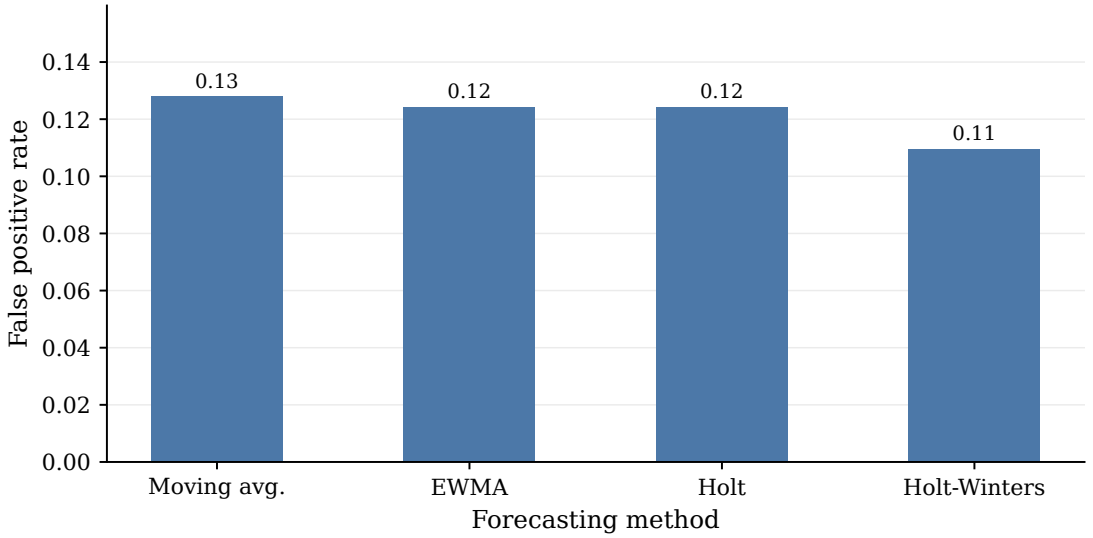

Figure 4.11: False positive rate of idle-window predictions. A false positive occurs when the scheduler predicts an idle window although the next window is actually busy.

slightly below Holt-Winters. In contrast, moving average and EWMA achieve lower recall, meaning that both methods miss a larger fraction of available idle windows. The lower recall is expected because simple averaging-based methods react more slowly to changes in the arrival process. Overall, the results indicate that forecasting methods with trend and seasonality components are better suited for identifying idle windows than methods that only smooth recent request history.

Figure 4.11 shows the false positive rate of the evaluated forecasting methods. In this evaluation, a false positive occurs when the scheduler predicts the next window to be idle although the window is actually busy. The false positive rate is particularly important for Kairos because false positives may cause the scheduler to release an interfering command during active request serving.

All evaluated methods keep the false positive rate within a relatively narrow range. Holt-Winters achieves the lowest false positive rate, followed closely by Holt and EWMA. Moving average shows the highest false positive rate among the evaluated methods. The lower false positive rate indicates that Holt-Winters provides the safest idle-window predictions in this workload, because it produces the smallest fraction of idle predictions during actually busy windows. Together with the prediction-quality results in Figure 4.10, the false positive analysis supports Holt-Winters as the strongest forecasting method among the evaluated candidates.

4.3 End-to-End Serving

After microbenchmarking individual components and mechanisms, we conduct an end-to-end evaluation of Kairos. Table 4.8 lists the models used in this evaluation. Qwen3-4B-Instruct-2507 serves as the base model, with FP8, INT8, and INT4 variants used as quantized alternatives. Llama-3.2-3B-Instruct serves as an independent lower-parameter alternative. Given the limited single-GPU setup, we choose a model set where at least one additional model fits into GPU memory alongside the base model. We evaluate Kairos with this model set under an ON/OFF arrival pattern, described in Section 3.3.2, and a periodic arrival pattern, described in Section 3.3.3.

Table 4.8: Models for end-to-end evaluation.

Model	**Precision**	**Parameters**
Qwen3-4B-Instruct-2507	BF16	4B
Qwen3-4B-Instruct-2507-FP8	FP8	4B
Qwen3-4B-Instruct-2507-quantized.w8a8	INT8	4B
Qwen3-4B-Instruct-2507-quantized.w4a16	INT4	4B
Llama-3.2-3B-Instruct	BF16	3B

We use the same Kairos configuration for both workload patterns. Table 4.9 summarizes the configuration, including the SLO targets, SLO weights, sampling rate, sample size, forecasting method, and forecasting window size.

Table 4.9: Kairos-internal setup.

Parameter	Value
accuracy	0.9
latency	100 ms
weight (accuracy)	0.7
weight (latency)	0.3
sample rate	10
sample size	50
forecaster	Holt-Winters
window size	3 s

For the end-to-end evaluation under the ON/OFF arrival pattern, we use the client configuration specified in Table 4.10. The workload alternates between ON phases of length 20,s at 10 requests/s and OFF phases of length 10,s at 0 requests/s.

Table 4.10: Client configuration the for ON/OFF arrival pattern.

Parameter	Value
Workload pattern	ON/OFF
on_duration_seconds	20 s
off_duration_seconds	10 s
on_rate	10 req/s
Dataset	BoolQ

Figure 4.12 shows the end-to-end request latency of Kairos under the ON/OFF arrival pattern. The y-axis uses a logarithmic scale to make both the normal low-latency serving regime and the higher latency spikes visible in the same plot. Before the dashed vertical line, Kairos serves requests with the initial base model. The dashed line marks the point at which Kairos switches the active model to the INT8 variant.

Most requests remain in the lower latency range throughout the experiment. The plot also shows several latency spikes, which correspond to periods in which reconfiguration commands interfere with active serving. Before the model switch, the system experiences pronounced spikes during model evaluation and reconfiguration. Given the selected sample rate and sample size, Kairos triggers reconfiguration after every 500 requests. The first reconfiguration is therefore triggered at request index 500. The resulting command sequence is usually executed with some delay, because the scheduler releases interfering commands only when it predicts an idle window.

After Kairos switches to the INT8 model, request latency decreases, indicating that the selected variant provides faster serving for the workload. Occasional spikes remain visible, however, showing that false-positive idle-window predictions can still lead to reconfiguration-related interference. For a deeper evaluation of idle-window prediction,

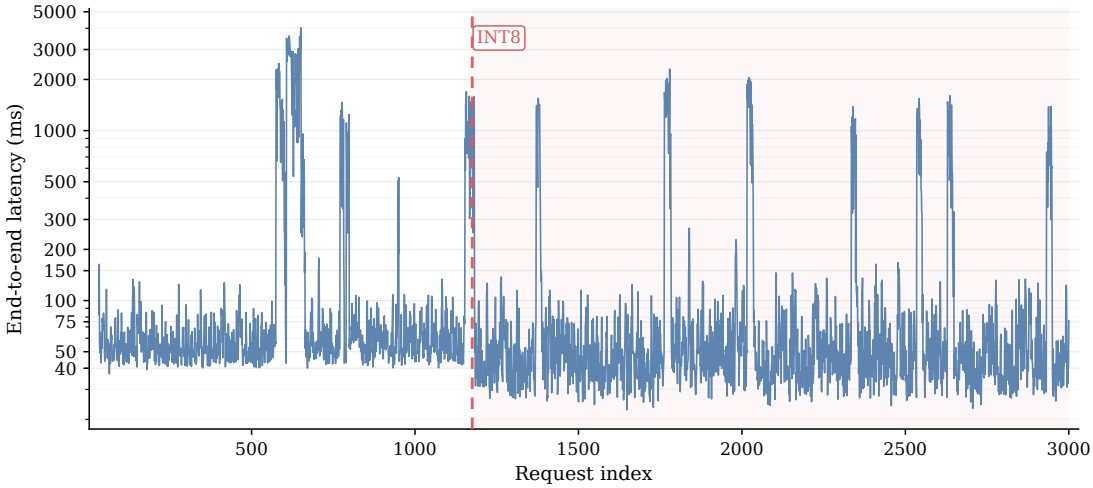

Figure 4.12: End-to-end evaluation under the ON/OFF arrival pattern. The dashed line marks the switch of the actively serving model. Latency spikes correspond to reconfiguration and subsequent control commands executed after missed idle-window predictions.

we refer to Section 4.2.4. Overall, the result demonstrates that Kairos can adapt the active model at runtime and reduce serving latency, while the remaining latency spikes highlight the limits of idle-window forecasting and interference-aware scheduling.

For the end-to-end evaluation under the periodic arrival pattern, we use the client configuration specified in Table 4.11. The configured workload follows a sinusoidal request-rate pattern over a period of 20 s, with request rates ranging from 0.5 to 11.5 requests/s.

Table 4.11: Client configuration for the periodic arrival pattern.

Parameter	**Value**
Workload pattern	periodoc
base_rate	6 req/s
amplitude	5.5 req/s
period_seconds	20 s
Dataset	BoolQ

Figure 4.13 shows the end-to-end request latency of Kairos under the periodic arrival pattern. As in the ON/OFF experiment, the y-axis uses a logarithmic scale to show both the normal serving regime and larger latency spikes in the same figure. The dashed vertical line marks the point at which Kairos switches the active model to the INT8 variant.

Kairos again triggers the first reconfiguration after 500 requests. The controller executes the resulting command sequence with delay, because the scheduler releases interfering commands only when it predicts an idle window. Compared to Figure 4.12,

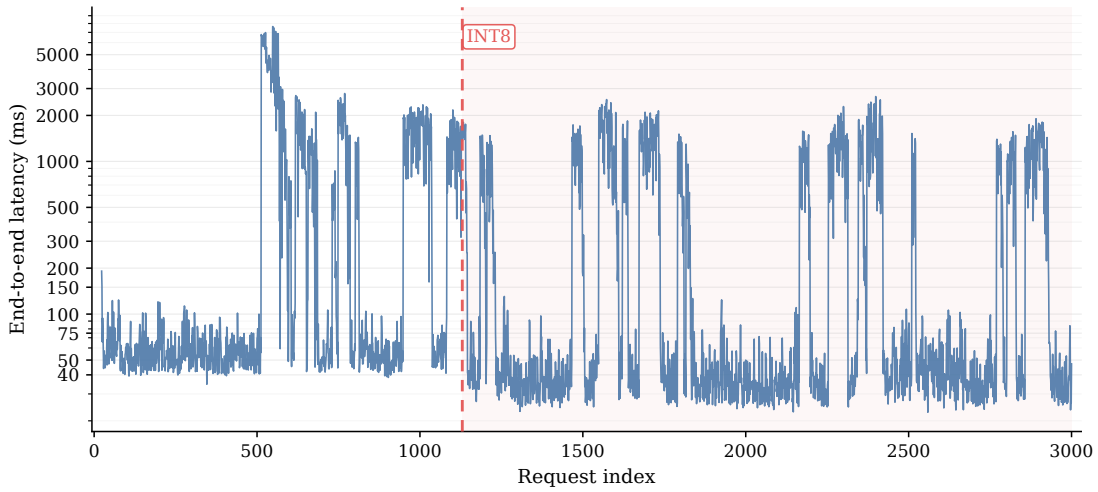

Figure 4.13: End-to-end evaluation under the periodic arrival pattern. The dashed line marks the switch of the actively serving model. Latency spikes correspond to reconfiguration and subsequent control commands executed after missed idle-window predictions.

the reconfiguration-induced latency spikes show greater variance. The periodic workload provides less clearly separated idle phases than the ON/OFF pattern, which makes command execution more likely to interfere with active serving.

After the switch to the INT8 model, the lower-latency serving regime becomes more pronounced. This observation indicates that the selected model variant also improves latency under the periodic workload. However, the plot also shows extended latency spikes caused by false-positive idle-window predictions. Overall, the result shows that Kairos can adapt the active model under a continuously changing workload, while also highlighting that periodic traffic patterns make interference-aware scheduling more challenging than workloads with explicit idle phases.

5 Related Work

This chapter discusses prior work related to Kairos. Since Kairos addresses LLM serving as a systems problem, the relevant literature spans several areas. First, we review LLM inference and serving systems that improve request execution through batching, scheduling, and runtime optimizations. Second, we discuss memory management and offloading techniques, which are closely related to moving model state across GPU memory, CPU memory, and disk. Third, we consider model compression and quantized serving, since Kairos uses different model variants to trade output quality against resource usage and latency. Finally, we discuss adaptive model selection and routing, which relates to the use of runtime observations for choosing between alternative models.

The chapter concludes by positioning Kairos with respect to these lines of work. Existing systems provide important mechanisms for efficient LLM execution, memory management, and model specialization. Kairos builds on these ideas but focuses on a different problem: adapting the serving configuration at runtime based on observed workload behavior, service-level objectives, and the cost of reconfiguration.

5.1 LLM Inference and Serving Systems

LLM inference systems aim to execute autoregressive generation efficiently under latency, throughput, and memory constraints. Since each request consists of a prefill phase followed by many decode steps, serving systems must coordinate requests whose prompt lengths, output lengths, and arrival times differ. Orca [65] addresses this execution pattern through iteration-level scheduling and continuous batching, allowing completed requests to leave a batch and newly arrived requests to enter between decoding iterations. FastServe [62] addresses a related scheduling problem by introducing preemption at the granularity of output tokens and using a multi-level feedback queue scheduler to reduce head-of-line blocking in distributed LLM serving.

vLLM [32] improves LLM serving through PagedAttention, which manages the KV cache using a paging-inspired memory layout. Instead of requiring contiguous KV-cache allocations for each request, vLLM stores cache blocks non-contiguously and tracks them through block tables. The reduced fragmentation allows larger effective batches and improves GPU memory utilization. SGLang [67] builds on efficient serving backends and adds runtime support for structured LLM programs, including mechanisms for reusing shared prefixes across related requests. Preble [54] studies this reuse problem in distributed settings and co-optimizes load balancing with KV-state reuse for prompts that share common prefixes.

Several systems focus on the different resource characteristics of prefill and decode.

Sarathi-Serve [1] introduces chunked prefill and stall-free scheduling to improve the throughput-latency trade-off. DeepSpeed-FastGen [24] uses Dynamic SplitFuse to combine prompt processing and token generation more effectively during batched serving. DistServe [68], Splitwise [46], and TetriInfer [25] go further by separating prefill and decode execution across different instances or machines, since prefill tends to be compute-intensive while decode is often more memory- and bandwidth-sensitive. Mooncake [48] also follows a disaggregated design, but centers the architecture around KV-cache management and uses CPU, DRAM, SSD, and network resources to support long-context serving.

Recent systems also study LLM serving in larger cloud and serverless environments. DeepFlow [26] provides a serverless platform for LLM serving and addresses resource allocation, cold-start latency, and scalable deployment of model-serving instances. Such systems extend the serving problem from single-engine request scheduling to cluster-level resource management and elastic deployment.

These systems provide the runtime foundations on which efficient LLM serving builds. They improve request scheduling, batching, KV-cache management, prefill-decode execution, and cluster-level serving efficiency. Kairos builds on this line of work, especially vLLM, but addresses a complementary problem. Instead of only optimizing execution within a fixed serving configuration, Kairos monitors the workload and adapts the serving configuration itself during runtime.

5.2 Memory Management and Offloading for LLM Serving

The memory footprint of LLM inference is dominated by model weights and the KV cache. Model weights consume memory before any request is processed, while the KV cache grows dynamically with the number of active requests and generated tokens. As a result, LLM serving systems must manage GPU memory carefully, especially when models or request batches exceed the capacity of a single accelerator.

Several systems address this problem by moving model state across different memory levels. FlexGen [52] targets high-throughput generation under limited GPU memory by offloading tensors across GPU memory, CPU memory, and disk, and by choosing offloading policies that balance memory capacity and data movement costs. DeepSpeed [3] Inference and ZeRO-Inference also use heterogeneous memory resources to enable inference for models that do not fit entirely in GPU memory. LLM in a Flash [2] studies an even more constrained setting and stores model parameters in flash memory, loading them into main memory on demand during inference.

Other systems focus on memory management for the KV cache. vLLM [32] introduces PagedAttention to reduce KV-cache fragmentation and improve effective GPU memory utilization. Mooncake [48] treats the KV cache as a central system resource and builds a disaggregated serving architecture around KV-cache placement and transfer. These systems show that memory management is not only a capacity problem, but also a scheduling and data-movement problem.

Kairos is related to this line of work because it also treats GPU memory, CPU memory,

and disk as distinct storage levels for model state. However, Kairos does not primarily aim to serve one oversized model under a fixed policy. Instead, Kairos uses memory movement as part of runtime reconfiguration. The system moves model weights between memory tiers to change which model can serve requests, which model can be prepared for future use, and which model can be demoted when resources are needed elsewhere.

5.3 Model Compression and Quantized Serving

Model compression techniques reduce the memory and computational cost of LLM inference. Among these techniques, quantization is particularly relevant for serving because it represents weights, activations, or both with lower-precision data types. Lower precision reduces the memory footprint of model parameters and can also improve throughput when the hardware and runtime provide efficient kernels for the chosen format. Quantization therefore provides a practical way to serve larger models on fixed hardware or to increase the number of models that can be kept available at the same time.

Several approaches study post-training quantization for LLMs. LLM.int8() [13] introduces mixed-precision decomposition to preserve outlier features while enabling 8-bit matrix multiplication. SmoothQuant [63] shifts quantization difficulty from activations to weights and enables efficient W8A8 quantization for transformer models. GPTQ [17] and AWQ [34] focus on low-bit weight quantization and aim to preserve model quality when compressing weights to formats such as INT4. QLoRA [14] uses 4-bit quantization with the NormalFloat format to reduce memory requirements during fine-tuning, and the same quantized representations are also relevant for memory-efficient deployment.

Quantized serving changes the trade-off between resource usage and output quality. A quantized model usually has a smaller memory footprint than its full-precision counterpart and can reduce the cost of moving weights across memory tiers. At the same time, quantization may affect task accuracy, and the magnitude of the effect depends on the model, the quantization scheme, and the workload. Serving systems that use quantized models therefore need to consider both system-level metrics, such as latency and throughput, and task-level metrics, such as accuracy.

Kairos uses quantized models as serving alternatives rather than treating quantization as a standalone optimization. The system can compare a base model, quantized variants, and independent alternatives on sampled requests. The resulting measurements allow Kairos to decide whether a more efficient model variant still satisfies the configured service-level objectives. In contrast to work that focuses on improving a specific quantization method, Kairos uses quantization as one source of runtime flexibility within a workload-aware serving system.

5.4 Adaptive Model Selection and Routing

Adaptive model selection uses differences between available models to improve the cost-quality trade-off of LLM inference. Instead of sending every request to the same model, these approaches choose among multiple models based on properties of the input, the

expected difficulty of the request, or a target budget. FrugalGPT [8] studies this problem in the setting of LLM APIs and proposes model cascades that reduce cost by routing requests through cheaper models when possible and escalating to stronger models when necessary. RouteLLM [44] learns routers from preference data and dynamically selects between stronger and weaker models during inference. More recent work further unifies routing and cascading by treating both as model selection strategies over a shared cost-quality frontier [11].

These systems are closely related to Kairos because they also exploit the fact that different models provide different efficiency and quality trade-offs. However, their main decision is usually which model should answer a given request. Kairos considers a complementary systems problem. The serving system must not only decide which model is suitable, but also ensure that the model is placed in the right memory tier, can be activated when needed, and can be switched in without excessive interference to ongoing serving.

Model routing therefore addresses the logical selection problem, while Kairos focuses on the runtime management problem that arises when such alternatives are served under limited hardware resources. Kairos uses runtime measurements and service-level objectives to evaluate candidate models, but it couples model selection with reconfiguration, memory movement, and scheduling. This connection between model choice and runtime state distinguishes Kairos from routing systems that assume the available models are already ready to serve.

5.5 Positioning of Kairos

The related work discussed above improves LLM serving along several dimensions. Existing inference systems optimize request execution through batching, scheduling, KV-cache management, and disaggregated prefill-decode execution. Memory-management systems extend the usable capacity of limited hardware by offloading model state or KV-cache data across heterogeneous memory levels. Quantization work reduces the memory and computational cost of individual model variants. Model routing systems exploit differences between available models to improve cost-quality trade-offs at the request level.

Kairos combines ideas from these areas, but addresses a different point in the design space. Instead of optimizing a single fixed serving configuration, Kairos treats the serving configuration itself as a runtime decision. The system continuously observes the workload, evaluates candidate models against service-level objectives, and reconfigures model placement when the collected information indicates that another configuration is preferable. Runtime adaptation therefore connects model choice, memory-tier placement, and scheduling decisions.

The closest relation to existing serving systems is the use of vLLM as the underlying inference engine. Kairos relies on vLLM for efficient request execution and extends it with additional model-management mechanisms for moving weights across GPU memory, CPU memory, and disk. In contrast to systems that primarily optimize batching or KV-cache allocation inside a running model server, Kairos coordinates multiple model servers

and decides when models should be active, staged, or evicted.

The closest relation to model routing work is the use of multiple candidate models with different efficiency and quality characteristics. However, routing systems usually assume that the candidate models are already available for inference. Kairos makes model availability part of the systems problem. A model that is accurate and efficient for a request is only useful if the serving system can place it in the required memory tier and activate it without violating latency constraints or interfering excessively with active serving.

Overall, Kairos positions itself as a workload-aware and reconfigurable LLM serving system. Its main contribution lies in connecting runtime monitoring, model evaluation, model placement, and interference-aware scheduling into one serving architecture.

6 Conclusions

LLM serving requires balancing output quality, latency, throughput, and memory usage under limited hardware resources. Static serving configurations are often insufficient for this setting because workloads and serving conditions may change during operation, while different model variants expose different trade-offs between accuracy and inference latency. This thesis addressed the problem of workload-aware and reconfigurable LLM serving. The goal was to design a system that can observe the request stream, evaluate alternative models, and adapt the serving configuration at runtime while respecting SLOs.

Kairos implements this idea through three design principles: continuous monitoring, runtime reconfigurability, and interference-aware scheduling. The system samples live requests, evaluates candidate models on observed workload data, and uses the resulting statistics to decide how the serving configuration should change. Runtime reconfiguration allows Kairos to switch the active model and move model weights across GPU memory, CPU memory, and disk. Interference-aware scheduling delays disruptive commands until the workload is predicted to provide an idle window, reducing the impact of adaptation on active request serving.

The microbenchmarks validated the main mechanisms required for Kairos. The model state transition benchmarks showed that sleep, wake-up, prefetching, and eviction provide different latency and memory trade-offs, and that prefetching can move expensive disk-to-CPU work out of the request-critical path. The model trade-off experiments showed that quantized variants and smaller independent models can provide useful alternatives to the base model, but that accuracy and latency depend strongly on the model type and quantization scheme. The dispatcher experiment confirmed that concurrent request forwarding is necessary to expose overlapping requests to vLLM and to benefit from continuous batching. The forecasting experiment showed that idle-window prediction is feasible, while also highlighting that false-positive idle predictions remain a source of reconfiguration-related interference.

The end-to-end experiments evaluated Kairos as a complete serving system. Under both the ON/OFF and periodic arrival patterns, Kairos adapted the active model at runtime and reduced request latency after switching to a faster model variant. The results show that the monitoring and reconfiguration loop can translate observed model behavior into an improved serving configuration. At the same time, the latency traces also show spikes caused by reconfiguration commands that interfered with active serving. These spikes demonstrate that runtime adaptation is beneficial, but that its effectiveness depends on accurate idle-window prediction and careful scheduling of interfering commands.

The evaluation also reveals several limitations. Kairos was evaluated on a single local workstation with one GPU, which limits the size and number of models that can

be considered simultaneously. The experiments therefore focus on single-node reconfiguration and do not study distributed placement across multiple machines. In addition, the current evaluation uses synthetic arrival patterns to create controlled workload conditions. These patterns are useful for isolating system behavior, but real production workloads may contain more diverse request distributions, longer temporal dependencies, and stronger variation in prompt and output lengths. The end-to-end experiments also show that idle-window forecasting remains a critical limitation. Even when the scheduler predicts most idle windows correctly, false-positive idle predictions can release interfering commands during active serving. On a single-GPU system, such interference has a strong effect because reconfiguration commands and foreground inference compete for the same accelerator resources.

Future work can extend Kairos in several directions. A natural next step is to evaluate the system in a multi-GPU setup. Separating active request serving from monitoring, evaluation, and reconfiguration work across different GPUs could reduce the interference observed in the single-GPU experiments. Such a setup would also make false-positive idle-window predictions less costly, because an interfering command would not necessarily compete with foreground inference on the same accelerator.

Another direction is to improve the scheduler itself. The current implementation uses lightweight time-series forecasters over recent request arrivals. Future versions could incorporate learned prediction models or richer workload features to identify idle windows more reliably. Better idle-window prediction would reduce unnecessary reconfiguration interference while still allowing Kairos to adapt promptly when the workload changes.

Future evaluations should also consider real production traces instead of only synthetic arrival patterns. Production traces would provide more realistic temporal structure, request diversity, and variation in prompt and output lengths. Finally, the reconfiguration policy could be extended with richer cost models that account for energy consumption, monetary serving cost, or more detailed quality metrics.

Overall, this thesis shows that workload-aware runtime reconfiguration is a viable direction for LLM serving. Kairos demonstrates that monitoring observed requests, evaluating alternative models, and changing the active serving configuration can reduce latency while preserving configured quality constraints. At the same time, the results show that reconfiguration must be treated as a systems problem: model movement, scheduling, and interference control are essential for making adaptation practical during active serving.

List of Acronyms

LLM	Large Language Model
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MHSA	Multi Head Self Attention
MLP	Multi Layer Perceptron
ReLU	Rectified Linear Unit
GPT	Generative Pretrained Transformer
KV Cache	Key-Value Cache
TP	Tensor Parallelism
DP	Data Parallelism
PP	Pipeline Parallelism
QAT	Quantization-Aware Training
PTQ	Post-Training Quantization
INT	Integer
FP	Floating Point
NF	Normal Float
IPC	Inter-Process Communication
SLO	Service Level Objective
JIT	Just-In-Time
RoPE	Rotary Position Embedding
OS	Operating System
MoE	Mixture of Experts

Bibliography

[1] Amey Agrawal et al. "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve". In: *OSDI*. 2024, pp. 117–134. URL: https://www.usenix.org/conference/osdi24/presentation/agrawal.

[2] Keivan Alizadeh et al. "LLM in a Flash: Efficient Large Language Model Inference with Limited Memory". In: *ACL*. 2024, pp. 12562–12584. DOI: 10.18653/v1/2024.acl-long.678.

[3] Reza Yazdani Aminabadi et al. "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale". In: *SC*. 2022, pp. 1–15. DOI: 10.1109/SC41404.2022.00051.

[4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. *Layer Normalization*. 2016. arXiv: 1607.06450 [stat.ML].

[5] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer Cham, 2023. DOI: 10.1007/978-3-031-45468-4.

[6] Rishi Bommasani et al. *On the Opportunities and Risks of Foundation Models*. 2021. arXiv: 2108.07258 [cs.LG].

[7] Tom B. Brown et al. "Language Models Are Few-Shot Learners". In: *NeurIPS*. Vol. 33. 2020, pp. 1877–1901. URL: https://papers.nips.cc/paper_files/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

[8] Lingjiao Chen, Matei Zaharia, and James Zou. "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance". In: *TMLR* (2024). URL: https://openreview.net/forum?id=cSimKw5p6R.

[9] Christopher Clark et al. "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions". In: *NAACL*. 2019, pp. 2924–2936. DOI: 10.18653/v1/N19-1300.

[10] Kevin Clark et al. "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators". In: *ICLR*. 2020. URL: https://openreview.net/forum?id=r1xMH1BtvB.

[11] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. "A Unified Approach to Routing and Cascading for LLMs". In: *ICML*. 2025, pp. 12987–13010. URL: https://proceedings.mlr.press/v267/dekoninck25a.html.

[12] Tim Dettmers and bitsandbytes contributors. *bitsandbytes: Accessible Large Language Models via k-bit Quantization for PyTorch*. 2022. URL: https://github.com/bitsandbytes-foundation/bitsandbytes.

[13] Tim Dettmers et al. "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *NeurIPS*. Vol. 35. 2022, pp. 30318–30332. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/c3ba4962c05c49636d4c6206a97e9c8a-Abstract-Conference.html.

[14] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *NeurIPS*. 2023. URL: https://openreview.net/forum?id=OUIFPHEgJU.

[15] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *NAACL*. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.

[16] Elias Frantar et al. "MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models". In: *PPoPP*. 2025, pp. 239–251. DOI: 10.1145/3710848.3710871.

[17] Elias Frantar et al. "OPTQ: Accurate Quantization for Generative Pre-trained Transformers". In: *ICLR*. 2023. URL: https://openreview.net/forum?id=tcbBPnfwxS.

[18] Georgi Gerganov and llama.cpp contributors. *llama.cpp: LLM Inference in C/C++*. 2023. URL: https://github.com/ggml-org/llama.cpp.

[19] Amir Gholami et al. "A Survey of Quantization Methods for Efficient Neural Network Inference". In: *Low-Power Computer Vision*. Chapman and Hall/CRC, 2022, pp. 291–326. DOI: 10.1201/9781003162810-13.

[20] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI].

[21] Dan Hendrycks et al. "Measuring Massive Multitask Language Understanding". In: *ICLR*. 2021. URL: https://openreview.net/forum?id=d7KBjmI3GmQ.

[22] Pieter Hintjens. *Advanced Request-Reply Patterns*. ØMQ Guide. Accessed: 2026-05-15. URL: https://zguide.zeromq.org/docs/chapter3/.

[23] Pieter Hintjens. *ZeroMQ Request-Reply Pattern*. ZeroMQ RFC 28/REQREP. Accessed: 2026-05-15. URL: https://rfc.zeromq.org/spec/28/.

[24] Connor Holmes et al. *DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference*. 2024. arXiv: 2401.08671 [cs.PF].

[25] Cunchen Hu et al. *Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads*. 2024. arXiv: 2401.11181 [cs.DC].

[26] Junhao Hu et al. "DEEPSERVE: Serverless Large Language Model Serving at Scale". In: *USENIX ATC*. 2025, pp. 57–72. URL: https://www.usenix.org/conference/atc25/presentation/hu-junhao.

[27] Yanping Huang et al. "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism". In: *NeurIPS*. Vol. 32. 2019, pp. 103–112. URL: https://papers.nips.cc/paper_files/paper/2019/hash/093f65e080a295f8076b1c5722a46aa2-Abstract.html.

[28] Hugging Face. *Understand Caching*. Hugging Face Hub Documentation. Accessed: 2026-05-19. URL: https://huggingface.co/docs/huggingface_hub/en/guides/manage-cache.

[29] Benoit Jacob et al. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference". In: *CVPR*. 2018, pp. 2704–2713. DOI: 10.1109/CVPR.2018.00286.

[30] Jared Kaplan et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG].

[31] Uygar Kurt. *Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct*. 2026. arXiv: 2601.14277 [cs.CL].

[32] Woosuk Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *SOSP*. 2023, pp. 611–626. DOI: 10.1145/3600006.3613165.

[33] Mike Lewis et al. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension". In: *ACL*. 2020, pp. 7871–7880. DOI: 10.18653/v1/2020.acl-main.703.

[34] Ji Lin et al. "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration". In: *MLSys*. Vol. 6. 2024, pp. 87–100. URL: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.

[35] Jian Liu et al. "LogiQA: A Challenge Dataset for Machine Reading Comprehension with Logical Reasoning". In: *IJCAI*. 2020, pp. 3622–3628. DOI: 10.24963/ijcai.2020/501.

[36] Yinhan Liu et al. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. 2019. arXiv: 1907.11692 [cs.CL].

[37] Zechun Liu et al. "LLM-QAT: Data-Free Quantization Aware Training for Large Language Models". In: *Findings of ACL*. 2024, pp. 467–484. DOI: 10.18653/v1/2024.findings-acl.26.

[38] Paulius Micikevicius et al. *FP8 Formats for Deep Learning*. 2022. arXiv: 2209.05433 [cs.LG].

[39] Todor Mihaylov et al. "Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering". In: *EMNLP*. 2018, pp. 2381–2391. DOI: 10.18653/v1/D18-1260.

[40] Moonmoon Mohanty et al. "Deferred Prefill for Throughput Maximization in LLM Inference". In: *EuroMLSys*. 2025, pp. 100–106. DOI: 10.1145/3721146.3721962.

[41] Guido F. Montúfar et al. "On the Number of Linear Regions of Deep Neural Networks". In: *NeurIPS*. Vol. 27. 2014, pp. 2924–2932. URL: https://papers.nips.cc/paper_files/paper/2014/hash/fa6f2a469cc4d61a92d96e74617c3d2a-Abstract.html.

[42] Markus Nagel et al. "Up or Down? Adaptive Rounding for Post-Training Quantization". In: *ICML*. 2020, pp. 7197–7206. URL: https://proceedings.mlr.press/v119/nagel20a.html.

[43] Deepak Narayanan et al. "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM". In: *SC*. 2021. DOI: 10.1145/3458817.3476209.

[44] Isaac Ong et al. "RouteLLM: Learning to Route LLMs with Preference Data". In: *ICLR*. 2025. URL: https://openreview.net/forum?id=8sSqNntaMr.

[45] OpenAI. *GPT-4 Technical Report*. 2023. arXiv: 2303.08774 [cs.CL].

[46] Pratyush Patel et al. "Splitwise: Efficient Generative LLM Inference Using Phase Splitting". In: *ISCA*. 2024, pp. 118–132. DOI: 10.1109/ISCA59077.2024.00019.

[47] Reiner Pope et al. "Efficiently Scaling Transformer Inference". In: *MLSys*. Vol. 5. 2023, pp. 606–624. URL: https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.

[48] Ruoyu Qin et al. "Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot". In: *FAST*. 2025, pp. 155–170. URL: https://www.usenix.org/conference/fast25/presentation/qin.

[49] Alec Radford et al. *Improving Language Understanding by Generative Pre-Training*. 2018. URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

[50] Alec Radford et al. *Language Models Are Unsupervised Multitask Learners*. 2019. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[51] Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *JMLR* 21.140 (2020), pp. 1–67. URL: https://jmlr.org/papers/v21/20-074.html.

[52] Ying Sheng et al. "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU". In: *ICML*. 2023, pp. 31094–31116. URL: https://proceedings.mlr.press/v202/sheng23a.html.

[53] Mohammad Shoeybi et al. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. 2019. arXiv: 1909.08053 [cs.CL].

[54] Vikranth Srivatsa et al. "Preble: Efficient Distributed Prompt Scheduling for LLM Serving". In: *ICLR*. 2025. URL: https://openreview.net/forum?id=meKEKDhdnx.

[55] Jianlin Su et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *Neurocomputing* 568 (2024), p. 127063. DOI: 10.1016/j.neucom.2023.127063.

[56] Richard S. Sutton. *The Bitter Lesson*. Incomplete Ideas. Accessed: 2026-05-14. 2019. URL: http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

[57] Matus Telgarsky. "Benefits of Depth in Neural Networks". In: *COLT*. 2016, pp. 1517–1539. URL: https://proceedings.mlr.press/v49/telgarsky16.html.

[58] Ashish Vaswani et al. "Attention Is All You Need". In: *NeurIPS*. Vol. 30. 2017. URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[59] vLLM Project. *FP8 W8A8*. vLLM Documentation. Accessed: 2026-05-31. URL: https://docs.vllm.ai/en/latest/features/quantization/fp8/.

[60] vLLM Project. *INT4 W4A16*. vLLM Documentation. Accessed: 2026-05-31. URL: https://docs.vllm.ai/en/latest/features/quantization/int4/.

[61] vLLM Project. *Zero-Reload Model Switching with vLLM Sleep Mode*. vLLM Blog. Accessed: 2026-05-19. 2025. URL: https://vllm.ai/blog/2025-10-26-sleep-mode.

[62] Bingyang Wu et al. "FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference". In: *NSDI*. 2026, pp. 57–74. URL: https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang.

[63] Guangxuan Xiao et al. "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models". In: *ICML*. 2023, pp. 38087–38099. URL: https://proceedings.mlr.press/v202/xiao23c.html.

[64] Ruibin Xiong et al. "On Layer Normalization in the Transformer Architecture". In: *ICML*. 2020, pp. 10524–10533. URL: https://proceedings.mlr.press/v119/xiong20b.html.

[65] Gyeong-In Yu et al. "Orca: A Distributed Serving System for Transformer-Based Generative Models". In: *OSDI*. 2022, pp. 521–538. URL: https://www.usenix.org/conference/osdi22/presentation/yu.

[66] Yilong Zhao et al. "Atom: Low-Bit Quantization for Efficient and Accurate LLM Serving". In: *MLSys*. Vol. 6. 2024, pp. 196–209. URL: https://proceedings.mlsys.org/paper_files/paper/2024/hash/5edb57c05c81d04beb716ef1d542fe9e-Abstract-Conference.html.

[67] Lianmin Zheng et al. "SGLang: Efficient Execution of Structured Language Model Programs". In: *NeurIPS*. Vol. 37. 2024. DOI: 10.52202/079017-2000.

[68] Yinmin Zhong et al. "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving". In: *OSDI*. 2024, pp. 193–210. URL: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

Appendix

This appendix provides supplementary material for the experimental evaluation. It contains additional figures and results that support the analysis in the main text but are not essential for the primary discussion. The material is organized by the corresponding experiment sections.

A.1 Additional Model Transition Results

This section provides additional results for the model state transition experiments discussed in Section 4.2.2. We repeat the same experiments on a different set of models, listed in Table A.1. This set includes Qwen-3 models of different sizes and OLMoE-1B-7B-0924-Instruct, which is a MoE with 7B total parameters and 1B active parameters.

Table A.1: Qwen-3 models across three different sizes and a MoE. We assume a `gpu_factor` of 1.3 to accommodate for KV cache and activations besides model weights.

Model	**Parameters**	**GPU footprint (MiB)**
Qwen3-1.7B	1.7B	6,188
Qwen3-4B	4B	11,468
Qwen3-8B	8B	22,342
OLMoE-1B-7B-0924-Instruct	7B (1B active)	18,786

Figure A1 shows the latency of the three vLLM sleep mechanisms for the Qwen models. Overall, we observe the same pattern as for the Llama models. The latency of standard L1 sleep increases with model size, whereas L2 sleep and L1 sleep with CPU-resident weights avoid expensive transfers from GPU to CPU memory and are therefore substantially faster. Figure A2 shows the latency of the three vLLM sleep mechanisms for the MoE model. The results align with the previous observations for the Llama and Qwen models. The wake-up and eviction results follow the same trends observed for the Llama models and are also reported in this section.

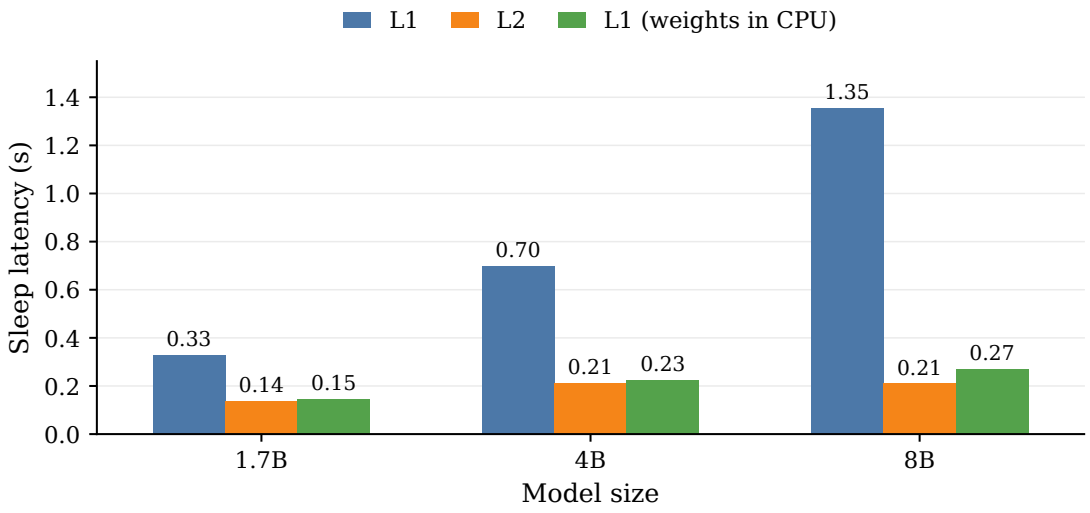

Figure A1: Latency of vLLM sleep mechanisms across Qwen-3 models of different sizes. Standard L1 sleep increases with model size due to the cost of copying weights from GPU to CPU memory. L2 sleep and L1 sleep with persistent CPU memory release GPU-resident data directly.

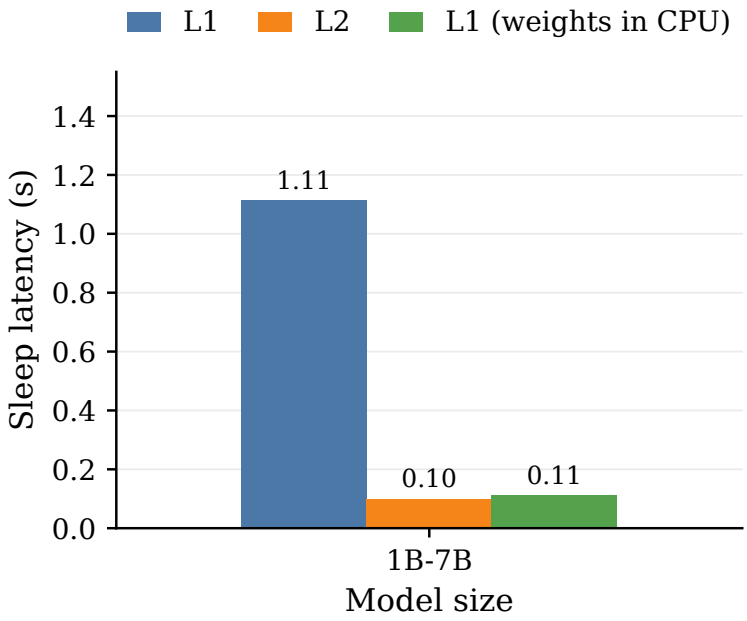

Figure A2: Latency of vLLM sleep mechanisms for OLMoE-1B-7B-0924-Instruct.

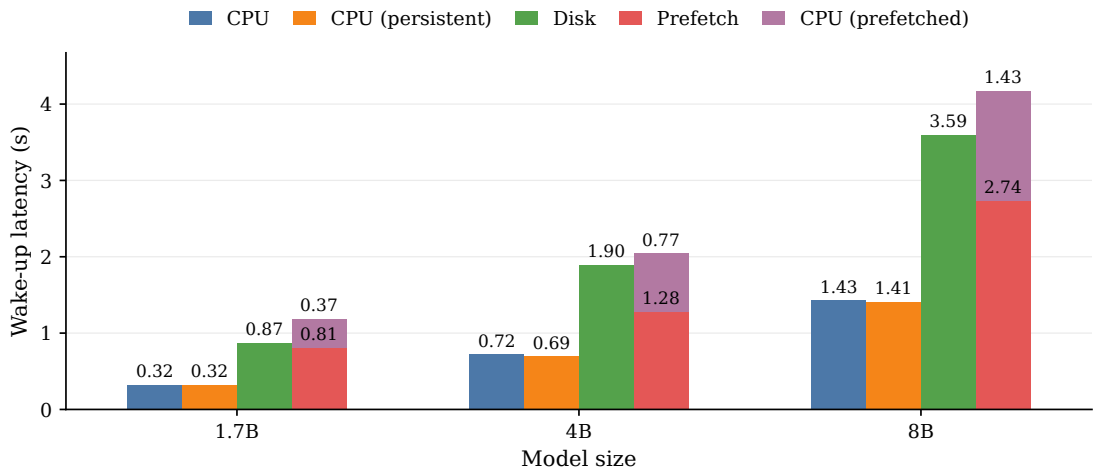

Figure A3: Latency of vLLM wake-up mechanisms across Qwen-3 models of different sizes. Wake-up from CPU memory and persistent wake-up show similar latency, while disk-based wake-up is substantially more expensive. The prefetch-based path separates disk-to-CPU loading from the request-critical CPU-to-GPU wake-up phase.

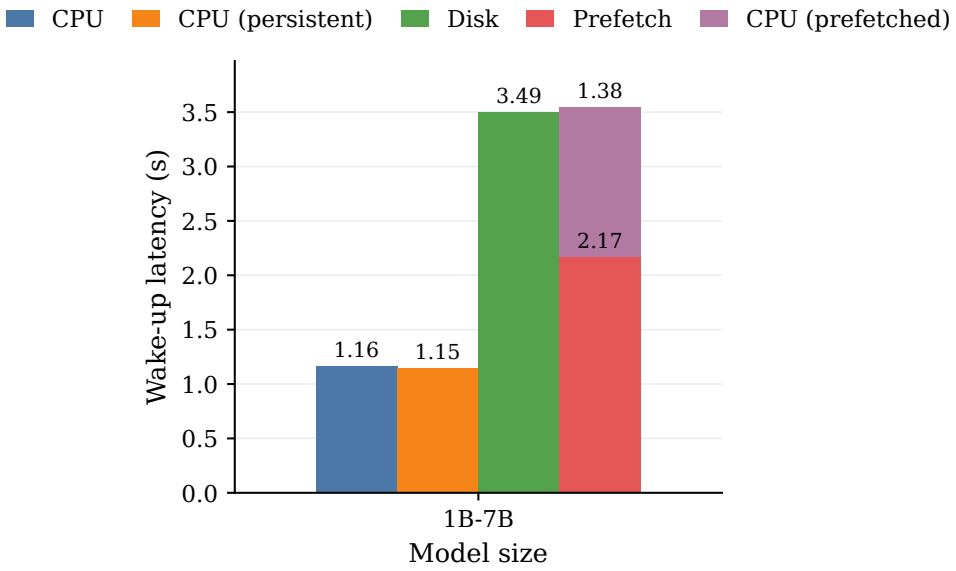

Figure A4: Latency of vLLM wake-up mechanisms for OLMoE-1B-7B-0924-Instruct.

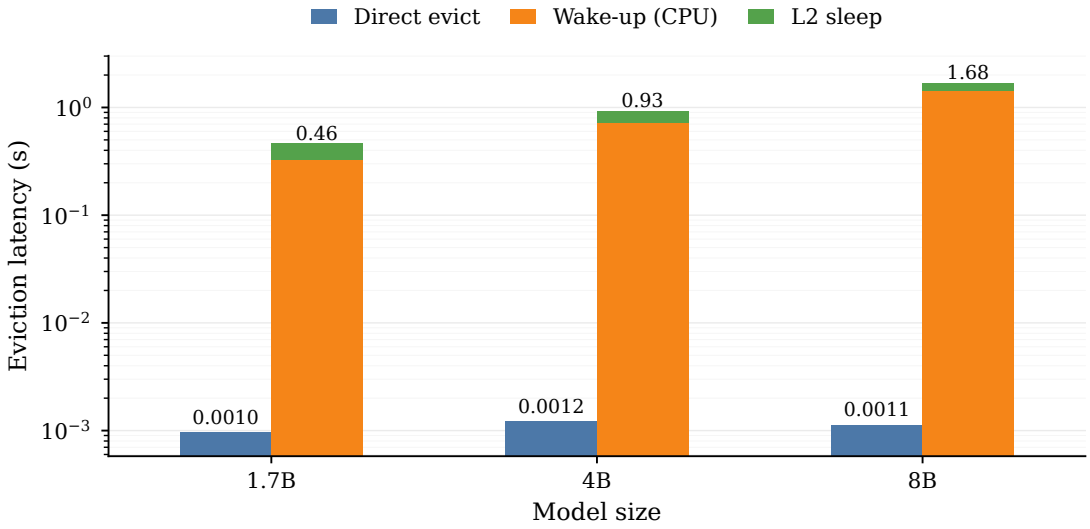

Figure A5: Eviction latency of Qwen-3 models. Direct eviction removes CPU-resident weights, while the indirect path restores the model to GPU memory before L2 sleep.

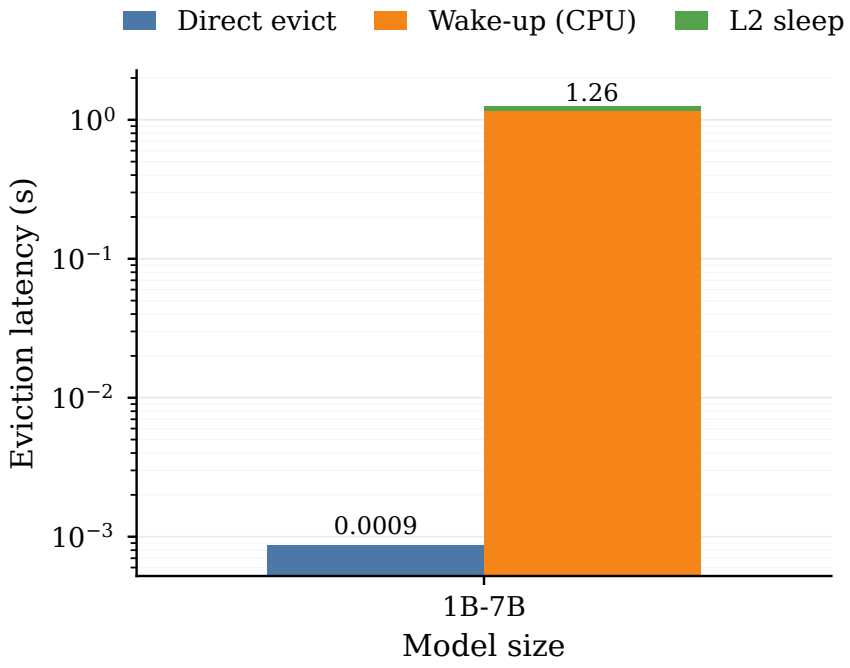

Figure A6: Eviction latency of OLMoE-1B-7B-0924-Instruct.

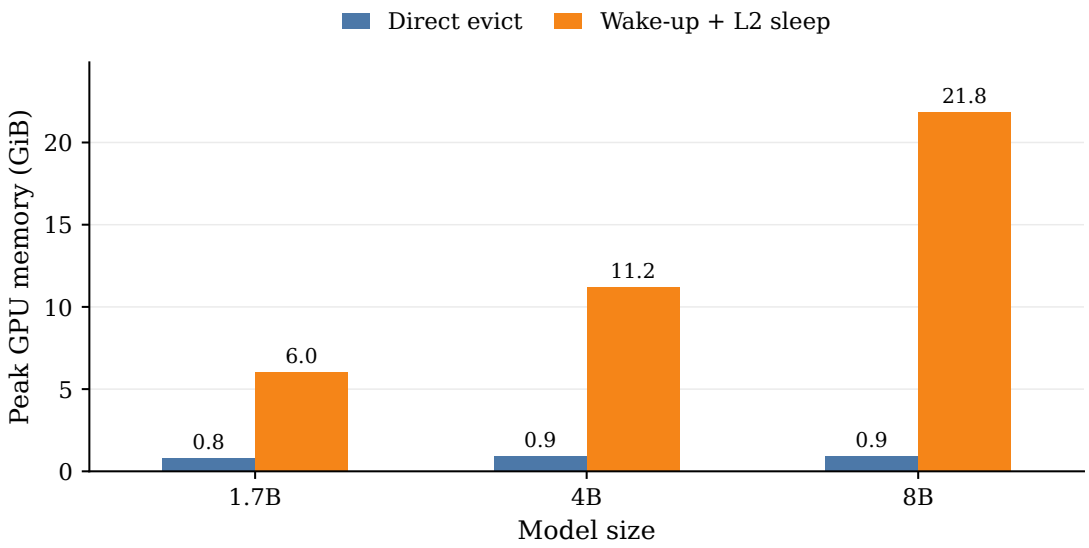

Figure A7: Peak GPU memory during eviction of Qwen-3 models. Direct eviction keeps the model in the sleeping state, whereas the wake-up followed by L2 sleep temporarily requires the full active GPU memory footprint.

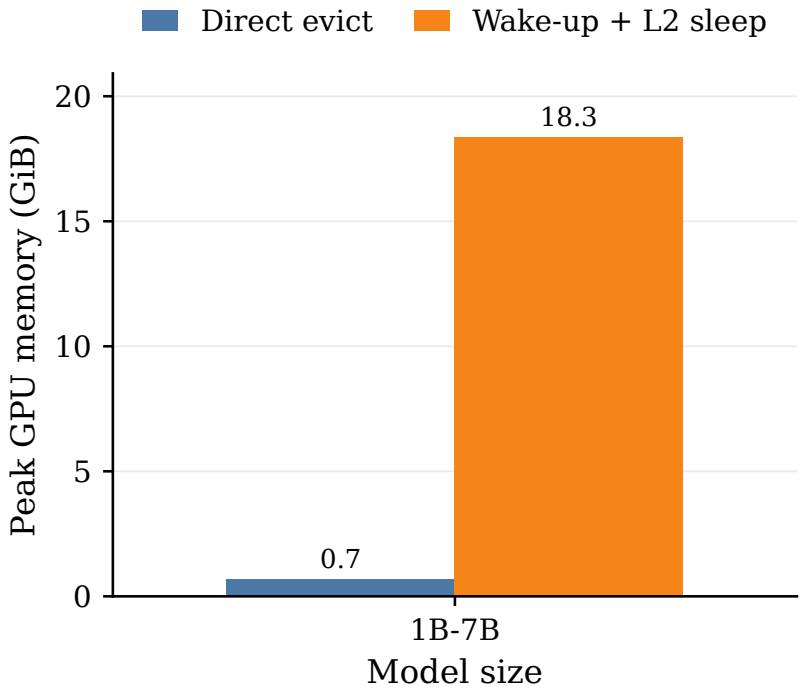

Figure A8: Peak GPU memory during eviction of OLMoE-1B-7B-0924-Instruct.